



IMPLEMENTING DEVOPS SOLUTIONS AND PRACTICES USING CISCO PLATFORM v1.0

Engineering Reliable Delivery Systems for Network Automation

**Independent Study Guide
Not an official Cisco publication**

Author / Instructor: _____
Version 1.0 | 2026

Implementing DevOps Solutions and Practices using Cisco Platforms (DevOps) v1.0

Professional Study Guide

Engineering Reliable Delivery Systems for Network Automation

Five-Day Instructor-Led Course

Version 1.0
2026

Author / Instructor: _____

Independent publication. Not an official Cisco publication.

Publication notice

This guide is independently authored and is not an official Cisco publication. Cisco and other product names may be trademarks of their respective owners. Product behavior varies by version and platform. Validate all commands and examples in an authorized environment. Network changes can affect production services; use approved test systems, change controls, credentials, and recovery procedures.

About this guide

This guide develops a controlled software-delivery system around network automation. It begins with familiar automation building blocks, then adds team ownership, Infrastructure as Code, reproducible application packaging, container platforms, CI/CD, security, observability, and operational feedback.

Who should read this guide

Network and infrastructure professionals who can automate technical work and now need to engineer, deliver, and operate that automation reliably as a team.

Prerequisite knowledge

Knowledge equivalent to DEVASC and DEVCOR is expected, including Python, Ansible, APIs, JSON and YAML, Git, and Linux command-line operation.

How to use this guide

Read modules in sequence. Use diagrams to establish the system model, examples to examine implementation choices, and knowledge checks to test engineering judgment. Advanced material may be revisited as reference.

Course engineering journey

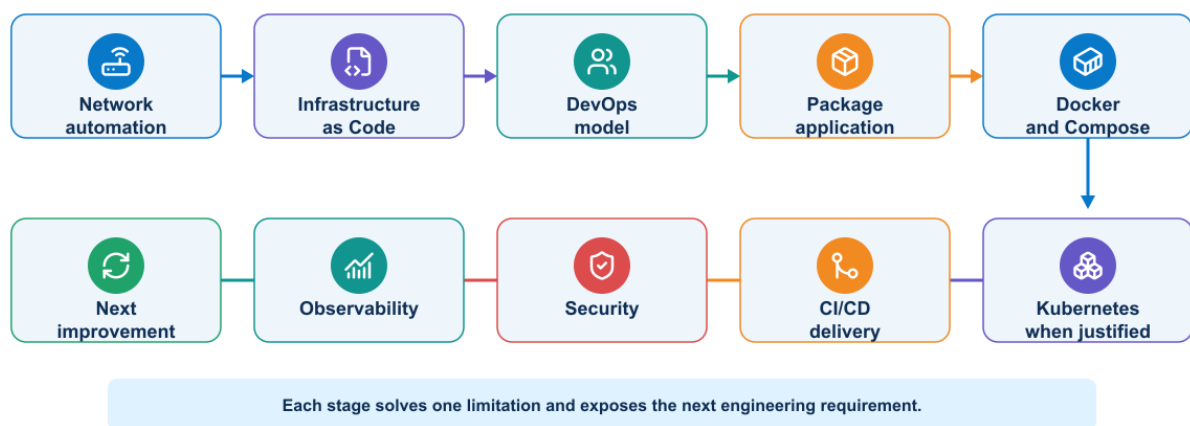


Figure I-1. Course engineering journey from network automation to continuous improvement

Five-day course map

Table I-1. Five-day learning sequence

Day	Engineering focus	Outcome
1	Automation review and DevOps model	Shared delivery and evidence model
2	Infrastructure as Code and packaging	Reproducible environment and image
3	Multitier operation and Kubernetes	Controlled application runtime
4	CI/CD and deployment validation	Qualified and promoted artifacts

Day	Engineering focus	Outcome
5	Security and observability	Trusted operation and feedback

Course reference architecture

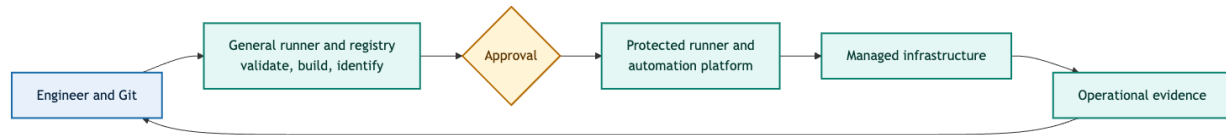


Figure I-2. Course reference delivery architecture and trust boundaries

Course reference scenario

A consistent change scenario provides concrete names and values for engineering discussions. It is an example, not a production authorization.

Table I-2. Course reference scenario

Context	Value
Site	campus-west
Target	distribution-01
Peer	routing-peer-01
Service	VLAN 120 USERS
Gateway	10.20.120.1/24
Routing	OSPF process 100, area 0
Change	CHG-2026-0042

Module overview

Table I-3. Module engineering questions

Module	Subject	Central question
0	Network Automation Review	Can we automate and verify the network outcome?
1	Introducing the DevOps Model	Can a team own, deliver, measure, and improve the system?
2	Infrastructure as Code and On-Demand Environments	Can we recreate and govern the environment?
3	Packaging and Operating Applications	Can we reproduce and operate the application runtime?
4	Continuous Integration Delivery and Deployment Validation	Can we automate the delivery process safely?
5	Security and Observability	Can we trust the system and explain what happened?

Table of contents

Publication notice	3
About this guide	4
Who should read this guide	4
Prerequisite knowledge	4
How to use this guide	4
Course engineering journey	4
Five-day course map	4
Course reference architecture	5
Course reference scenario	5
Module overview	5
Table of contents	6
List of figures	13
List of tables	15
0 Network Automation Review	16
Learning objectives	16
0.1 Network automation as a software system	17
0.2 Core review: foundation knowledge	18
0.2.1 Networking remains the operational foundation	18
0.2.2 Python provides application logic	18
0.2.3 Structured data forms a contract	18
0.2.4 Git records source and decisions	21
0.3 Optional refresher and reference: automation interfaces	22
0.3.1 CLI over SSH	23
0.3.2 REST APIs	23
0.3.3 NETCONF, RESTCONF, and YANG	23
0.3.4 Controllers and platform APIs	24
0.3.5 Flask as an automation service interface	24
0.3.6 Model-driven telemetry review	24
0.4 Core review: Ansible, orchestration, and tool ownership	26
0.4.1 Example Ansible playbook	26
0.5 Source of truth, intent, and state	27
0.5.1 Intent events and downstream consumers	28
0.6 Safety and reliability fundamentals	28
0.7 Testing network automation	29
0.8 Limits of individually operated automation	29
0.9 Readiness check	30
0.10 Summary	30
1 Introducing the DevOps Model	31

Learning objectives	31
1.1 From ad hoc automation to DevOps.....	32
1.2 CALMS applied to software delivery.....	32
1.2.1 Culture: shared responsibility for the outcome	33
1.2.2 Automation: make the safe path repeatable	33
1.2.3 Lean: improve flow and reduce batch risk.....	33
1.2.4 Measurement: use evidence to guide improvement	34
1.2.5 Sharing: make knowledge part of the system	34
1.2.6 How the CALMS dimensions reinforce one another	34
1.2.7 CALMS assessment questions	34
1.3 Three delivery models	34
1.4 Complete DevOps lifecycle	35
1.4.1 Plan: define the problem and expected outcome	36
1.4.2 Design: decide how the change fits the system	36
1.4.3 Develop: implement a small, reviewable change	36
1.4.4 Integrate: combine work and obtain early feedback.....	36
1.4.5 Build: create an identifiable artifact	36
1.4.6 Test: build confidence at several boundaries	37
1.4.7 Release: make a version eligible for deployment.....	37
1.4.8 Deploy: change the target environment safely	37
1.4.9 Operate: keep the service dependable	37
1.4.10 Observe: compare actual behavior with expectations.....	37
1.4.11 Learn and improve: close the loop	38
1.4.12 Retire: remove software and access deliberately	38
1.4.13 Gates, evidence, and promotion	38
1.4.14 Feedback loops at different speeds	39
1.4.15 Applied example: from commit to operating automation service.....	39
1.5 A practical software delivery architecture	40
1.6 Continuous integration, delivery, and deployment.....	41
1.7 Value stream and constraints	41
1.7.1 DORA measures for network delivery	42
1.8 Applying DevOps controls to existing network intent.....	42
1.8.1 Example input consumed by an existing application.....	42
1.9 Where implementation depth belongs	44
1.10 Knowledge check	44
1.11 Summary	44
2 Infrastructure as Code and On-Demand Environments	45
Learning objectives	45
2.1 Three automation responsibilities	46
2.1.1 Practical tool-selection decisions	46

2.2 Infrastructure as Code	47
2.3 Desired state and orchestration.....	47
2.4 Terraform model	47
2.4.1 Network example	48
2.5 Terraform lifecycle	48
2.6 State	48
2.7 Drift	49
2.8 Ansible model	49
2.8.1 Inventory and variables	49
2.8.2 Playbooks, roles, and templates	50
2.9 Provisioning and configuration boundary	50
2.10 On-demand test environments	50
2.11 Network test environment options	51
2.11.1 Cisco Modeling Labs as an on-demand test target.....	52
2.11.2 The create–configure–verify–destroy contract	52
2.12 Network drift and compliance	52
2.13 Pipeline integration	53
2.13.1 Practical ownership boundary: Terraform hands off to Ansible.....	53
2.14 Secrets and access	53
2.15 Infrastructure testing	53
2.16 Network platform considerations	54
2.17 Knowledge check	54
2.18 Summary	54
3 Packaging and Operating Applications	55
Learning objectives	55
3.1 Container runtime fundamentals	56
3.1.1 The application consistency problem	56
3.1.2 Why teams containerize applications.....	56
3.1.3 Why containers help delivery	56
3.1.4 Containers and virtual machines.....	57
3.1.5 Linux foundations	57
3.1.6 Docker architecture	57
3.1.7 Image references and identity.....	58
3.1.8 Image layers and cache	58
3.1.9 Container lifecycle	59
3.1.10 Configuration and secrets	59
3.1.11 Container storage.....	59
3.1.12 Container networking	59
3.1.13 Runtime inputs for network jobs.....	60
3.1.14 Persistent and ephemeral network data.....	61

3.1.15 Device connection behavior inside containers	61
3.1.16 Network automation container lifecycle.....	61
3.1.17 Resource and health considerations	61
3.1.18 Container tooling workflow	62
3.2 Secure image packaging	62
3.2.1 Application image contents	62
3.2.2 Dockerfile responsibilities.....	63
3.2.3 Base-image selection	64
3.2.4 Build context and <code>.dockerignore</code>	64
3.2.5 Dependency control	64
3.2.6 Layer design.....	64
3.2.7 Multistage builds	64
3.2.8 Runtime user and privileges.....	65
3.2.9 Secrets during builds.....	65
3.2.10 Image metadata and traceability	65
3.2.11 Testing an image.....	66
3.2.12 Image signing and provenance	67
3.2.13 Image supply-chain evidence.....	67
3.2.14 Compromised dependency scenario.....	67
3.2.15 Registries and retention	67
3.2.16 Example network automation packaging pattern	68
3.3 Multitier application deployment	69
3.3.1 Multitier service architecture	69
3.3.2 Service contracts.....	70
3.3.3 Docker network model	71
3.3.4 Docker Compose model.....	71
3.3.5 Configuration separation	73
3.3.6 Persistent state	73
3.3.7 Startup order and readiness.....	73
3.3.8 Timeouts and retries	73
3.3.9 Reverse proxy role	73
3.3.10 Worker safety and concurrency	74
3.3.11 Mock and real targets.....	74
3.3.12 Health model	74
3.3.13 Logging across services.....	75
3.3.14 Failure analysis	75
3.4 Kubernetes orchestration and multidata-center operation.....	76
3.4.1 Platform architecture	76
3.4.2 Should the team use Kubernetes?	77
3.4.3 Three valid platform architectures	77

3.4.4 Kubernetes worker-to-device security	78
3.4.5 Kubernetes model	79
3.4.6 Cluster architecture	79
3.4.7 Core objects	80
3.4.8 Manifests and API use	81
3.4.9 Scheduling and resources.....	81
3.4.10 Probes.....	81
3.4.11 Scaling and self-healing	82
3.4.12 Rolling updates	82
3.4.13 Advanced deployment patterns.....	83
3.4.14 Kubernetes networking	83
3.4.15 RBAC and service accounts	83
3.4.16 Configuration and secrets	83
3.4.17 Monitoring and logging.....	84
3.4.18 Troubleshooting workflow	84
3.4.19 Multiple data-center deployments	84
3.5 Knowledge check	85
3.5.1 Container runtime fundamentals	85
3.5.2 Secure image packaging.....	85
3.5.3 Multitier deployment	85
3.5.4 Kubernetes orchestration	85
3.6 Summary	85
4 Continuous Integration Delivery and Deployment Validation	87
Learning objectives	87
4.1 CI/CD delivery flow	88
4.1.1 Software delivery pipeline	88
4.1.2 Pipeline objectives	88
4.1.3 GitLab pipeline model	88
4.1.4 Pipeline design.....	89
4.1.5 Validation and testing layers	90
4.1.6 Build once and promote	92
4.1.7 Artifacts and caches.....	92
4.1.8 Runner design.....	92
4.1.9 Variables and secrets.....	95
4.1.10 Pipeline rules.....	95
4.1.11 Environments and promotion	95
4.1.12 Pipeline efficiency	95
4.1.13 Pipeline failure categories	96
4.1.14 Illustrative <code>.gitlab-ci.yml</code>	97
4.1.15 Explanation of the pipeline blocks.....	99

4.1.16 Why each gate exists	99
4.1.17 Kubernetes delivery integration.....	100
4.2 Deployment validation and recovery	102
4.2.1 Three forms of state in an automation application	102
4.2.2 Network change state machine.....	102
4.2.3 Evidence chain.....	103
4.2.4 Build validation	103
4.2.5 Infrastructure validation.....	103
4.2.6 Pre-deployment health checks.....	104
4.2.7 Configuration generation and diff	104
4.2.8 Deployment interfaces and transactions	105
4.2.9 Scoped deployment controls.....	105
4.2.10 Network post-checks.....	106
4.2.11 pyATS and Genie validation.....	107
4.2.12 Applied failure scenario: configuration accepted, routing service fails.....	107
4.2.13 Deployment strategies	108
4.2.14 Post-deployment validation	109
4.2.15 Smoke, integration, and acceptance checks.....	109
4.2.16 Idempotence and repeatability	109
4.2.17 Failure handling.....	109
4.2.18 Rollback and remediation.....	110
4.2.19 Improved deployment flow	111
4.3 Knowledge check	111
4.3.1 CI/CD delivery flow.....	111
4.3.2 Deployment validation and recovery	111
4.4 Summary	111
5 Security and Observability	113
Learning objectives	113
5.1 Security across the delivery system	114
5.1.1 DevOps trust boundaries	114
5.1.2 Security throughout delivery.....	114
5.1.3 Threat modelling a NetDevOps workflow.....	115
5.1.4 Secrets management.....	115
5.1.5 Pipeline security	116
5.1.6 Compromised-runner scenario.....	118
5.1.7 Secure protocol use	119
5.1.8 Application security	119
5.1.9 Container and orchestrator security.....	120
5.1.10 Advanced architecture reference	120
5.2 Observability and stability engineering	122

5.2.1 Monitoring, observability, and telemetry.....	122
5.2.2 Feedback architecture requirements.....	122
5.2.3 Monitoring and observability	123
5.2.4 Operational signals	123
5.2.5 Network data collection methods	124
5.2.6 Health checks.....	125
5.2.7 Service objectives	125
5.2.8 Metrics collection architecture.....	125
5.2.9 Log collection architecture	126
5.2.10 Dashboards.....	126
5.2.11 Alert design	126
5.2.12 Webhook notifications	127
5.2.13 Application instrumentation	127
5.2.14 Change-aware observability and correlation.....	127
5.2.15 Telemetry quality.....	129
5.2.16 Stability engineering.....	130
5.2.17 Chaos engineering	130
5.3 Knowledge check	130
5.3.1 Security across the delivery system.....	130
5.3.2 Observability and stability engineering.....	130
5.4 Summary	131

List of figures

Figure I-1. Course engineering journey from network automation to continuous improvement.....	4
Figure I-2. Course reference delivery architecture and trust boundaries	5
Figure 0-1. Icon-led flow from intent and source of truth through logic, validation, and evidence	17
Figure 0-2. YAML, JSON, and XML normalized into one Python dictionary through parsing and validation.....	19
Figure 0-3. Comparison of SSH CLI, REST APIs, NETCONF with YANG, and controller APIs.....	22
Figure 0-4. Model-driven telemetry flow from devices through subscriptions, collection, normalization, and observation	25
Figure 0-5. Intended, configured, and operational state shown as three distinct views.....	28
Figure 1-1. Icon-led CALMS model showing Culture, Automation, Lean, Measurement, and Sharing as a continuous improvement loop	33
Figure 1-2. Visual comparison of manual delivery, ad hoc automation, and a DevOps pipeline	35
Figure 1-3. Icon-led DevOps lifecycle from planning through development, qualification, release, operation, and learning	36
Figure 1-4. Evidence chain connecting requirement, commit, pipeline, artifact, deployment, and runtime evidence.....	38
Figure 1-5. Feedback loops grouped by seconds, minutes, hours, and weeks	39
Figure 1-6. Delivery architecture separating unprivileged validation from protected deployment.....	40
Figure 1-7. Value stream showing work time, waiting time, and the delivery constraint.....	41
Figure 2-1. Tool ownership decision flow for Terraform, Ansible, and Python.....	46
Figure 2-2. Terraform desired-state lifecycle from authoring through planning, application, and drift detection.....	48
Figure 2-3. Drift collection, classification, ownership resolution, and controlled reconciliation	49
Figure 2-4. Ownership and data handoff between Terraform provisioning and Ansible configuration	50
Figure 2-5. On-demand environment lifecycle with failure-safe evidence and cleanup	51
Figure 3-1. Comparison of container and virtual-machine runtime boundaries	57
Figure 3-2. Docker client, Engine API, daemon, BuildKit, container runtime, registry, networks, and storage	57
Figure 3-3. Lifecycle from Docker build inputs through immutable image layers and a disposable runtime container	58
Figure 3-4. Container image layers arranged by expected rate of change and cache reuse	59
Figure 3-5. Separation of the application service network from the protected device-management network.....	60
Figure 3-6. Application content inside an automation image and configuration, credentials, and state outside it.....	60
Figure 3-7. Layered view of the responsibilities expressed by common Dockerfile instructions	63
Figure 3-8. Builder and final runtime stages separated by an explicit artifact boundary.....	65
Figure 3-9. Evidence lineage from source and dependencies to the deployed automation image and post-change results.....	66
Figure 3-10. Secure image supply chain with build evidence and a prohibited secret path	67
Figure 3-11. Docker Compose architecture for the network automation API, worker, supporting services, and network targets	70

Figure 3-12. Service contract connecting producer, consumer, reliability, and security responsibilities	70
Figure 3-13. Approved job sequence across API, queue, protected worker, secret service, and device	70
Figure 3-14. Readiness and failure behavior across the automation service dependency chain	74
Figure 3-15. Failure boundaries across request, execution, and dependency paths in a Compose application	75
Figure 3-16. Kubernetes-hosted network automation platform and restricted management paths	76
Figure 3-17. Decision flow for choosing Kubernetes, Compose, or a protected runner for network automation	77
Figure 3-18. Comparison of a protected runner, Docker Compose platform, and Kubernetes automation platform	78
Figure 3-19. Kubernetes security controls from validated queue input through an isolated worker to explicitly authorized devices	79
Figure 3-20. Kubernetes control plane, worker nodes, API clients, state store, and cluster network .	79
Figure 3-21. Distinct startup, readiness, and liveness probe decisions	81
Figure 4-1. Progressive commit, qualification, release, and deployment gates	90
Figure 4-2. Test-confidence model from static checks through unit, integration, and acceptance tests	90
Figure 4-3. Authoritative pipeline artifact compared with a disposable cache	92
Figure 4-4. Separation of the general validation runner from the protected network runner and management zone	94
Figure 4-5. Separation between the platform delivery pipeline and network job pipeline	101
Figure 4-6. Relationship among intended, configured, and operational network state	102
Figure 4-7. Network change states including unknown and recovery paths	103
Figure 4-8. Nested configuration, device, domain, and environment blast-radius controls	106
Figure 4-9. Comparison of traffic-oriented software deployment patterns and external-state automation controls	108
Figure 4-10. Decision tree for investigation, rollback, or forward remediation	110
Figure 5-1. NetDevOps trust boundaries through protected execution and independent audit	114
Figure 5-2. Layered security responsibilities from design and build through operation and retirement	114
Figure 5-3. Compromised runner incident-response cycle from containment through independent verification	118
Figure 5-4. Relationship between telemetry, monitoring, observability, context, and action	122
Figure 5-5. Observability architecture joining network, application, and pipeline signals	123
Figure 5-6. Operational signals grouped by behavior, causality, and network state	123
Figure 5-7. Change-aware feedback from commit and image digest through deployment, network behavior, telemetry, investigation, and improvement	128
Figure 5-8. Correlation model joining delivery events, device events, service signals, and evidence	128

List of tables

Table I-1. Five-day learning sequence.....	4
Table I-2. Course reference scenario	5
Table I-3. Module engineering questions	5
Table 0-1. Network Automation Component Responsibilities	17
Table 0-2. Intended Configured and Operational State Comparison	28
Table 0-3. Network Automation Test Types and Evidence	29
Table 1-1. Ad Hoc Automation and DevOps Delivery Comparison	32
Table 1-2. CALMS Dimensions in Software Delivery	33
Table 1-3. CALMS Assessment Questions.....	34
Table 1-4. Manual Ad Hoc and DevOps Delivery Models.....	35
Table 1-5. Testing Boundaries and Required Evidence	37
Table 1-6. Delivery Gates Decisions and Evidence	38
Table 1-7. DORA Measures for Network Delivery	42
Table 1-8. Intent Validation and Delivery Stages	43
Table 2-1. Terraform Ansible and Python Responsibility Boundaries.....	46
Table 2-2. Network Test Environment Options	51
Table 3-1. Containerization Benefits and Operational Tradeoffs	56
Table 3-2. Automation Image Contents and Runtime Inputs	62
Table 3-3. Packaging Failure Modes and Controls	68
Table 3-4. Compose Application Failure Boundaries and Controls	75
Table 3-5. Container Platform Selection Criteria	77
Table 3-6. Protected Runner Compose and Kubernetes Comparison	78
Table 4-1. GitLab Components and Network Automation Responsibilities	88
Table 4-2. Test Environments Capabilities and Limitations	91
Table 4-3. GitLab Runner Executor Tradeoffs	93
Table 4-4. Pipeline Purposes Triggers and Protected Resources	95
Table 4-5. Pipeline Gates and Their Evidence	99
Table 4-6. Intended Configured and Operational State Evidence	102
Table 4-7. Routing Service Predeployment Checks	104
Table 4-8. Deployment Strategies and Network Automation Controls.....	108
Table 5-1. Delivery Assets Threats and Controls	115
Table 5-2. Security Test Types and Evidence Boundaries	117
Table 5-3. Compromised Runner Response Actions and Evidence	118
Table 5-4. Telemetry Monitoring and Observability Comparison	122
Table 5-5. Metrics Collection Components and Responsibilities	125
Table 5-6. Pipeline Audit Events and Required Context	129

MODULE 0

0 Network Automation Review

Central engineering question

Can we automate and verify the network outcome?

INTENT → LOGIC → DEVICE → EVIDENCE

Learning objectives

- Describe the main components of a network automation solution.
- Explain how Python, Ansible, APIs, models, inventory, and source control fit together.
- Parse equivalent YAML, JSON, and XML data into a normalized Python dictionary.
- Recognize appropriate uses and safety considerations for Netmiko, `ncclient`, `requests`, and Flask.
- Explain Ansible inventory, variables, playbooks, modules, templates, idempotence, and execution behavior.
- Explain model-driven telemetry subscriptions, collection, normalization, storage, and correlation.

Estimated theory time	2 hours
Lab connection	Apply the module controls to the cumulative application and delivery system
Core classroom topics	Concepts, decisions, workflows, and required controls
Advanced reference	Detailed implementation and platform considerations

0.1 Network automation as a software system

Network automation is sometimes introduced as a faster way to execute commands. That description is incomplete. A useful automation solution is a software system that interprets intent, obtains trusted data, communicates with external systems, changes or observes state, handles failure, and produces evidence.

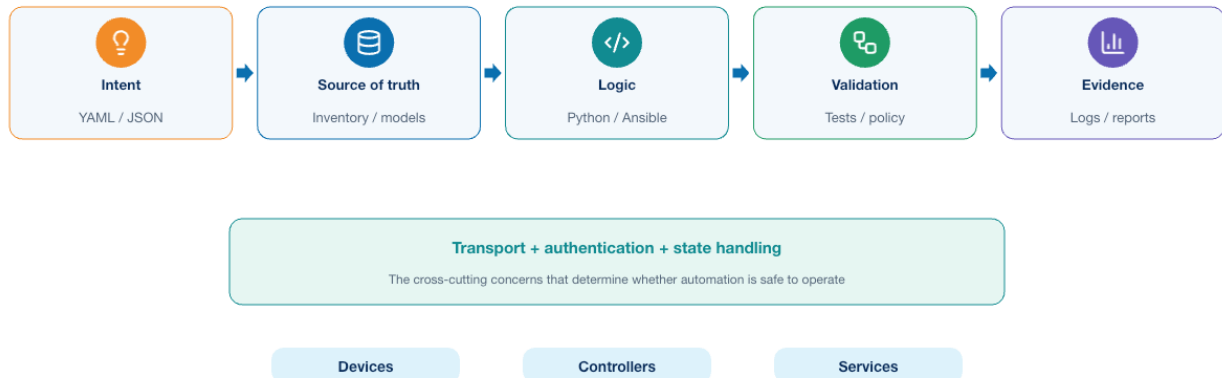


Figure 0-1. Icon-led flow from intent and source of truth through logic, validation, and evidence

A typical solution contains several responsibilities. The transport is only one stage; inventory, validation, policy, verification, and evidence determine whether the software can be trusted as an operational system.

Table 0-1. Network Automation Component Responsibilities

Responsibility	Typical implementation	Engineering question
Intent or request	YAML, JSON, form, ticket, source-of-truth record	What outcome is requested, and who authorized it?
Inventory	YAML, database, NetBox, controller inventory	Which objects are valid targets?
Validation	Python model, JSON Schema, policy function	Is the input complete, correctly typed, and permitted?
Transformation	Python, Jinja2, Ansible filters	How is platform-neutral data converted into an executable request?
Transport	SSH, NETCONF, RESTCONF, controller API	How does the application communicate with the target?
Orchestration	Python workflow, Ansible playbook, job worker	In what order should actions occur, and what happens on failure?
Verification	API read, structured parser, pyATS, service probe	Did the intended operational outcome occur?
Evidence	Structured logs, reports, diffs, metrics	Can another engineer explain what happened later?

The implementation may be a small command-line program or a multitier service. The responsibilities still exist. When they are hidden inside one script, they become harder to test and govern independently.

0.2 Core review: foundation knowledge

Reliable delivery depends on several disciplines working together. Networking knowledge defines the intended behavior, programming and data models express it, and version control preserves both the implementation and the decisions behind it.

0.2.1 Networking remains the operational foundation

Automation does not remove the need to understand the system being automated. An engineer must still reason about addressing, routing, switching, DNS, transport protocols, management reachability, security policy, and failure domains.

For example, a Python HTTPS request can fail because of name resolution, routing, a firewall, TLS identity, authentication, rate limiting, or an application error. Repeating the request does not address all of those causes. Effective automation separates transport failure, authorization failure, invalid input, remote-system rejection, timeout, and uncertain completion.

The same principle applies after a change. A successful response means that an endpoint accepted or processed a request according to its contract. It does not prove that users can reach the service, that a routing protocol converged, or that an unrelated policy was left untouched.

0.2.2 Python provides application logic

At associate level, learners should be comfortable with variables, collections, conditions, loops, functions, exceptions, modules, packages, file handling, and virtual environments. At this stage, Python should be treated as application code rather than as a collection of copied snippets.

A maintainable automation program separates concerns. Input parsing should not open device sessions. Business policy should not be buried in a Jinja2 template. Reporting should not decide which targets are authorized. Small functions and modules make behavior easier to test and change.

The following function illustrates the expected style of thinking:

```
def select_targets(inventory: dict, site: str, limit: int = 10) -> list[dict]:
    """Return enabled devices for one known site within an explicit limit."""
    if site not in inventory:
        raise ValueError(f"Unknown site: {site}")

    targets = [device for device in inventory[site] if device.get("enabled")]
    if not targets:
        raise ValueError(f"No enabled targets for site: {site}")
    if len(targets) > limit:
        raise ValueError(f"Target count {len(targets)} exceeds limit {limit}")
    return targets
```

The important features are not the Python syntax. The function has one responsibility, validates assumptions, fails explicitly, and enforces a scope limit. Unit tests can exercise unknown sites, empty groups, disabled devices, and an excessive target count without contacting a network.

0.2.3 Structured data forms a contract

JSON, YAML, XML, and CSV represent data; they do not automatically make the data valid. Parsing answers whether a document is syntactically readable. Schema validation answers whether required fields, types, ranges, and structures follow a contract. Policy validation answers whether the requested values are permitted in a particular organization or environment.

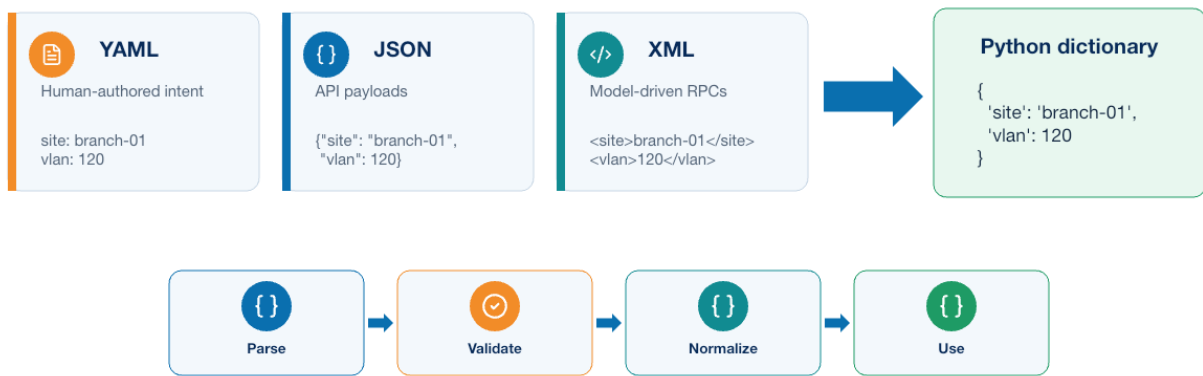


Figure 0-2. YAML, JSON, and XML normalized into one Python dictionary through parsing and validation

Learners should recall the usual roles:

- JSON is common in REST API requests and responses.
- YAML is convenient for human-reviewed configuration and inventory.
- XML is important in NETCONF and XML-encoded YANG data.
- CSV is useful for simple tabular exchange but expresses nested relationships poorly.

Data should be normalized before it reaches templates or API clients. Addresses, prefixes, interface names, booleans, enumerated values, and identifiers should have one internal representation. This prevents each downstream component from interpreting the same input differently.

One inventory represented in YAML, JSON, XML, and Python

The examples below describe the same two devices. Seeing the equivalent structures helps when an application reads one format, uses Python dictionaries internally, and sends another format to an API.

YAML emphasizes readability and is often used for inventory or reviewed intent:

```
site: campus-west
devices:
  - name: distribution-01
    address: 192.0.2.11
    platform: network-os-a
    enabled: true
    tags: [distribution, production]
  - name: access-01
    address: 192.0.2.21
    platform: network-os-b
    enabled: false
    tags: [access, maintenance]
```

JSON uses objects and arrays and is common in REST APIs:

```
{
  "site": "campus-west",
  "devices": [
    {
      "name": "distribution-01",
      "address": "192.0.2.11",
      "platform": "network-os-a",
      "enabled": true,
      "tags": ["distribution", "production"]
    },
    {
      "name": "access-01",
      "address": "192.0.2.21",
      "platform": "network-os-b",
      "enabled": false,
      "tags": ["access", "maintenance"]
    }
  ]
}
```

XML expresses the same hierarchy through elements and attributes:

```
<?xml version="1.0" encoding="UTF-8"?>
<inventory site="campus-west">
  <device enabled="true">
    <name>distribution-01</name>
    <address>192.0.2.11</address>
    <platform>network-os-a</platform>
    <tags>
      <tag>distribution</tag>
      <tag>production</tag>
    </tags>
  </device>
  <device enabled="false">
    <name>access-01</name>
    <address>192.0.2.21</address>
    <platform>network-os-b</platform>
    <tags>
      <tag>access</tag>
      <tag>maintenance</tag>
    </tags>
  </device>
</inventory>
```

The equivalent Python dictionary is:

```
inventory = {
    "site": "campus-west",
    "devices": [
        {
            "name": "distribution-01",
            "address": "192.0.2.11",
            "platform": "network-os-a",
            "enabled": True,
            "tags": ["distribution", "production"],
        },
        {
            "name": "access-01",
            "address": "192.0.2.21",
            "platform": "network-os-b",
            "enabled": False,
            "tags": ["access", "maintenance"],
        },
    ],
}
```

Parsing and normalizing the formats

Python uses a different parser for each external representation:

```
import json, xml.etree.ElementTree as ET
import yaml

yaml_data = yaml.safe_load(yaml_text)
json_data = json.loads(json_text)
xml_root = ET.fromstring(xml_text)
```

Parsing only creates language objects. The application should then validate the expected schema and normalize values into one internal dictionary—for example, lowercase site names, canonical IP addresses, explicit booleans, and a consistent device-list structure. Use `yaml.safe_load()` for reviewed YAML and a hardened XML parser when XML is untrusted. Tests should cover missing fields, unexpected fields, invalid addresses, duplicates, and boundary values.

0.2.4 Git records source and decisions

Git stores versions of source code and supporting definitions. Learners should be able to create a branch, inspect a diff, stage deliberate changes, commit them with a useful message, resolve straightforward conflicts, and participate in review.

A basic local workflow uses a small set of commands:

```
git clone <repository-url>
git status
git switch -c feature/short-description
git diff
git add <file>
git diff --staged
git commit -m "Describe the reason for the change"
git fetch origin
git push -u origin feature/short-description
```

`git status` shows the current branch and file states. `git diff` reviews unstaged work; `git diff --staged` reviews exactly what the next commit will contain. `git add` selects changes for that commit, and `git commit` records the staged snapshot locally. `git fetch` updates knowledge of the remote without modifying the working branch. `git push` publishes the feature branch for review.

Useful inspection and correction commands include:

```
git log --oneline --graph --decorate -10
git show <commit>
git restore <file>          # discard an unstaged local edit
git restore --staged <file> # unstage without deleting the edit
git branch --show-current
```

Before restoring or discarding anything, inspect `git status` and `git diff`. Pulling, rebasing, merging, resolving conflicts, or rewriting published history should follow the team's collaboration policy.

Four states are especially important when diagnosing a delivery problem. The working tree contains current local files; the index contains the exact changes selected for the next commit; a commit is an immutable snapshot with parent history and author metadata; and a remote-tracking reference records the last fetched view of a remote branch. `git status` and `git diff` answer different questions depending on which two states are compared. A clean working tree proves only that local files match the checked-out commit; it does not prove that the branch contains the latest reviewed change or that the commit was released.

Branches isolate proposed work, tags can identify release points, and merge requests add review and automated evidence around integration. For an automation repository, the review unit should be small enough that another engineer can understand the intent, generated difference, test effect, and recovery consequence. Generated configuration may be retained as pipeline evidence, but reviewed intent and source remain authoritative.

A network automation repository can contain:

- Python source and tests
- Ansible playbooks, roles, inventory structure, and collection requirements
- Schemas, templates, and safe example data
- Dependency declarations
- Documentation and runbooks
- Pipeline, container, infrastructure, and deployment definitions

It should not contain live credentials, private keys, tokens, uncontrolled state files, or sensitive device output. Git history is durable: deleting a secret in a later commit does not remove the earlier exposure.

Git provides useful history only when changes are committed with meaningful context, reviewed where appropriate, and tied to the inputs and results of an automation run.

0.3 Optional refresher and reference: automation interfaces

The available interfaces overlap, but they expose different control and failure semantics. The following visual provides a quick comparison before the detailed review.

INTERFACE	BEST FIT	ENGINEERING CONCERN
SSH / CLI	Broad coverage	Parsing and command context
REST API	Service integration	Schema, status codes, pagination
NETCONF + YANG	Model-driven configuration	Capabilities, datastore, transaction
Controller API	Intent and domain abstraction	Platform workflow and asynchronous state

Figure 0-3. Comparison of SSH CLI, REST APIs, NETCONF with YANG, and controller APIs

An automation application reaches infrastructure through an interface with its own data model, failure modes, and security properties. Choosing an interface therefore affects not only how code is written, but also how safely the resulting change can be validated and repeated.

0.3.1 CLI over SSH

SSH CLI automation remains useful when a required function lacks a suitable structured interface. Libraries such as Netmiko or Scrapli handle prompts, command timing, and platform behavior more reliably than a general-purpose interactive shell implementation.

CLI output is intended primarily for people and may vary by platform, release, privilege, width, localization, or command form. Structured parsing with TextFSM or Genie is preferable to fragile `split()` logic, but the parser and its expected data shape still require tests. Configuration workflows also need target verification, configuration preview, timeouts, failure classification, post-checks, and a recovery plan.

Short Netmiko example

A short read-only call is sufficient for this review:

```
from netmiko import ConnectHandler

with ConnectHandler(**device) as connection:
    output = connection.send_command("show interfaces", use_textfsm=True)
```

The device dictionary, command, parser availability, and returned data are platform-dependent. Production code still needs credentials from a protected source, explicit timeouts, authentication and transport error handling, target verification, and tests built from sanitized output fixtures.

0.3.2 REST APIs

Learners should recognize the parts of an HTTP request: method, URL, headers, authentication, query parameters, and optional body. They should interpret status codes and parse the response only after checking that its status and media type match expectations.

Reliable API clients address more than the successful `200` path. They use TLS verification, explicit timeouts, pagination, rate-limit handling, bounded retries, and clear error classification. They understand whether an operation is idempotent and whether a `202 Accepted` response represents completion or only the creation of an asynchronous job.

Retry behavior deserves particular care. A read request may be safe to retry after a transient connection failure. Repeating a create or change request after an uncertain timeout can duplicate work. The client may first need to query a request identifier or rediscover actual state.

Short `requests` example

```
import requests

response = requests.get(url, headers=headers, timeout=(5, 20), verify=ca_bundle)
response.raise_for_status()
data = response.json()
```

The client should verify TLS identity, check the returned media type and schema, handle pagination and rate limits, and classify failures. Automatic retry is normally safer for reads than for a change request whose earlier outcome is uncertain.

0.3.3 NETCONF, RESTCONF, and YANG

YANG defines structured configuration and operational data. NETCONF exchanges RPC messages and can provide datastores and transaction capabilities. RESTCONF exposes YANG-modeled resources through HTTP. Actual model and capability support varies across platforms and software releases.

A model-driven workflow should discover capabilities, identify the correct schema path, validate the payload, select the appropriate datastore or HTTP method, interpret protocol errors, and read the resulting state. A syntactically correct XML or JSON payload can still violate model constraints or business policy.

Model-driven does not mean risk-free. The application still needs authorization, target control, transaction handling, diff or preview, post-change validation, and evidence.

Short `ncclient` example

```
from ncclient import manager

with manager.connect(**connection_parameters) as session:
    reply = session.get(filter=("subtree", subtree_filter))
```

Connection parameters must enable host-key verification and define a timeout. Model namespaces, filters, datastore behavior, locking, confirmed commit, and error handling depend on the target's advertised capabilities. A successful RPC confirms protocol acceptance, not the final network outcome.

0.3.4 Controllers and platform APIs

Controllers provide higher-level inventory, policy, assurance, topology, or service abstractions. Their API often has a different consistency and task model from a device API. A request may create a background task, and several reads may be required before the final state becomes visible.

Before integrating with a controller, determine:

- Which system owns the object being changed
- How identity and authorization are applied
- Whether operations are synchronous or asynchronous
- How pagination and rate limits work
- What constitutes success, partial success, and failure
- How state is verified independently
- How changes made outside the controller create drift

0.3.5 Flask as an automation service interface

Flask can place an HTTP interface around existing Python logic. A small service may expose a liveness endpoint and accept a validated job request, but Flask does not automatically provide production authentication, authorization, rate limiting, durable job processing, TLS termination, or observability.

A production design should define request schemas, reject unsupported fields, authorize the requested target and operation, return a correlation or job identifier, move slow work to a durable worker, and provide a status resource. Run the application behind an appropriate production server rather than Flask's development server.

0.3.6 Model-driven telemetry review

Model-driven telemetry publishes structured operational data identified by model paths. Unlike periodic CLI scraping, the collector does not need to reconstruct meaning from human-formatted text. Unlike traditional polling, a subscription can stream updates at a requested interval or when state changes, subject to platform capability.



Figure 0-4. Model-driven telemetry flow from devices through subscriptions, collection, normalization, and observation

A typical path is:

Device or controller → subscription transport → collector → validation and enrichment → time-series or event storage → dashboard and alerting

Common transport and subscription concepts include:

- **gNMI:** a gRPC-based interface using paths and typed values, commonly secured with TLS.
- **NETCONF notifications:** event notifications delivered over an established NETCONF session.
- **Dial-in:** the collector connects to the device and establishes the subscription.
- **Dial-out:** the device initiates a stream toward a configured receiver.
- **Periodic or sample:** values are sent at a defined interval.
- **On-change:** updates are sent when a subscribed value changes, when supported.
- **Once:** a finite snapshot is returned for the selected paths.

Model-driven telemetry and OpenTelemetry solve related but different problems. Model-driven network telemetry represents device or controller state through network data models. OpenTelemetry standardizes application traces, metrics, logs, and context propagation. An automation platform may use both: gNMI for interface and protocol state, and OpenTelemetry for the API request, queue delay, worker span, and database call.

The collector is not merely a forwarding process. It must authenticate endpoints, validate certificates, negotiate supported encodings, track subscription state, add stable device and site metadata, normalize timestamps and units, manage backpressure, expose its own health, and prevent unbounded label cardinality.

After decoding a telemetry message, the collector should validate required fields, reject unknown devices or paths, normalize timestamps and units, and add stable site and interface metadata. Keep transport-specific response objects behind an adapter so that collection code and business rules can be tested independently.

Counter interpretation requires more than storing values. Octet and packet counters normally increase monotonically and may reset after reboot or process restart. A collector calculates rates from successive samples only when timestamps are ordered and the counter has not reset or wrapped. Missing updates, duplicated timestamps, clock error, subscription loss, and collector backlog must be visible; otherwise a flat graph can be mistaken for a healthy interface when data collection has actually failed.

Telemetry is most useful when network observations can be correlated with collection time, target identity, software version, and the operation that preceded a change. This allows an engineer to distinguish an automation defect, a collector failure, and a genuine network-state change.

0.4 Core review: Ansible, orchestration, and tool ownership

Ansible provides inventories, variables, collections, modules, roles, handlers, conditions, and playbooks for describing ordered work across targets. Agentless operation is particularly familiar in network environments, although module behavior and platform support still depend on collection versions and device capabilities.

Learners should recall these principles:

- Use modules or resource modules instead of raw commands when they provide the required behavior.
- Keep inventory and environment values separate from reusable roles.
- Keep credentials outside committed inventory.
- Pin and test collection versions.
- Use `--limit`, groups, batching, and `serial` to control scope.
- Inspect check-mode or diff output where the module supports it reliably.
- Use handlers for dependent actions such as restarting a service.
- Design blocks and error handling so that failures remain visible.
- Verify operational results instead of relying only on an `ok` or `changed` count.

Idempotence means that repeated execution converges on the intended state without creating unnecessary additional changes. It does not mean that every task is automatically safe to retry. A playbook can contain a non-idempotent command, call an asynchronous API, or repeat an operation whose previous outcome is uncertain.

0.4.1 Example Ansible playbook

The following vendor-neutral pattern validates scope, collects a baseline, applies a role in small batches, and verifies the result. Concrete module names and returned data vary by collection and platform.

```
---
- name: Validate and deploy an approved network service
  hosts: managed_network
  gather_facts: false
  serial: 2
  max_fail_percentage: 0

  vars:
    approved_change_id: "{{ lookup('env', 'CHANGE_ID') }}"

  pre_tasks:
    - name: Require an approved change identifier
      ansible.builtin.assert:
        that:
          - approved_change_id is match('^CHG-[0-9]{4}-[0-9]{4}$')
          - inventory_hostname in approved_targets
        fail_msg: "Change identity or target authorization is invalid"

    - name: Collect a read-only operational baseline
      ansible.netcommon.cli_command:
        command: show interfaces
        register: interface_baseline
        changed_when: false
        no_log: false

  roles:
    - role: network_service

  post_tasks:
```

```

- name: Collect state after the role runs
  ansible.netcommon.cli_command:
    command: show interfaces
    register: interface_result
    changed_when: false

- name: Run the project acceptance validator
  ansible.builtin.command:
    argv:
      - python
      - -m
      - automation.validate_result
      - --host
      - "{{ inventory_hostname }}"
      - --change-id
      - "{{ approved_change_id }}"
  delegate_to: localhost
  changed_when: false

```

The `network_service` role would contain platform-aware, preferably idempotent resource modules. The playbook does not embed credentials, limits concurrent targets with `serial`, stops after a failure, distinguishes collection from change, and delegates acceptance logic to testable application code. In a real pipeline, baseline and post-check results should be sanitized and retained as artifacts. `no_log: false` is shown only because the illustrative read command is nonsensitive; tasks handling credentials or sensitive payloads require deliberate log protection.

0.5 Source of truth, intent, and state

A source of truth is the authoritative record for a defined class of data. It may contain device identity, site membership, addressing, connections, services, or policy. Authority must be explicit. If a spreadsheet, controller, inventory file, and live device can all overwrite the same value, the organization has several competing sources rather than one source of truth.

NetBox is a common example of a network source of truth. It models objects such as sites, devices, platforms, interfaces, prefixes, IP addresses, connections, tenants, and services through related records rather than unrelated inventory variables. Its value is not that it automatically makes every record correct. Its value is that ownership, validation, relationships, and API access can be defined around one authoritative model.

A useful source of truth has several properties:

- **Defined authority:** the team knows which data NetBox owns and which data belongs to another system.
- **Stable identity:** devices and interfaces have durable identifiers rather than being inferred from display text.
- **Validated relationships:** an address is associated with the interface and device that use it.
- **Controlled mutation:** permissions and workflow determine who or what may change intent.
- **Machine-readable access:** applications retrieve the same records through an API instead of maintaining private copies.
- **Change visibility:** object history shows when intent changed and by whom.

NetBox should not become a password database or an unfiltered copy of live configuration. Device credentials belong in a secrets system. Operational facts belong in telemetry or device-state collection. The source of truth records what the organization intends and the metadata required to resolve the correct target.

0.5.1 Intent events and downstream consumers

An authoritative object change can be more useful than a scheduled poll. An event can tell a downstream consumer that a completed piece of intent is ready for evaluation. For example, an address assignment becomes meaningful only when the address, interface, and device relationship is complete. Emitting an event at that boundary avoids consumers guessing whether several partially edited records form one request.

The event is a notification, not unquestioned authority to make a change. A consumer should retrieve the current object through the authenticated API, validate its type and relationships, record an immutable fingerprint, and reject stale, incomplete, duplicated, or out-of-scope events. This preserves the source of truth as the origin of intent while allowing each consumer to apply its own authorization and safety policy.

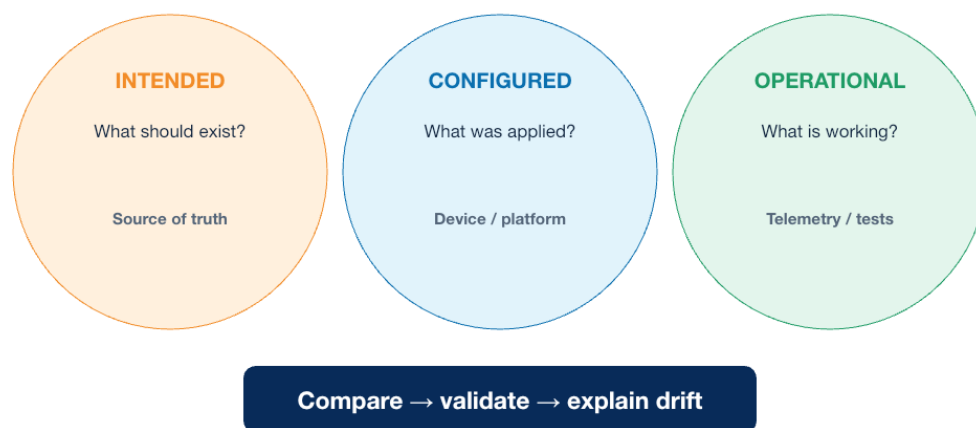


Figure 0-5. Intended, configured, and operational state shown as three distinct views

Three states should remain distinct:

Table 0-2. Intended Configured and Operational State Comparison

State	Question	Example evidence
Intended state	What should exist?	Reviewed data, approved policy, requested application version
Configured state	What has the managed system stored?	API read, configuration retrieval, controller object
Operational state	What is actually working now?	Reachability, protocol state, service health, telemetry

Configuration can match intent while operation remains unhealthy. Operational state can appear healthy temporarily while configuration has drifted from policy. Reliable verification selects evidence appropriate to the requested outcome.

0.6 Safety and reliability fundamentals

Network automation can apply the same error consistently across many targets, so scale increases both value and risk. The following controls should already be familiar:

- Validate input before opening a privileged connection.
- Resolve requested targets through an approved inventory.
- Display or preserve the selected target set.
- Begin with read-only collection and a known baseline.
- Generate and review the proposed effect where possible.
- Limit concurrency and batch size according to system capacity and blast radius.

- Use authentication appropriate to a workload rather than embedding personal credentials.
- Set connection and operation timeouts.
- Classify retryable, permanent, and uncertain failures differently.
- Verify configuration and service behavior after execution.
- Preserve evidence and provide a tested recovery method.

Concurrency is not merely a performance setting. Fifty simultaneous sessions may overload a management plane, controller, authentication service, or WAN link. Operations touching the same device or shared service may require locking even when the worker platform can execute them in parallel.

0.7 Testing network automation

Tests should be selected according to the boundary they can prove:

Table 0-3. Network Automation Test Types and Evidence

Test type	Suitable target	What it cannot prove alone
Unit test	Validation function, transformation, retry calculation	Live protocol or platform behavior
Schema and policy test	Input contract and organizational rules	Correct rendering or deployment
Fixture or parser test	Sanitized CLI/API response handling	Timing, scale, or current platform behavior
Mock integration test	Client authentication, error, pagination, task polling	Vendor implementation semantics
Virtual or sandbox test	Protocol, model, configuration, and topology interaction	Full production scale and hardware behavior
Production pre/post-check	Actual baseline and operational outcome	Elimination of all change risk

Tests become more expensive and realistic toward the bottom of the table. A sound strategy runs many fast offline tests and a smaller number of controlled system tests. A live production target should not be the first place a predictable parser, validation, or error-handling defect is discovered.

0.8 Limits of individually operated automation

The capabilities added around an existing script remove different dependencies on the original author's workstation or memory. Containerization alone is not the destination. The result becomes a product when a team can review, release, operate, diagnose, and improve it through a documented delivery system.

An individual automation script can be technically correct and still be difficult to operate safely as team software. Common symptoms include:

- The author is the only person who knows the required execution order.
- Dependencies are installed manually and vary between workstations.
- Credentials are read from a personal file or shell history.
- The latest source is copied between directories rather than released.
- Tests run only when the author remembers to run them.
- A change is applied from whatever commit happens to be checked out.
- Logs are console text with no change, version, or request identifier.
- Failure leaves uncertain state and no recorded recovery decision.
- A second engineer cannot reproduce the result on a clean system.

These are delivery problems, not failures of Python, Ansible, or the API. Adding more application logic does not solve them. The team needs practices around the automation.

0.9 Readiness check

Learners should be able to answer these questions:

1. What is the difference between parsing data, validating its schema, and enforcing policy?
2. Why does a successful API response not necessarily prove the final service outcome?
3. When is retrying an operation unsafe?
4. How do intended, configured, and operational state differ?
5. What makes an Ansible task idempotent, and why can an idempotent-looking playbook still be unsafe to retry?
6. Which tests can run without network access, and what evidence requires a sandbox or authorized target?
7. Which files belong in Git, and which values must remain in a secret or protected runtime system?
8. What prevents a locally successful script from being reproducible by another engineer?
9. How do model-driven network telemetry and OpenTelemetry complement one another?
10. Which production controls must be added around a basic Flask API?

0.10 Summary

Network automation combines Python, structured data, Git, CLI and API interfaces, models, Ansible, telemetry, authentication, and operational verification. NetBox can provide authoritative, related intent and inventory without becoming a credential or telemetry store. Netmiko, `ncclient`, `requests`, and Flask address different application boundaries, while Ansible supplies reusable inventory, templating, orchestration, and network modules. Reliable automation depends on validated inputs, explicit targets, bounded operations, secure credentials, deterministic processing, structured evidence, and verification of actual network behavior.

What the learner now has: a refreshed model of automation inputs, execution, state, validation, and operational evidence.

What is still missing: a repeatable, governed environment in which the automation can be recreated and operated by a team.

What the next module adds: explicit ownership and lifecycle control for the supporting environment.

The automation logic works. The delivery system does not yet exist.

MODULE 1

1 Introducing the DevOps Model

Central engineering question

Can a team own, deliver, measure, and improve the system?

FLOW → FEEDBACK → EVIDENCE → IMPROVEMENT

Learning objectives

- Explain the DevOps operating model and CALMS.
- Relate flow, feedback, evidence, and shared ownership to reliable delivery.
- Compare manual delivery, ad hoc automation, and governed pipelines.

Estimated theory time	3 hours
Lab connection	Apply the module controls to the cumulative application and delivery system
Core classroom topics	Concepts, decisions, workflows, and required controls
Advanced reference	Detailed implementation and platform considerations

1.1 From ad hoc automation to DevOps

Task automation often begins with a Python script, an API integration, or an Ansible playbook created to meet an immediate operational need. This approach can be effective at small scale, but weaknesses emerge when the solution must be reviewed by a team, reproduced in a clean environment, released safely, diagnosed consistently, and supported independently of its original author.

DevOps addresses that delivery problem. It brings development and operational responsibilities into one feedback system and applies proven software practices to the complete path from source change to operating service. When the application automates infrastructure or networks, the same model is sometimes called infrastructure DevOps or NetDevOps; the underlying delivery principles do not change.

The contrast is important:

Table 1-1. Ad Hoc Automation and DevOps Delivery Comparison

Ad hoc automation	DevOps delivery
An engineer runs a local script	A versioned application runs in a defined environment
Dependencies are installed from memory	Dependencies are declared, locked, built, and scanned
Testing depends on the author	Automated tests run for every proposed change
Credentials live on a workstation	Workload identities and secrets are supplied at runtime
Success means the command completed	Acceptance checks prove the software and service outcome
Knowledge remains with an individual	Code, evidence, runbooks, and decisions are shared

Network delivery has several characteristics that affect the implementation:

- A single configuration error can affect many shared services.
- Devices are long-lived and mutable; teams rarely rebuild a production router for each change.
- Actual service behavior depends on protocol relationships among devices.
- Configuration acceptance does not prove forwarding or routing success.
- Multiple management interfaces and controllers may own overlapping state.
- Maintenance windows, approvals, and regulatory evidence may remain necessary.

NetDevOps therefore emphasizes scoped targets, intended state, pre-change facts, configuration diffs, post-change operational validation, and tested recovery.

1.2 CALMS applied to software delivery

CALMS assesses DevOps as an operating model rather than a collection of tools. It represents **Culture, Automation, Lean, Measurement, and Sharing**. The dimensions work together: automation without ownership can accelerate a poor process, while collaboration without repeatable execution remains dependent on individuals.

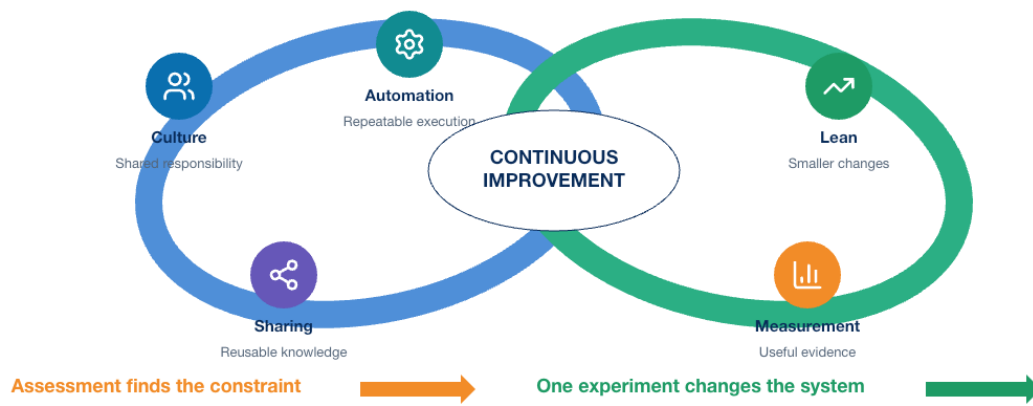


Figure 1-1. Icon-led CALMS model showing Culture, Automation, Lean, Measurement, and Sharing as a continuous improvement loop

Table 1-2. CALMS Dimensions in Software Delivery

Dimension	Software-delivery interpretation	Evidence in this course
Culture	Developers, security, platform, and operations teams share service outcomes	Merge-request review and joint acceptance criteria
Automation	Repeatable building, testing, deployment, and recovery replace engineer-specific procedures	GitLab jobs, automated tests, containers, and deployment code
Lean	Small, independently reviewable changes move through a visible flow with limited work in progress	One application change on a short-lived branch
Measurement	Delivery and application behavior produce usable measures	Pipeline duration, failure rate, latency, availability, and telemetry
Sharing	Code, intent, runbooks, findings, and reusable tests remain available to the team	One repository and retained pipeline evidence

1.2.1 Culture: shared responsibility for the outcome

Culture replaces isolated handoffs with shared responsibility for delivery and operation. Specialists retain distinct roles and access, but application, platform, security, and operations owners agree on acceptance criteria, release scope, observability, and recovery before deployment. Useful evidence includes cross-functional review, explicit ownership, accessible runbooks, and incident reviews that improve controls rather than assign blame.

1.2.2 Automation: make the safe path repeatable

Automation makes reviewed work repeatable across build, test, security, deployment, verification, recovery, and cleanup. Dependable automation is versioned, testable, observable, bounded by timeouts and scope, and explicit about failure. Teams should first understand the process being automated; otherwise ambiguity and unsafe behavior merely execute faster. Manual approval may remain, but it should reference an exact commit, artifact, environment, proposed effect, and evidence.

1.2.3 Lean: improve flow and reduce batch risk

Lean improves the flow of small, reviewable changes. It exposes queues, repeated setup, oversized releases, late testing, and work waiting for a specialist. Short-lived branches, early checks, limited

work in progress, and reusable test environments reduce delay and batch risk. Lean does not remove necessary controls; it designs them to provide evidence quickly and consistently.

1.2.4 Measurement: use evidence to guide improvement

Measurement connects delivery activity with service outcomes. Useful measures include feedback time, deployment frequency, lead time, change failure rate, recovery time, availability, latency, and error rate. Definitions must remain consistent, and the measures should guide improvement rather than rank individuals. A balanced view considers speed, quality, reliability, and recovery together.

1.2.5 Sharing: make knowledge part of the system

Sharing moves knowledge from private notes and memory into version-controlled code, reviews, decisions, tests, dashboards, incident findings, and runbooks. Access alone is insufficient: another engineer should be able to build, release, diagnose, and recover the service from the recorded context. Shared knowledge shortens onboarding and makes assumptions available for review.

1.2.6 How the CALMS dimensions reinforce one another

The dimensions form a feedback system. Culture makes failures discussable; Sharing preserves what was learned; Automation embeds the improved procedure; Lean reduces the delay and size of the next change; and Measurement shows whether the change helped. CALMS exposes imbalance—for example, a sophisticated pipeline that still depends on one engineer to approve, diagnose, and recover every release.

1.2.7 CALMS assessment questions

An engineering team can use the following questions during a retrospective or maturity review:

Table 1-3. CALMS Assessment Questions

Dimension	Questions worth asking
Culture	Who owns the service after deployment? Can team members challenge an unsafe release? Are incidents used to improve the system?
Automation	Can a clean runner reproduce the build and tests? Are errors, retries, cleanup, and recovery automated safely?
Lean	Where does work wait? How large are release batches? Which manual approval or environment dependency is the current constraint?
Measurement	Do measures cover both delivery and runtime outcomes? Are definitions consistent? Does the team act on what it measures?
Sharing	Can another engineer build, release, troubleshoot, and recover the service from repository and operational records?

Do not reduce the result to a maturity score. Select an observable weakness and a small improvement experiment, such as building on a clean runner, adding one reliable acceptance test, or publishing a tested recovery procedure.

1.3 Three delivery models

DevOps is easier to understand when it is compared with the delivery models it replaces or improves. The following comparison shows how ownership, evidence, execution, and recovery change as work moves from manual delivery through isolated automation to an engineered pipeline.

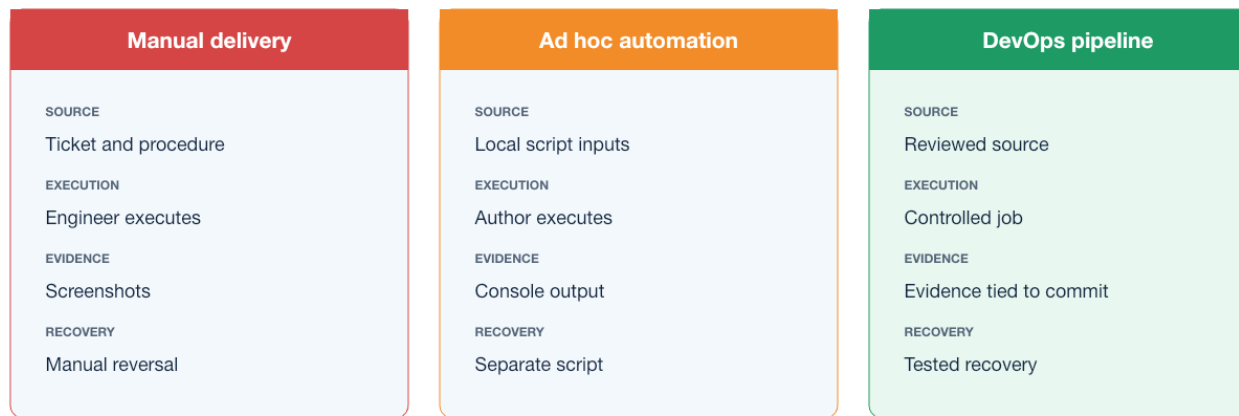


Figure 1-2. Visual comparison of manual delivery, ad hoc automation, and a DevOps pipeline

Table 1-4. Manual Ad Hoc and DevOps Delivery Models

Characteristic	Manual delivery	Ad hoc automation	DevOps pipeline
Source of truth	Ticket and engineer notes	Script inputs or local files	Reviewed source in version control
Execution	Engineer follows a procedure	Author runs a script or playbook	Controlled job deploys an identified artifact
Review	Release checklist	Code review may occur	Source, tests, dependencies, policy, and evidence are reviewed
Validation	Engineer performs selected checks	Script may run checks	Required build, integration, security, and acceptance tests
Credentials	Personal account	Often local environment values	Short-lived or protected workload identity
Environment control	Manually prepared host	Author's workstation or shared server	Defined, reproducible, protected environment
Evidence	Screenshots or copied output	Console log	Structured artifacts tied to commit and pipeline
Recovery	Engineer reverses steps	Separate rollback script	Tested rollback or forward-remediation workflow
Feedback	Incident or ticket closure	Script result	Pipeline result plus application telemetry

Automation improves consistency, but DevOps connects automation to collaboration, governance, and operational truth.

1.4 Complete DevOps lifecycle

The DevOps lifecycle connects an idea to an operating service and returns operational knowledge to the next decision. Deployment is therefore not the end: software must be operated, observed, improved, secured, and eventually retired.

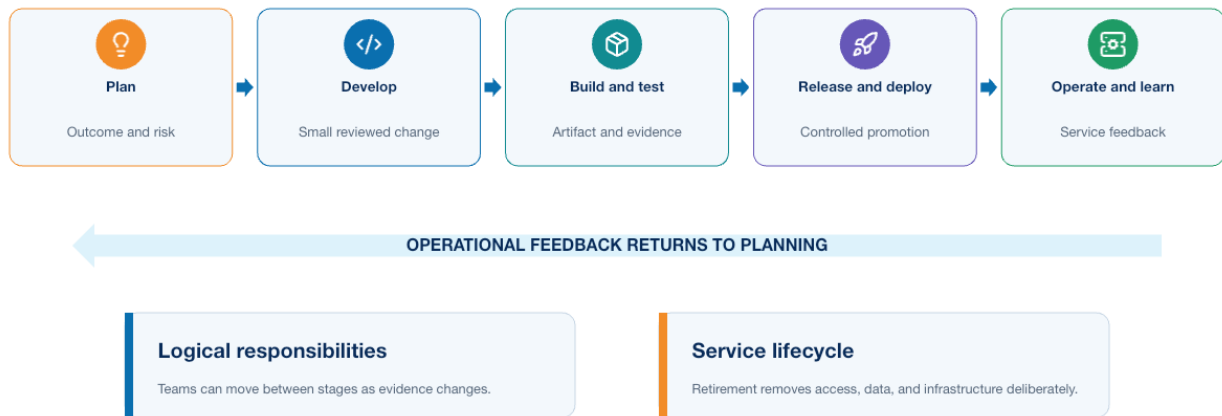


Figure 1-3. Icon-led DevOps lifecycle from planning through development, qualification, release, operation, and learning

Plan → Design → Develop → Integrate → Build → Test → Release → Deploy → Operate → Observe → Learn

These are logical responsibilities, not separate departments or a rigid sequence. Teams move between them continuously: a runtime failure may create a test, a security finding may change the design, and a build failure returns directly to development.

1.4.1 Plan: define the problem and expected outcome

Planning defines the problem, scope, owner, risk, and measurable outcome. It includes functional behavior and operating constraints such as performance, authorization, evidence, and recovery. For the supplied application, success is not merely that a Python function works; another engineer must be able to review, release, observe, and support it through a controlled process.

1.4.2 Design: decide how the change fits the system

Design translates the requirement into components, interfaces, state ownership, trust boundaries, failure behavior, and deployment assumptions. The team decides how dependencies fail, which credentials and paths are required, what can be tested offline, how health is proven, and whether versions can overlap or be restored safely. Important assumptions and trade-offs belong in a short architecture decision record.

1.4.3 Develop: implement a small, reviewable change

Development occurs in version control on a small, reviewable branch. Code, tests, dependency declarations, contracts, and documentation change together when they describe one behavior. Local checks provide early feedback, but the shared pipeline must reproduce them. Secrets remain outside the commit, and implementation quality includes explicit errors, timeouts, safe defaults, and testable boundaries.

1.4.4 Integrate: combine work and obtain early feedback

Integration validates a proposed change against shared source. Fast checks run first, independent checks run concurrently, and expensive environments are created only after basic validation succeeds. Automation evaluates repeatable conditions; reviewers assess intent, design, risk, maintainability, and whether the tests prove the right outcome. Both are required.

1.4.5 Build: create an identifiable artifact

The build converts reviewed source and declared dependencies into an immutable artifact. The record connects its digest to the commit, build, dependencies, tests, SBOM, scan results,

provenance, and signature where required. Build once and promote the same digest; rebuilding for production creates a different artifact without the evidence collected from the tested one.

1.4.6 Test: build confidence at several boundaries

No single test proves a release. A practical test strategy layers evidence:

Table 1-5. Testing Boundaries and Required Evidence

Test layer	Question answered	Typical environment
Static checks	Is the source structurally valid and free of selected defects?	CI runner
Unit tests	Does isolated application logic behave correctly?	CI runner
Component tests	Does the packaged component start and honor its runtime contract?	Container runtime
Integration tests	Do application, database, queue, and API boundaries work together?	Compose or ephemeral services
Security tests	Does the artifact meet dependency, secret, identity, and runtime policy?	Build and test environment
System tests	Does the assembled application perform its important workflows?	On-demand test environment
Acceptance tests	Does the release satisfy the user or operational outcome?	Representative environment

No single test proves a release. Use the least expensive layer that can provide the required evidence, then advance toward representative environments. Flaky tests and unrealistic mocks weaken trust and must be treated as defects.

1.4.7 Release: make a version eligible for deployment

A release is a versioned artifact plus the evidence needed for promotion: digest, tests, vulnerability policy, compatibility, approval, and recovery plan. Release and deployment are different; an artifact can be eligible without running in production. Continuous delivery keeps a validated release deployable, whereas continuous deployment automatically promotes every qualifying release and therefore requires stronger confidence and recovery.

1.4.8 Deploy: change the target environment safely

Deployment verifies the artifact, target, baseline health, capacity, configuration, and credentials before changing an environment through an appropriate strategy. Platform acceptance does not prove service success. Use bounded timeouts, stop conditions, post-checks, and explicit handling for uncertain outcomes; after a timed-out change request, rediscover actual state before retrying.

1.4.9 Operate: keep the service dependable

Operation covers availability, capacity, backup, patching, incident response, credential rotation, dependency maintenance, and recovery. Runbooks define symptoms, evidence, safe actions, and escalation. For an automation service, operators also manage queue health, worker concurrency, API limits, credentials, evidence retention, and per-target locking.

1.4.10 Observe: compare actual behavior with expectations

Observation combines metrics, logs, traces, health checks, events, and release context. Signals must identify the version, environment, and request or job. Immediate checks reveal obvious failure;

an observation window reveals delayed degradation. Liveness, readiness, and end-to-end acceptance prove different conditions and must not be treated as interchangeable.

1.4.11 Learn and improve: close the loop

Operational evidence should change future work. A failure may reveal a missing test, unsafe retry, fragile dependency, capacity assumption, or approval gap. Add the resulting improvement to code, tests, policy, design, or runbooks. Findings need owners and measurable follow-up; collecting incident information without changing the system does not complete the learning loop.

1.4.12 Retire: remove software and access deliberately

Retirement stops traffic and scheduled work, handles data according to policy, revokes credentials, removes owned infrastructure, updates documentation, and preserves required evidence. Before deletion, confirm that no consumer remains and prove ownership of every targeted resource.

1.4.13 Gates, evidence, and promotion

A gate is a decision supported by evidence, not an unexplained approval step.



Figure 1-4. Evidence chain connecting requirement, commit, pipeline, artifact, deployment, and runtime evidence

Table 1-6. Delivery Gates Decisions and Evidence

Gate	Decision	Minimum useful evidence
Merge	Is this change suitable for the shared source branch?	Diff, review, required CI results
Build acceptance	Is the artifact identifiable and compliant?	Digest, tests, SBOM, scan and provenance results
Test promotion	Is the release suitable for a representative environment?	Artifact identity, configuration, test plan
Production promotion	Is the exact tested release acceptable at this time and scope?	Test evidence, target, risk, approval, recovery plan
Rollout continuation	Should exposure increase?	Readiness, acceptance, error and performance signals
Completion	Has the intended outcome been achieved?	Deployment record, acceptance result, observation evidence
Recovery	Is rollback safe, or is forward remediation required?	Actual state, data compatibility, failure classification

requirement → source change → commit → pipeline → test results → artifact digest → approval → deployment → runtime evidence

Evidence must remain connected along this chain. A new commit, artifact, target, or configuration can invalidate an earlier approval and require renewed validation.

1.4.14 Feedback loops at different speeds

Feedback arrives at different speeds, and each loop needs an owner and a path back into source, tests, policy, documentation, or design:



Figure 1-5. Feedback loops grouped by seconds, minutes, hours, and weeks

- **Seconds to minutes:** formatter, linter, schema validation, and unit tests guide the developer.
- **Minutes to hours:** integration, security, packaging, and system tests guide merge and release decisions.
- **Hours to days:** deployment and runtime signals reveal behavior under representative or real workloads.
- **Weeks to months:** delivery measures, incident patterns, dependency health, capacity, and architecture reviews guide investment.

Fast feedback reduces correction cost, but slower operational feedback remains essential because tests cannot reproduce every production condition.

1.4.15 Applied example: from commit to operating automation service

Suppose a developer improves error classification in the Python worker:

1. Planning defines retryable, permanent, and uncertain outcomes; design specifies safe retry behavior.
2. Development changes the classifier, tests, and operating guidance in one reviewed branch.
3. CI runs static, unit, fixture, and security checks; the build produces one scanned image and records its digest.
4. Integration tests inject a timeout and confirm that uncertain work is not repeated automatically.
5. Reviewers approve the commit, digest, and evidence; deployment introduces that digest and verifies readiness and a representative job.
6. If uncertain jobs increase, rollout stops and the team recovers according to actual state. The finding becomes a regression test and runbook improvement.

The code change may be small; the lifecycle makes it safe for a team to deliver and support repeatedly.

1.5 A practical software delivery architecture

The following responsibilities appear in most mature delivery systems, although the products and team boundaries vary.

The reference architecture makes the most important trust transition visible: unprivileged validation produces an identified artifact before an approved protected runner receives management access.

Design Insight

A runner is not safe merely because the CI platform labels it protected. Its effective trust boundary is determined by which jobs it accepts, which identities it can obtain, and which endpoints it can reach.



Figure 1-6. Delivery architecture separating unprivileged validation from protected deployment

Operational evidence returns to the engineer and repository; it is not an isolated monitoring destination.

- The Git repository stores intent, automation code, tests, policy, pipeline definitions, and operational documentation.
- An unprivileged validation runner parses and normalizes intent, applies schema and policy, renders candidates, executes offline tests, and performs supply-chain checks. It has no device route or deployment credential.
- An approval gate binds the reviewed commit, target inventory fingerprint, rendered difference, test evidence, and automation image digest.
- A protected network runner or restricted worker receives only the approved job, a short-lived identity, explicit targets, and the minimum management route.
- Devices or controllers expose SSH, NETCONF, RESTCONF, or platform APIs according to capability and transaction requirements.
- Post-checks and telemetry produce evidence about configuration, protocol, forwarding, and platform health.
- The final decision promotes the result, stops further rollout, rolls back safely, or initiates forward remediation.

Module 2 assigns infrastructure ownership and lifecycle controls. Module 3 applies reproducibility to container packaging, multitier services, and Kubernetes. Module 4 implements CI/CD, protected execution, validation, and recovery. Module 5 secures every boundary and builds the operational feedback path.

1.6 Continuous integration, delivery, and deployment

This section defines the three practices and their decision boundaries. Module 4 implements them through GitLab jobs, runners, artifacts, environments, rules, and protected deployment.

These terms describe different levels of automation.

Continuous integration means developers merge small changes frequently and automated checks validate the combined code. The goal is to find integration problems while the relevant change remains small and understandable.

Continuous delivery means the pipeline produces a validated release that the team can deploy through a controlled decision. Production promotion may require approval.

Continuous deployment means every change that passes the required controls proceeds automatically into production. This requires strong tests, reliable rollback or remediation, good observability, and confidence in the platform.

The course begins with continuous integration, adds automated deployment to a training environment, and may finish with a controlled Kubernetes platform exercise. Production deployment remains a design decision rather than an assumption.

One repository may support several delivery purposes without forcing them into one execution path. A software-release pipeline answers whether an application version can be built, secured, and deployed. An infrastructure pipeline creates or changes resources. A network-change pipeline evaluates intent, proves the change on a representative target, and controls production promotion. A scheduled assurance pipeline detects drift or expiring dependencies without deploying anything.

These pipelines may share code, images, policies, runners, and evidence services, but they should not share an ambiguous trigger. Each pipeline needs a defined initiating event, input contract, privilege level, target type, acceptance criteria, and completion condition. Separation by purpose reduces accidental coupling: releasing an application does not silently authorize a network change, and changing network intent does not rebuild an unrelated platform.

1.7 Value stream and constraints

A value stream describes the work from request to operational outcome. Useful questions include:

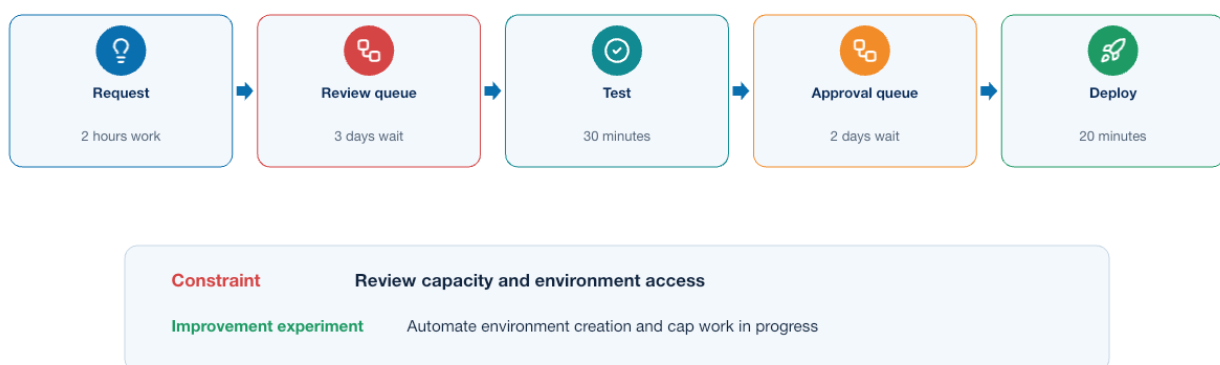


Figure 1-7. Value stream showing work time, waiting time, and the delivery constraint

- Where does work wait?
- Which steps depend on one person?
- Which actions are repeated manually?
- Where do defects first become visible?
- How quickly can the team restore service after a failed change?

Four delivery measures are especially helpful:

- Deployment frequency: how often the team releases successfully
- Lead time for changes: how long a committed change takes to reach operation
- Change failure rate: how often a release causes degradation or remediation
- Time to restore service: how long recovery takes after a failure

The measures should guide improvement, not reward artificial activity. Increasing deployment frequency by weakening tests would damage the complete system.

1.7.1 DORA measures for network delivery

DORA measures require careful interpretation in a network context:

Table 1-7. DORA Measures for Network Delivery

Measure	Network interpretation	Example
Deployment frequency	Successful approved changes delivered per period	Number of automation releases promoted each week
Lead time for changes	Time from reviewed commit to verified operational outcome	Merge of a policy update to confirmed enforcement
Change failure rate	Portion of changes requiring rollback, remediation, or incident response	A parser upgrade produces incomplete validation results
Time to restore service	Time from detected degradation to verified recovery	Alert to restored adjacency and reachability

The team should also measure pipeline feedback time, percentage of changes with complete evidence, drift age, policy failure reasons, and automation job reliability. Do not compare teams without accounting for network scope, risk, and change type.

1.8 Applying DevOps controls to existing network intent

DEVASC/DEVCOR-level knowledge of structured data, APIs, and network intent is assumed. The DevOps concern is how an existing intent contract becomes a controlled pipeline input. Depending on the application, YAML might describe interface addressing, a service, routing policy, compliance rules, or validation expectations. A JSON Schema or Python model validates structure and types before a build or deployment job receives privileged access.

YAML is convenient for human review but has traps: indentation controls structure, unquoted values can receive unexpected types, and duplicate keys may be accepted differently by parsers. The pipeline must parse with a controlled library and validate against a schema.

JSON has stricter syntax and maps naturally to REST payloads. XML remains important for NETCONF and many YANG-encoded operations. Jinja2 converts validated data into platform configuration when a structured API is unavailable or unsuitable.

The data path moves from reviewed intent through schema validation and a normalized model to a renderer or API payload. After deployment, the workflow collects device state and compares it with the same intent.

Templates must not contain business logic that belongs in validation or normalization. A rendered configuration is derived output; reviewed intent remains the source.

1.8.1 Example input consumed by an existing application

The following supplied application input is illustrative. Learners are not expected to design its schema or network logic during this course. They use it to implement linting, schema validation, policy checks, rendering tests, artifact handling, promotion, and operational evidence.

```

---
schema_version: 1
change_id: CHG-2026-0042
site: campus-west
device: distribution-01
platform: lab-nos
service:
  vlan:
    id: 120
    name: USERS
interface:
  name: Vlan120
  ipv4: 10.20.120.1/24
routing:
  protocol: ospf
  process_id: 100
  area: 0
  passive: true
validation:
  expected_neighbor_state: FULL
  expected_prefix: 10.20.120.0/24
  maximum_devices: 1

```

Table 1-8. Intent Validation and Delivery Stages

Field group	Meaning	Validation responsibility
<code>schema_version</code> , <code>change_id</code>	Contract and traceability identity	Required, correctly formatted, and recognized
<code>site</code> , <code>device</code> , <code>platform</code>	Target selection context	Must resolve to an approved inventory object; never trust free text as authorization
<code>service.vlan</code>	Requested logical service	VLAN range, reserved identifiers, naming, and local uniqueness
<code>interface</code>	Platform-neutral gateway intent	Interface-name policy, valid prefix, address ownership, and overlap detection
<code>routing</code>	Required advertisement behavior	Approved protocol and process, valid area, safe passive-interface policy
<code>validation</code>	Observable acceptance criteria	Supported state vocabulary, derived network prefix, and explicit blast-radius ceiling

The processing stages have different responsibilities:

1. YAML parsing determines whether the document is syntactically readable.
2. JSON Schema or a Pydantic model validates required fields, types, ranges, and allowed values.
3. Normalization converts the address into canonical interface and network objects, normalizes names, and rejects ambiguous values.
4. Business policy checks ownership, allocation, protocol standards, reserved resources, maintenance rules, and maximum scope.
5. A platform renderer produces reviewed device CLI, NETCONF XML, or a RESTCONF/controller payload. The chosen interface depends on discovered capabilities.
6. The device accepts or rejects the operation and exposes stored configuration state.
7. pyATS, Genie, modeled reads, and probes compare operational state with the validation contract.

Schema success does not grant authorization, policy success does not prove correct rendering, configuration acceptance does not prove service health, and an immediate post-check does not replace continued telemetry.

1.9 Where implementation depth belongs

Module 1 establishes the delivery model and the questions that each control must answer. Later chapters apply it in increasing operational depth. Module 2 introduces infrastructure lifecycle, state, drift, and tool ownership; Module 3 covers containers, image construction, multitier services, and Kubernetes orchestration; Module 4 covers GitLab jobs, runners, artifacts, promotion, mutable state, blast radius, convergence, and recovery; and Module 5 covers trust boundaries, credentials, operational feedback, and stability.

1.10 Knowledge check

Use these questions to test whether you can connect DevOps principles to engineering decisions rather than recall definitions alone.

1. Why can extensive pipeline automation still represent weak DevOps maturity?
2. How do Culture and Sharing make Automation more sustainable?
3. How can a team apply Lean thinking without weakening release controls?
4. Which delivery and runtime measures should be evaluated together to avoid misleading conclusions?
5. How does continuous delivery differ from continuous deployment?
6. What evidence should a reviewer see before approving a release?
7. Why should an artifact be built once and promoted by digest through later lifecycle stages?
8. How do release, deployment, readiness, and acceptance represent different outcomes?
9. Why must a deployment workflow rediscover actual state before retrying an operation with an uncertain result?
10. How do fast CI feedback and slower production feedback contribute different knowledge?

1.11 Summary

DevOps changes the way a team makes and proves a change; it is not a synonym for scripting or CI software. CALMS provides a balanced way to examine that change: Culture creates shared responsibility, Automation makes the safe path repeatable, Lean improves flow, Measurement tests whether the system is improving, and Sharing makes knowledge reusable. The lifecycle connects those principles to concrete controls from planning through operation and retirement.

What the learner now has: a lifecycle model based on CALMS, flow, feedback, evidence, measurement, promotion, and shared responsibility.

What is still missing: controlled ownership and lifecycle for the infrastructure and test environments through which software will move.

What the next module adds: Module 2 applies the delivery model to declarative infrastructure, tool ownership, protected state, on-demand test environments, and cleanup. Continue to Infrastructure as Code and On-Demand Environments.

The DevOps model now requires a repeatable environment with explicit ownership and lifecycle controls.

MODULE 2

2 Infrastructure as Code and On-Demand Environments

Central engineering question

Can we recreate and govern the environment?

PLAN → CREATE → CONFIGURE → VERIFY → DESTROY

Learning objectives

- Separate infrastructure provisioning, configuration, and application logic.
- Explain Terraform state, drift, lifecycle, and safe cleanup.
- Design an on-demand environment that hands explicit outputs to configuration automation.

Estimated theory time	3 hours
Lab connection	Apply the module controls to the cumulative application and delivery system
Core classroom topics	Concepts, decisions, workflows, and required controls
Advanced reference	Detailed implementation and platform considerations

2.1 Three automation responsibilities

The ownership flow distinguishes resource lifecycle, ordered configuration, and custom application logic before a team selects Terraform, Ansible, or Python for a task.

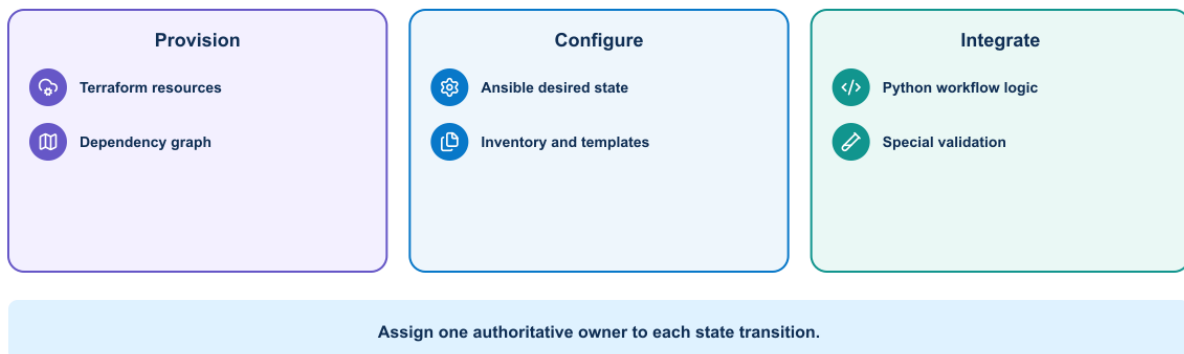


Figure 2-1. Tool ownership decision flow for Terraform, Ansible, and Python

Infrastructure provisioning, device configuration, and custom workflow logic overlap, but they are not identical. The course assigns clear ownership:

Table 2-1. Terraform Ansible and Python Responsibility Boundaries

Need	Terraform	Ansible	Python
Create API-managed infrastructure resources	Strong fit	Possible with modules	Possible but more lifecycle code
Produce a proposed resource plan	Native strength	Limited and module-dependent	Must be designed
Track resource lifecycle and dependencies	State and dependency graph	Inventory and task order	Application must implement it
Configure many network devices	Provider-dependent	Strong fit with supported network collections	Strong for custom protocols and logic
Render Jinja2 configuration	Possible but not preferred here	Strong fit	Strong fit
Run custom schema and policy validation	External checks	Assertions and plugins	Strong fit
Implement RESTCONF or NETCONF workflow	Provider-dependent	Supported modules/collections	Maximum protocol control
Parse and correlate evidence	Outputs and state queries	Registered results	Strong fit
Manage idempotence	Provider/resource model	Module behavior	Team must implement
Best course role	Create disposable test environment	Configure and orchestrate devices	Normalize, validate, integrate, and analyze

Select the tool that has the clearest ownership and most reliable model for the target. Do not call Terraform, Ansible, and Python against the same configuration object without an explicit handoff.

2.1.1 Practical tool-selection decisions

Tool choice should follow the type of resource, the available interface, and the operating model of the team. These guidelines highlight common strengths without treating any tool as a universal solution.

- Use **Terraform** when an API resource has a stable lifecycle, dependency relationships, and a provider that can plan and reconcile it reliably. Avoid it for procedural diagnostics, unsupported device features, or objects also owned by another controller.
- Use **Ansible** when the work is ordered, human-readable orchestration across inventories and supported network modules express the intended change. Validate module idempotence and check/diff behavior on the actual platform.
- Write **Python** when the workflow requires custom data normalization, policy, correlation, API pagination, transaction handling, or error logic that modules do not expose. Accept that the team owns tests, retries, idempotence, packaging, and maintenance.
- Use **pyATS/Genie** to verify operational outcomes with structured data; do not use parser success as proof that the expected state exists.

2.2 Infrastructure as Code

Infrastructure as Code records intended infrastructure in machine-readable files. It provides reviewable change history and repeatable execution. The definition may describe resources declaratively or express a procedural workflow.

IaC benefits include:

- Repeatable environments
- Reviewable changes
- Automated validation
- Faster recovery and recreation
- Consistent tagging and policy
- Traceability between a release and its environment

IaC does not guarantee correctness. Incorrect code can reproduce an unsafe design consistently.

2.3 Desired state and orchestration

Declarative tools describe the desired result. The tool compares desired state with observed or recorded state and determines actions. Procedural automation specifies operations in order.

Most delivery systems use both. Terraform can create infrastructure resources, while Ansible configures operating systems, network devices, or application prerequisites. Shell and Python scripts may coordinate specialized checks.

Ownership must remain clear. Two tools attempting to control the same setting can create oscillation and drift.

2.4 Terraform model

Terraform configuration commonly contains:

- Providers that communicate with a platform API
- Resources that describe managed objects
- Data sources that read existing information
- Variables that accept inputs
- Local values that derive reusable expressions
- Outputs that expose selected results
- Modules that package related infrastructure

Terraform builds a dependency graph from references and can perform independent operations concurrently.

2.4.1 Network example

Terraform may create a controller-managed tenant policy, cloud network, virtual network-device instance, simulated lab definition through an available provider, or infrastructure supporting the automation platform. Provider maturity and platform API behavior determine suitability.

Illustrative resource relationships are:

The test environment contains a management network, a virtual network device, a test peer, scoped security policy, and temporary DNS or inventory output.

Terraform outputs can generate a sanitized inventory input, but credentials should come from a protected identity system.

2.5 Terraform lifecycle

Terraform uses a repeating desired-state lifecycle.

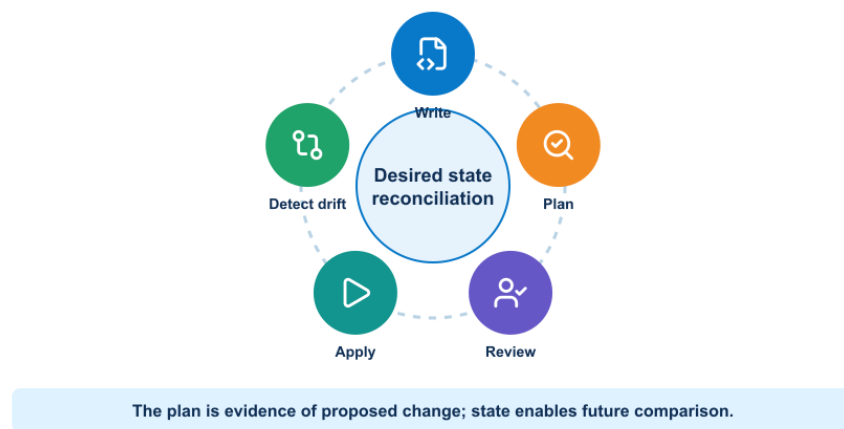


Figure 2-2. Terraform desired-state lifecycle from authoring through planning, application, and drift detection

The normal lifecycle includes:

1. `terraform fmt` for consistent format
2. `terraform init` for providers and modules
3. `terraform validate` for structural validation
4. `terraform plan` to compare configuration and state
5. Review of target, actions, replacements, and sensitive impact
6. `terraform apply` using an approved plan where possible
7. Output and independent operational verification
8. `terraform destroy` for an intentionally disposable environment

A plan can become stale when configuration, state, variables, credentials, or real infrastructure changes. Apply should use the reviewed plan artifact within a controlled window.

2.6 State

Terraform state maps configuration addresses to real resources and stores attributes used for planning. State can contain sensitive values even when configuration marks output as sensitive.

Team use requires a protected remote backend with access control, locking, encryption, versioning, and recovery. State files should not be committed to Git.

Before an unusual recovery or import action, inspect the configuration, state, and real platform. Guessing can cause replacement or loss.

2.7 Drift

The reconciliation flow asks who owns a changed attribute and whether the correct response is acceptance, controlled correction, or escalation rather than automatic overwrite.

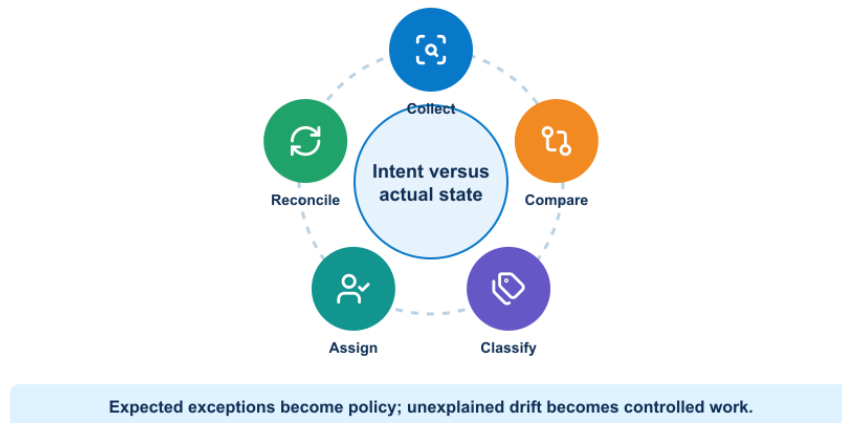


Figure 2-3. Drift collection, classification, ownership resolution, and controlled reconciliation

Drift occurs when real infrastructure differs from the controlled definition. It may result from manual changes, another tool, platform defaults, or failed operations.

A scheduled plan can detect drift. The response depends on ownership and intent: revert the manual change, update code to adopt it, import an existing resource, or investigate a provider difference. Automatically applying every detected change may be unsafe.

2.8 Ansible model

Ansible uses inventories, variables, playbooks, roles or collections, and modules. Network automation often connects through SSH or APIs without installing an agent on the managed device.

Ansible tasks should prefer purpose-built modules over free-form shell commands. Modules can understand current state, provide structured results, and support idempotent behavior.

Useful design practices include:

- Separate inventory from reusable automation logic.
- Keep secrets outside inventory files.
- Validate required variables.
- Use check or diff capabilities when trustworthy.
- Limit the target group explicitly.
- Capture before-and-after evidence.
- Use handlers for changes that require a restart.
- Make failure and rollback behavior visible.

2.8.1 Inventory and variables

An inventory identifies targets and groups. It should separate environment-specific addressing from reusable roles. Group variables can define platform defaults, while host variables define required exceptions.

```

all:
  children:
    lab_routers:
      hosts:
        distribution-01:
          ansible_host: 192.0.2.11
          role_id: edge
          intended_state: examples/compliance-intent.yml
      vars:
        ansible_network_os: vendor.collection.network_os
        ansible_connection: ansible.netcommon.network_cli

```

Do not store the password in this file. The playbook receives a protected runtime credential or uses an approved credential plugin.

2.8.2 Playbooks, roles, and templates

A playbook connects targets and ordered outcomes. A role packages reusable tasks, handlers, templates, defaults, and tests. A Jinja2 template converts normalized intent into platform configuration.

Keep validation separate from rendering. Ansible should fail before device access when an intended object or prefix violates policy. Use `serial` or explicit batching for a controlled rollout. Use `--limit distribution-01` or an equivalent pipeline constraint and display the selected hosts before change.

2.9 Provisioning and configuration boundary

Terraform normally owns lifecycle-oriented infrastructure resources. Ansible normally owns configuration within reachable hosts or devices. The exact boundary depends on provider quality and team design.

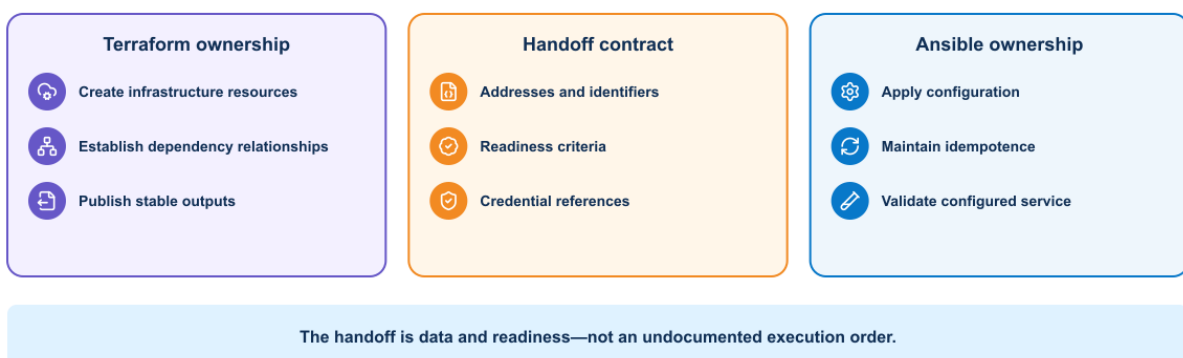


Figure 2-4. Ownership and data handoff between Terraform provisioning and Ansible configuration

For an illustrative automation platform:

- Terraform creates the disposable test environment and exports connection information.
- Ansible configures the environment for application deployment.
- The application pipeline deploys the tested image.
- Validation scripts assess the running service.
- Terraform removes the environment after evidence collection.

2.10 On-demand test environments

An ephemeral environment has a complete lifecycle, including evidence collection and controlled cleanup after failure.



Figure 2-5. On-demand environment lifecycle with failure-safe evidence and cleanup

Cleanup retains the original failure and targets only resources whose ownership is proven.

An on-demand environment gives a branch or merge request an isolated place for integration and system testing. A useful design includes:

- Unique, validated naming
- Restricted network scope
- Short-lived credentials
- Resource and cost limits
- Automatic expiration
- Traceable ownership
- Repeatable initialization
- Reliable cleanup

The environment should resemble the target environment in the characteristics relevant to the test. It does not need production scale for every merge request.

2.11 Network test environment options

Test environments trade speed and cost against behavioral fidelity. The table compares common choices so that an engineering team can match the environment to the risk and evidence required from a test.

Table 2-2. Network Test Environment Options

Platform	Strength	Limitation	Suitable course use
Vendor sandbox	Authorized access to hosted network platforms with low setup effort	Reservation, VPN, shared or reset behavior, Internet dependency	Real API and device capability exercises
Network simulator or emulator	Controlled multi-device topologies	Feature fidelity and compute requirements	Routing convergence, topology, and rollback tests
Virtual network appliance	High platform realism	Image access, compute, and licensing	Dedicated device and peer validation
Mock RESTCONF/API service	Fast, deterministic, safe, easy in CI	Cannot prove real YANG or protocol operation	Error handling, schema, pagination, and offline integration
Container-based test components	Rapid service and API testing	Network OS behavior is incomplete	Automation API, queue, database, and telemetry integration

Platform	Strength	Limitation	Suitable course use
Physical lab	Real interfaces and platform behavior	Cost, contention, reset, and blast radius	Advanced instructor-controlled validation

The course must remain usable without production access. Offline checks and mocks run for every learner. Real-device jobs use an authorized sandbox, virtual lab, or instructor-provided environment.

2.11.1 Cisco Modeling Labs as an on-demand test target

Cisco Modeling Labs (CML) can provide a controlled topology running virtual network operating systems. Through its API and an appropriate Terraform provider, a delivery system can create a lab only when a realistic integration test is required. The environment can exercise SSH behavior, network modules, configuration semantics, parsers, and operational verification that a mock service cannot reproduce.

An on-demand CML design must define the lab and node definition, approved image, management connectivity, isolated addressing, external-connector policy, bootstrap configuration, platform capacity, unique owner, maximum lifetime, and deterministic deletion. CML API access is privileged, virtual router images remain licensed assets, and an external connector may join an ephemeral topology to a real management network. Provider compatibility, certificate validation, address allocation, and concurrency therefore belong to platform policy.

2.11.2 The create–configure–verify–destroy contract

Ephemeral test infrastructure has a stronger contract than “create when needed”:

1. **Create:** Terraform creates only resources carrying the execution's identity.
2. **Bootstrap:** the platform makes the target reachable through an approved management path.
3. **Configure:** Ansible applies configuration inside the resource without owning its lifecycle.
4. **Verify:** an independent test observes configured and operational state.
5. **Preserve evidence:** the system records what was created, changed, and observed.
6. **Destroy:** Terraform deletes its resources even when configuration or verification fails.
7. **Confirm cleanup:** the workflow verifies that state contains no remaining owned resources.

Cleanup is a release gate, not housekeeping. An environment that cannot be deleted consumes licenses and compute, creates address conflicts, and may retain credentials or unintended management reachability. Production promotion should not depend on an environment that the platform has failed to control.

2.12 Network drift and compliance

Configuration drift is a difference between controlled intent and actual configuration. Operational drift is a difference between the expected and observed service state. The two can occur independently.

Examples include:

- An engineer manually removes the passive-interface command.
- A controller overwrites an interface description.
- The configuration remains correct but an OSPF neighbor fails.
- A route appears through an unintended protocol or next hop.

A scheduled read-only pipeline can collect modeled configuration and operational state, normalize it, compare it with intent, and open a review item. Automatic reconciliation may suit low-risk objects, but shared routing changes should follow policy and approval.

Compliance rules should identify ownership, severity, tolerated exceptions, and remediation path. A permanent exception belongs in reviewed policy data, not a hidden `if` statement.

2.13 Pipeline integration

This section describes only the future handoff: a delivery pipeline will consume reviewed plans, environment identifiers, validation results, and cleanup evidence. Pipeline jobs, runners, rules, and promotion are implemented in Module 4.

A safe infrastructure pipeline separates responsibilities:

The infrastructure pipeline formats and validates the definitions, applies policy, creates and reviews a plan, provisions the environment, configures it, deploys the automation components, runs tests, collects evidence, and destroys only the disposable resources it created.

Cleanup should run when tests fail, but it must target only the environment created for the pipeline. Store the exact environment identifier as an artifact. Avoid a wildcard cleanup operation.

2.13.1 Practical ownership boundary: Terraform hands off to Ansible

The handoff is a machine-readable inventory artifact, making the ownership boundary visible. Consider an on-demand test environment. Terraform creates the isolated network, compute instances, security rules, and DNS records, then exports a machine-readable inventory. Ansible consumes that inventory to install the container runtime, configure trust anchors, and start the application. Terraform should not run a long sequence of remote shell provisioners, and Ansible should not create cloud networks through ad hoc tasks. The handoff artifact lets the pipeline prove that configuration targeted only resources created by that run.

Before `apply`, a policy job can reject a plan that creates a public address or opens a management port to `0.0.0.0/0`. Before cleanup, the job compares the recorded environment identifier and ownership tags with current state. A missing or mismatched tag is a stop condition, not a reason to broaden the destroy command.

2.14 Secrets and access

Infrastructure jobs often hold powerful credentials. Use separate identities for planning, applying, configuration, and deployment when practical. Restrict each identity to the target environment and required operations.

Prefer short-lived credentials issued to the job. Protect state, plans, logs, and artifacts because they may expose addresses, identifiers, or sensitive values.

2.15 Infrastructure testing

Testing can include:

- Syntax and schema validation
- Policy evaluation
- Unit tests for modules or transformations
- Plan assertions
- Resource-count and target checks
- Configuration verification
- Connectivity and service tests
- Destruction and recreation tests
- Drift detection

Independent verification is valuable. A tool reporting successful apply confirms API operations, but it does not prove that users can reach the service or that the security policy works as intended.

2.16 Network platform considerations

Controllers and network devices may expose declarative APIs, model-driven interfaces, and Ansible or Terraform integrations. Before automation, determine object hierarchy, transaction behavior, eventual consistency, rate limits, and rollback capability.

Shared network infrastructure requires strict target validation and change scoping. A sandbox, simulator, or dedicated training environment is appropriate for learning destructive lifecycle operations.

2.17 Knowledge check

Use these questions to evaluate infrastructure state, test-environment selection, and the controls required for repeatable provisioning.

1. Why should Terraform state not be committed to Git?
2. What should a reviewer inspect in a Terraform plan?
3. How can Terraform and Ansible divide responsibility in a network automation workflow?
4. Why must cleanup use the exact environment identifier created by the pipeline?
5. What evidence proves more than a successful infrastructure API response?

2.18 Summary

Infrastructure delivery introduces the first complete delivery control chain in the guide: declared intent, authoritative ownership, a proposed plan, review, controlled execution, observed state, retained evidence, and reconciliation. Terraform is strongest at resource lifecycle; Ansible is strongest at configuration and orchestration. CML can supply a realistic on-demand network environment when mocks are insufficient. The handoff between tools must be explicit, state must be protected, and cleanup must prove ownership and completion before later promotion.

What the learner now has: repeatable infrastructure with explicit ownership among Terraform, Ansible, Python, and operational verification.

The system can now recreate its supporting environment, but it still cannot guarantee that the automation application behaves identically for every engineer and execution host. Infrastructure definitions do not lock the Python interpreter, Python packages, Ansible collections, system libraries, or application startup behavior. IaC solves environment lifecycle; it does not solve application runtime reproducibility.

What is still missing: a reproducible application runtime that can move through the established delivery model without being reconstructed differently on every execution host.

What the next module adds: Module 3 makes the application runtime reproducible through container images, explicit service contracts, multitier deployment, and orchestration. Continue to Packaging and Operating Applications.

MODULE 3

3 Packaging and Operating Applications

Central engineering question

Can we reproduce and operate the application runtime?

SOURCE → IMAGE → RUNTIME → PLATFORM

Learning objectives

- Build and assess reproducible container images.
- Explain multitier application contracts, Docker Compose, and failure boundaries.
- Select Kubernetes only when its orchestration capabilities are justified.

Estimated theory time	4.5 hours
Lab connection	Apply the module controls to the cumulative application and delivery system
Core classroom topics	Concepts, decisions, workflows, and required controls
Advanced reference	Detailed implementation and platform considerations

3.1 Container runtime fundamentals

3.1.1 The application consistency problem

An engineer runs `app.py` successfully from a laptop. The GitLab job fails with a different Python package version. Another engineer has an older Ansible collection, and a system Python upgrade changes a parser. The team describes this as “works on my laptop,” but the underlying problem is an undefined application runtime.

A network automation container makes the runtime explicit:

The versioned automation image combines a fixed Python version, locked packages, controlled Ansible collections, pyATS and Genie parsers, NETCONF/RESTCONF/SSH clients, Jinja2 templates, validation code, certificate trust, and required utilities.

The same image can render the proposed configuration, execute offline tests, collect pre-checks, and run a controlled deployment. Environment-specific inventory and credentials remain outside the image.

3.1.2 Why teams containerize applications

Containerization creates a consistent unit of packaging and execution, but it does not remove every environmental dependency. The comparison below identifies where containers provide strong control and where platform or operational responsibilities remain.

Table 3-1. Containerization Benefits and Operational Tradeoffs

Workload	Benefit of a container	Important boundary
Python API or SSH client	Reproducible interpreter and packages	Device addresses and credentials remain external
Ansible execution	Fixed Ansible and collection versions	Inventory and secrets are mounted or retrieved at runtime
pyATS validation	Consistent Genie parsers and test dependencies	Parser support must match device output and release
Configuration rendering	Stable Jinja2 and schema libraries	Reviewed intent is supplied as a controlled input
CI runner job	Disposable job runtime	Runner access still controls reachable networks
Automation API and worker	Independent service scaling and upgrades	Queue and result state require external services
Mock device API	Repeatable offline integration tests	Mock behavior cannot prove real protocol operation

Containerization does not grant device reachability or make unsafe code safe. It standardizes the execution boundary.

3.1.3 Why containers help delivery

An application may behave differently across hosts because language runtimes, libraries, files, environment values, and operating-system packages differ. A container image captures many of these dependencies in a versioned artifact. The pipeline can build the image once, test that exact artifact, and promote it without rebuilding.

Containers improve consistency, but they do not eliminate environmental differences. Kernel behavior, CPU architecture, network policy, storage, secrets, and external services remain outside the image.

3.1.4 Containers and virtual machines

A virtual machine includes a guest operating system and runs through a hypervisor. A container normally shares the host kernel while receiving isolated process, network, filesystem, and resource views.

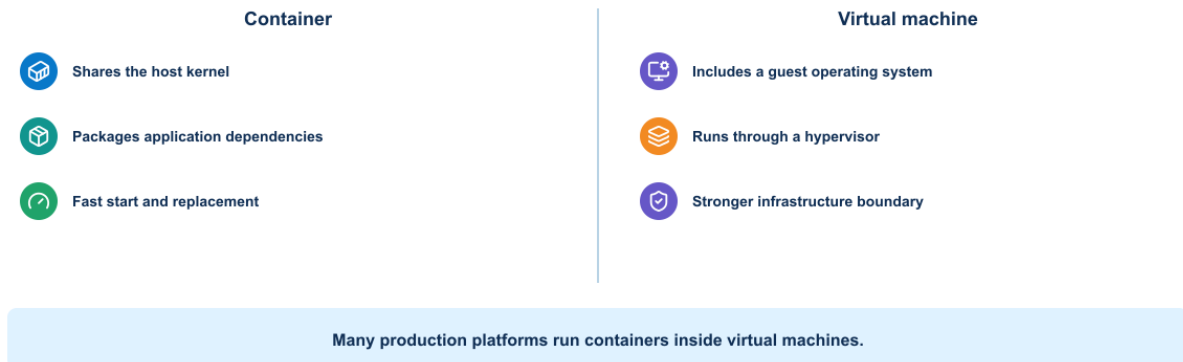


Figure 3-1. Comparison of container and virtual-machine runtime boundaries

Containers usually start faster and use fewer resources than virtual machines. Virtual machines provide a stronger boundary and can run a different guest kernel. Many platforms run containers inside virtual machines to combine infrastructure isolation with application packaging efficiency.

3.1.5 Linux foundations

Container isolation relies mainly on operating-system features:

- Namespaces provide separate views of processes, networking, mounts, hostnames, users, and other resources.
- Control groups account for and limit CPU, memory, and I/O use.
- Filesystem layers provide an efficient image and writable-container model.
- Capabilities divide privileged root operations into smaller units.
- Security profiles restrict system calls and resource access.

A container remains a process on the host. If it receives excessive privileges or access to the Docker socket, it can weaken the host boundary.

3.1.6 Docker architecture

The Docker architecture diagram answers a practical control question: which component receives the engineer's command, which component holds host-level authority, and where image distribution, runtime isolation, networking, and storage enter the path?

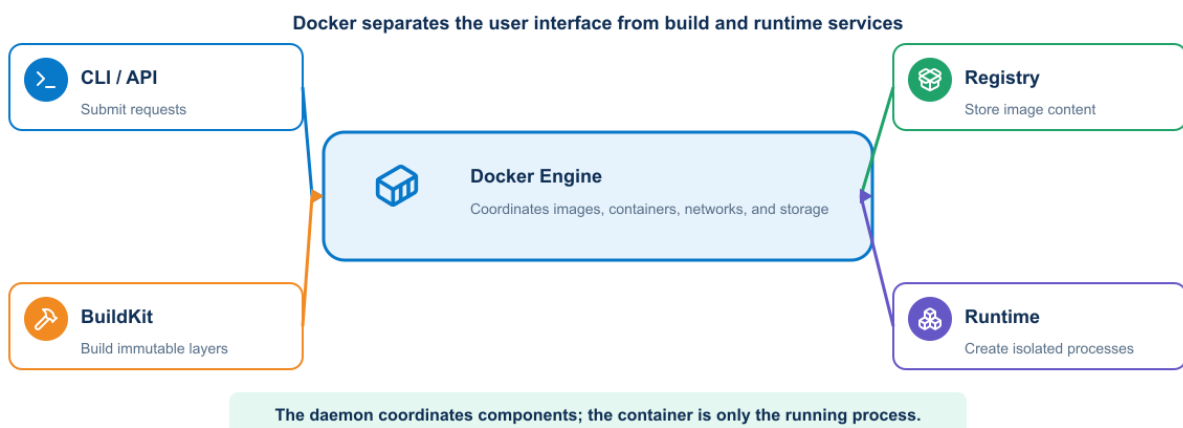


Figure 3-2. Docker client, Engine API, daemon, BuildKit, container runtime, registry, networks, and storage

The Docker client sends API requests to the daemon. The daemon coordinates the container runtime; manages images, networks, volumes, and containers; and communicates with registries that store and distribute images.

Important objects include:

- Image: immutable packaging template identified by content and metadata
- Container: runtime instance of an image with a writable layer
- Registry: service that stores repositories of images
- Volume: Docker-managed persistent data
- Bind mount: host path exposed inside a container
- Network: communication boundary connecting selected containers

For network automation, there are two distinct network planes:

- The Docker application network connects the automation API, worker, queue, database, and telemetry services.
- The management network connects an authorized worker or runner to network devices and controllers.

Joining a container to both planes creates a security boundary. The public API service does not need direct device access if it places an authorized job on a queue for a restricted worker.

3.1.7 Image references and identity

An image reference commonly contains a registry, repository, and tag. Tags such as `latest` are mutable labels and do not guarantee identical content over time. A digest identifies image content cryptographically.

Why this matters to a network engineer

The image digest identifies the exact Python runtime, client libraries, parsers, and Ansible content that executed a network job. Without it, operational evidence cannot prove which automation behavior produced a change.

Development workflows may use readable version tags. Promotion and controlled deployment should preserve the immutable digest or another verifiable identity. The application version, source commit, and image identity should remain traceable to one another.

3.1.8 Image layers and cache

The image lifecycle distinguishes reusable build content from a disposable container instance and its writable runtime layer.

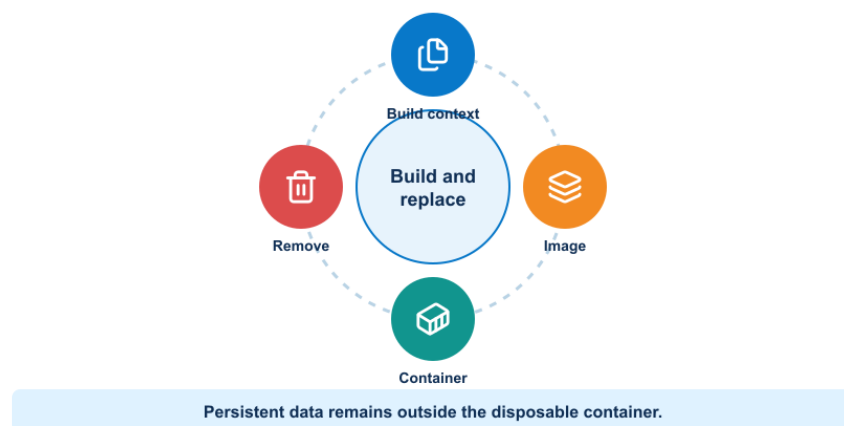


Figure 3-3. Lifecycle from Docker build inputs through immutable image layers and a disposable runtime container

Most Dockerfile instructions create layers. Docker can reuse unchanged layers during later builds. Layer order therefore affects build speed. Stable dependency installation usually belongs before frequently changing application source.

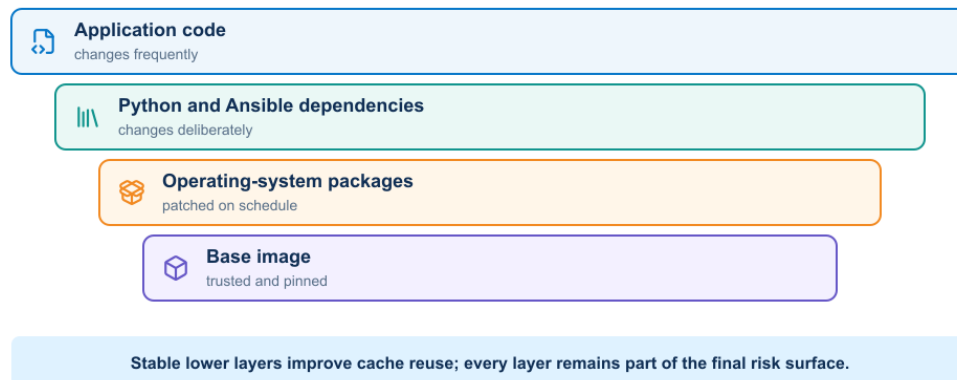


Figure 3-4. Container image layers arranged by expected rate of change and cache reuse

The cache affects efficiency, not correctness. A build process must declare all inputs and should not depend on accidental files remaining from an earlier build.

3.1.9 Container lifecycle

A container can be created, started, stopped, restarted, inspected, and removed. Stopping a container preserves its writable layer until removal, but important data should not depend on that layer. Deployment platforms replace containers routinely.

The application should respond predictably to termination signals, stop accepting new work when appropriate, finish or abandon work safely, and exit within the platform timeout.

3.1.10 Configuration and secrets

An image should contain application code and fixed runtime dependencies. Environment-specific configuration belongs outside the image. Common inputs include environment variables, mounted configuration files, platform configuration objects, and secret stores.

Do not bake passwords, tokens, certificates, or private keys into image layers. Removing a secret in a later Dockerfile instruction does not remove it from earlier layers.

3.1.11 Container storage

The writable container layer is temporary. Persistent data belongs in a volume, external database, object store, or another managed service.

Volumes offer Docker-managed storage and portability across container recreation on the same host. Bind mounts provide direct access to a host path and are useful for development, but they create a stronger host dependency and may expose sensitive files.

Applications should state clearly which data is persistent, which is cache, and which can disappear safely.

3.1.12 Container networking

The following boundary view explains why application-service connectivity and privileged management connectivity should not be treated as one undifferentiated container network.

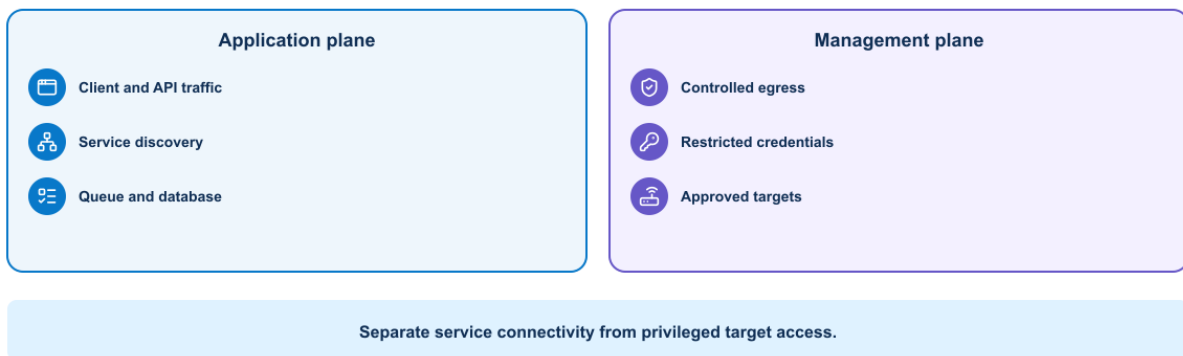


Figure 3-5. Separation of the application service network from the protected device-management network

Containers on a user-defined Docker network can normally resolve one another by service name. Applications should connect to the logical service name rather than a temporary container IP address.

Publishing a port maps a host address and port to a container port. Internal services such as databases normally do not need a public host port. Expose only the entry points that users or external systems require.

Network segmentation reduces unintended communication. The automation API, job worker, queue, database, dashboard, and telemetry collector do not all need identical reachability.

Management-network choices

A container can reach network devices through several patterns:

- Bridge networking with routed host reachability
- Host networking in a controlled lab, with reduced isolation
- A dedicated container network attached to a management VLAN
- A runner located inside the management zone
- A VPN established on the host, subject to Docker routing behavior

The team must test source addressing, DNS, MTU, firewall policy, certificate identity, and return routing. `ping` success does not prove that SSH, NETCONF, or RESTCONF works.

Avoid host networking as an unexplained fix. It removes a useful boundary and can create port conflicts.

3.1.13 Runtime inputs for network jobs

The container boundary is easiest to understand by separating fixed image content from values and state that must remain external.

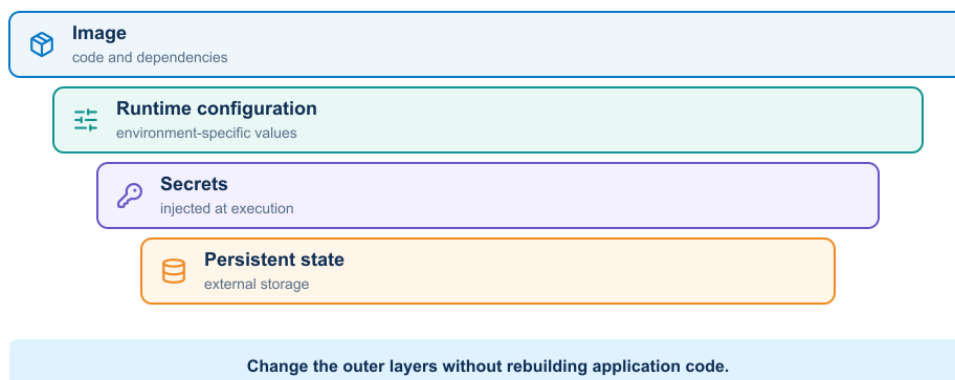


Figure 3-6. Application content inside an automation image and configuration, credentials, and state outside it

The container receives the minimum runtime inputs required for one job and writes durable results to an external destination. Rebuilding the image is not a configuration-management or secret-rotation mechanism.

The image should not contain live inventory or device credentials. A job receives:

- A reviewed intended-state file from the repository
- An inventory selected for the target environment
- A change identifier and commit SHA
- Short-lived credentials or a Vault reference
- Trusted CA material required for HTTPS
- An evidence-output location

Mount input files read-only where practical. Give the job a dedicated writable location for generated configuration and evidence. Never mount the entire engineer home directory or Docker socket without a documented requirement.

3.1.14 Persistent and ephemeral network data

Automation code and templates belong in the image or repository checkout. Generated candidate configuration, test reports, and backups are job artifacts. Job status, approval history, or scheduling data may belong in a database. Telemetry belongs in purpose-built storage.

Device backups can expose topology, usernames, addresses, and security configuration. Treat them as sensitive artifacts with controlled retention rather than ordinary container logs.

3.1.15 Device connection behavior inside containers

Automation tools must handle interactive and model-driven protocols correctly:

- SSH host-key validation should use a controlled known-hosts file.
- NETCONF commonly uses TCP 830 and exchanges server capabilities before RPCs.
- RESTCONF uses HTTPS and should validate the device certificate and hostname.
- Controller REST APIs may use tokens, sessions, pagination, and rate limits.
- Long-running collection needs explicit connect, read, and operation timeouts.

Container clocks must remain accurate because TLS, token expiration, telemetry timestamps, and evidence correlation depend on time.

3.1.16 Network automation container lifecycle

A pipeline job container should be disposable:

1. Receive immutable code and intended state.
2. Retrieve a scoped credential.
3. Confirm target identity.
4. Perform its one assigned operation.
5. Write structured evidence.
6. Revoke or release credentials.
7. Exit with a meaningful status.

Long-running API or worker containers follow service lifecycle rules and need readiness, graceful shutdown, queue handling, and durable result storage.

3.1.17 Resource and health considerations

Without resource boundaries, one container can consume enough CPU or memory to affect others. Production platforms need requests, limits, quotas, or equivalent controls based on observed behavior.

A running process is not necessarily a healthy service. Health checks should test a meaningful but inexpensive behavior. They should distinguish startup delay from ongoing failure and avoid creating excessive load.

3.1.18 Container tooling workflow

A disciplined Docker workflow includes:

1. Inspect the source and Dockerfile.
2. Build with an explicit version tag.
3. Review build output and base-image selection.
4. Inspect metadata and layer history.
5. Run with controlled configuration and networking.
6. Exercise the application.
7. Review logs, health, and resource behavior.
8. Stop and remove disposable resources.

Useful commands include `docker build`, `docker image inspect`, `docker history`, `docker run`, `docker ps`, `docker logs`, `docker exec`, `docker stats`, and `docker network inspect`.

Practical run pattern

This example keeps configuration outside the image, mounts it read-only, limits resources, removes unnecessary Linux capabilities, and gives generated evidence a dedicated writable location:

```
docker run --rm --name automation-check \
  --env-file ./lab.env \
  --mount type=bind,src="$PWD/config",dst=/app/config,readonly \
  --mount type=volume,src=automation-evidence,dst=/evidence \
  --read-only --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --cap-drop ALL --memory 512m --cpus 1 \
  registry.example/automation@sha256:APPROVED_DIGEST validate
```

This is a pattern, not a command to copy unchanged into production. The environment file must contain no long-lived device password, the digest must resolve in the chosen registry, and the application must support a read-only root filesystem. Because `--rm` deletes the stopped container, durable logs and reports must reach the evidence volume or a collector before exit.

3.2 Secure image packaging

3.2.1 Application image contents

The image should contain only the runtime components required by the application. In this course, that may include:

Table 3-2. Automation Image Contents and Runtime Inputs

Component	Purpose
Python	Data normalization, API clients, policy, and evidence processing
<code>requests</code> or <code>httpx</code>	REST and RESTCONF requests
<code>ncclient</code> or another supported client	NETCONF capability discovery and RPCs
Netmiko or Scrapli	Controlled SSH CLI collection and fallback configuration
Ansible and selected collections	Multi-device orchestration and idempotent modules
pyATS and Genie	Structured parsing and operational validation

Component	Purpose
Jinja2	Platform-specific configuration rendering
JSON Schema or typed models	Intended-state contract validation
CA certificates and SSH known-hosts tooling	Endpoint identity validation

Tool selection should remain narrow. Installing every vendor collection and parser increases build time, image size, vulnerabilities, and dependency conflicts.

Complex delivery systems benefit from several purpose-built images rather than one oversized automation toolbox. A Terraform image can contain the CLI, approved providers, certificate trust, and state-backend client required for resource lifecycle. An Ansible image can contain `ansible-core`, explicitly versioned network collections, inventory logic, and SSH trust tools. A pyATS image can contain the parsers and acceptance code required for independent verification. Keeping them separate limits dependency conflicts and makes the image digest evidence of the exact capability that executed a job.

Environment data remains outside these images. The same Ansible image should be able to operate against an authorized test or production target selected at runtime, while the same pyATS image evaluates the same frozen intent independently. Credentials are obtained only after the container starts and must not be retained in a layer or published artifact.

3.2.2 Dockerfile responsibilities

A Dockerfile records how the builder creates an image. It should make the runtime dependency boundary understandable and repeatable.

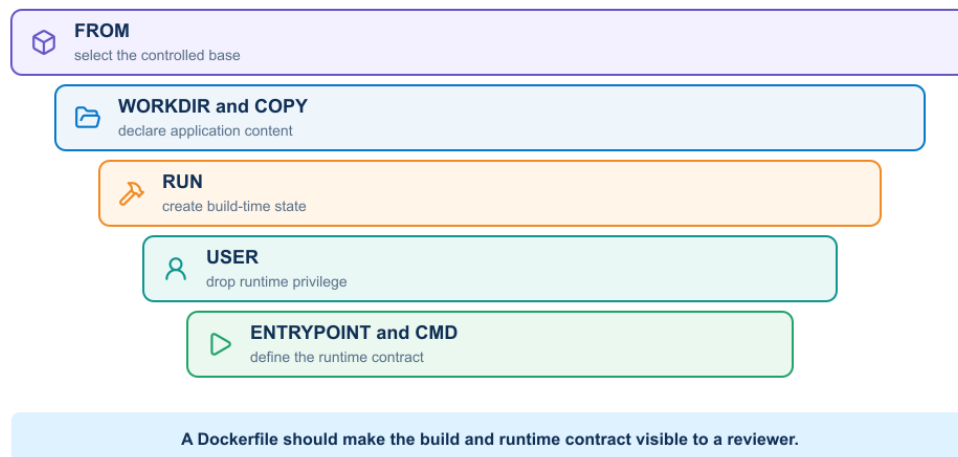


Figure 3-7. Layered view of the responsibilities expressed by common Dockerfile instructions

Common instructions include:

- `FROM` selects the base image.
- `WORKDIR` establishes a predictable directory.
- `COPY` adds declared files from the build context.
- `RUN` executes build-time commands.
- `ENV` defines non-sensitive default environment values.
- `USER` selects the runtime identity.
- `EXPOSE` documents an intended listening port.
- `HEALTHCHECK` can define container-level health behavior.
- `ENTRYPOINT` and `CMD` define the executable and default arguments.

The Dockerfile should reveal the runtime contract without requiring hidden manual changes.

3.2.3 Base-image selection

A good base image supports the required runtime, processor architecture, patch process, and operational tools while minimizing unnecessary software.

A very small image is not automatically the safest choice. The team must be able to patch, scan, troubleshoot, and support it. Pinning a digest improves reproducibility, but the team still needs a process to review and adopt patched base images.

Questions for base-image review include:

- Who maintains the image?
- How quickly does it receive security updates?
- Which operating-system packages does it include?
- Does it support the target architecture?
- Can the scanning and runtime platforms inspect it correctly?
- How will the project detect that the pinned base has become outdated?

3.2.4 Build context and `.dockerignore`

The build context contains files available to `COPY` and `ADD`. A broad context can send credentials, Git history, test output, local databases, or large development directories to the builder.

`.dockerignore` should exclude at least local environment files, `.git`, virtual environments, caches, logs, test artifacts, editor state, and private keys. The exact list depends on the project.

Exclusion protects both build efficiency and information security. A file does not need to appear in the final filesystem to create risk; build systems may preserve context or intermediate layers.

3.2.5 Dependency control

Reproducible builds need declared dependency versions. Loose ranges can cause identical source commits to produce different images at different times. Fully pinned dependencies improve repeatability but require an update process.

For Python, the project separates human-reviewed top-level requirements from a generated lock or constraints file. The pipeline should verify dependency integrity and report known vulnerabilities without silently changing versions during a release build.

Ansible collections also need controlled versions. A change in a vendor collection, `ansible.netcommon`, or a parser can alter commands, return data, or supported parameters. Record collection versions beside Python dependencies and test upgrades against representative device output.

Offline parser fixtures reduce risk. Store sanitized `show interfaces`, `show ip ospf neighbor`, and `show ip route` samples in tests so a dependency update can reveal changed structured output before a live device job.

3.2.6 Layer design

Layer design affects cache behavior, image size, and secret exposure. Copy dependency declarations and install dependencies before copying frequently changing application code. Remove package-manager caches in the same layer that creates them.

Do not combine unrelated actions merely to minimize the number of layers. Readability and predictable behavior matter more than a superficial layer count.

3.2.7 Multistage builds

A multistage Dockerfile separates build tools from the final runtime. The builder stage can compile code or install dependencies. The final stage receives only the artifacts required to run.

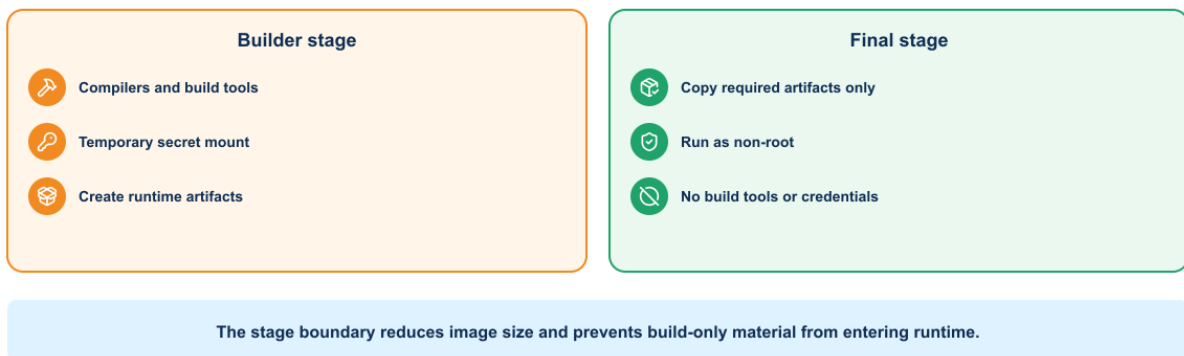


Figure 3-8. Builder and final runtime stages separated by an explicit artifact boundary

This approach can reduce size and attack surface. It also makes the boundary between build-time and runtime dependencies explicit. The final stage still needs certificates, timezone data, shared libraries, and other runtime components required by the application.

3.2.8 Runtime user and privileges

Applications should run as a non-root user unless a documented requirement prevents it. File ownership and port selection must support that user. Binding to high-numbered ports avoids unnecessary privilege.

At runtime, remove Linux capabilities that the application does not need, avoid privileged mode, use a read-only root filesystem when practical, and mount only required writable locations.

3.2.9 Secrets during builds

Passing a secret through `ARG`, copying it into the context, or embedding it in a `RUN` instruction can expose it through layers, build history, logs, or metadata.

When a private dependency requires authentication, use the builder's secret-mount capability or a short-lived credential mechanism that does not persist in the final image. The pipeline should mask secrets in output and avoid commands that print environment values.

Do not copy SSH private keys, NETCONF usernames, RESTCONF passwords, controller tokens, Vault tokens, GitLab deploy tokens, or lab `.env` files into the image. A later `RUN rm` cannot erase a value preserved in an earlier layer.

3.2.10 Image metadata and traceability

The evidence-lineage view asks whether an operator can trace a deployed runtime and its results back to the exact source, dependencies, build, tests, and approved digest.

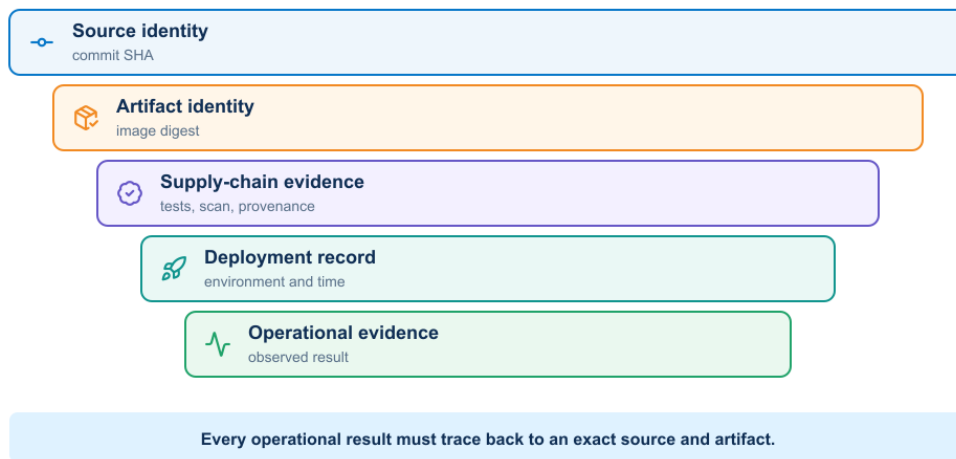


Figure 3-9. Evidence lineage from source and dependencies to the deployed automation image and post-change results

OCI labels can record the source repository, source revision, version, description, authorship, and license. The pipeline should apply a tag derived from the release version or commit and record the resulting digest.

This is the build-and-artifact portion of the delivery architecture introduced in Module 1. The protected worker should receive the resulting digest and evidence; it should not rebuild the application inside the management trust zone.

Traceability should answer:

- Which source commit produced this image?
- Which pipeline built and tested it?
- Which base image and dependencies were used?
- Which environments deployed this digest?

3.2.11 Testing an image

Testing should cover more than successful process startup. A build workflow can perform:

- Dockerfile linting
- Dependency and source tests
- Image build
- Vulnerability and secret scanning
- Metadata and configuration inspection
- Non-root execution check
- Health endpoint test
- Read-only filesystem or capability test where required
- Software bill of materials generation

A scanner finding requires interpretation. Severity, exploitability, exposure, available remediation, and application context influence the decision. Suppression should include an owner, justification, and expiration.

Network-specific runtime tests

The pipeline should also verify that:

- The container runs as a non-root UID.
- The intended-state schema accepts the approved fixture and rejects invalid VLANs and prefixes.
- Jinja2 rendering is deterministic.
- The automation can parse sanitized fixtures from the supported network operating systems.

- Read-only mock RESTCONF and NETCONF tests succeed.
- Deployment modules are absent from untrusted validation images if the team uses separate images by privilege.
- The runtime trusts only approved CA material and SSH host keys.

Separating a validation image from a deployment image can reduce capability. The validation image needs schemas, linters, renderers, and offline tests. The deployment image also needs device clients and may run only on a protected runner.

3.2.12 Image signing and provenance

Signing allows a consumer to verify who approved an image. Build provenance records information about how the artifact was produced. These controls are most useful when the deployment platform enforces them. A signature stored but never verified provides limited protection.

The delivery process should retain and promote an already tested image digest. Rebuilding from the same source for another environment creates a different artifact and breaks the evidence chain.

3.2.13 Image supply-chain evidence

Each output has a purpose. The SBOM inventories components. The vulnerability scan compares those components with known findings. A signature binds an identity to the digest. Provenance describes how the build occurred. None of these controls substitutes for the others.

The supply chain below shows that evidence is attached to the immutable digest before it reaches a registry. The deployment verifies that identity rather than rebuilding or resolving a mutable tag.

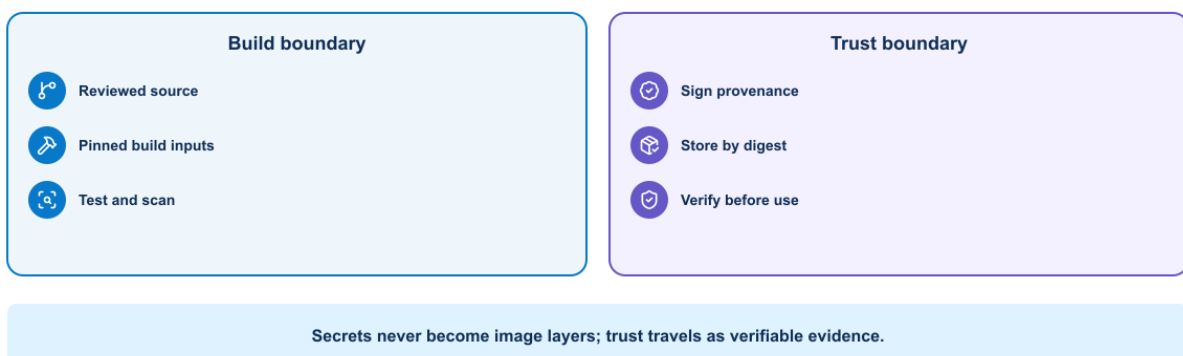


Figure 3-10. Secure image supply chain with build evidence and a prohibited secret path

Secrets may be exposed temporarily through a supported build-secret mechanism when private dependencies require them, but they must not enter the build context, layer history, final image, or provenance output.

3.2.14 Compromised dependency scenario

Assume a new parsing package executes unexpected code during installation. If the build job can reach the management network or access deployment variables, the dependency can steal credentials before an image is created.

Build isolation therefore matters as much as runtime hardening. The untrusted dependency-resolution stage should not receive network device secrets or management-plane access. Protected deployment occurs later with the already scanned and identified image.

3.2.15 Registries and retention

A registry should enforce authentication, encrypted transport, access control, immutability where appropriate, scanning, and retention policy. Developer accounts may push to development repositories, while production promotion should use a controlled service identity.

A registry is not the same as a pipeline artifact store. The registry distributes versioned packages or OCI images to later environments; pipeline artifacts retain job outputs such as test reports, rendered candidates, SBOMs, and deployment evidence. An image digest belongs in both records so that a reviewer can connect the distributable runtime with the evidence that qualified it.

Promotion should normally copy or authorize the existing digest rather than rebuild it. Repository paths and tags can express release channels, but access policy must prevent an unreviewed job from overwriting or relabeling a production identity. Pull permissions should also be narrower than they first appear: a runner that can read every private image may expose embedded intellectual property or vulnerable historical releases even when it cannot push.

Retention must balance storage cost, investigation needs, and rollback. Removing every previous image immediately can make recovery impossible.

Packaging failure patterns

The image review should connect a defect to its operational consequence rather than report only that a rule failed.

Table 3-3. Packaging Failure Modes and Controls

Failure pattern	Why it matters	Control and evidence
Mutable base tag	A rebuild can change without a source commit	Pin and record the base digest; use a scheduled update workflow
Unlocked Python or collection dependency	Identical source can resolve to different behavior	Generate a reviewed lock or constraints file and test dependency updates
Secret copied during an early layer	Later deletion does not remove it from image history	Exclude it from context and use an ephemeral build-secret mechanism
Root runtime user	Application compromise gains unnecessary container privileges	Create a numeric non-root UID and test the effective identity
Build tools retained in the final stage	Attack surface and vulnerability count increase	Use a multistage build and inspect installed packages
Release rebuilt after testing	Production bytes are not the tested bytes	Promote the original digest and verify it at deployment
Scan exception without expiry	Temporary risk acceptance becomes permanent	Record owner, justification, affected digest, compensating control, and expiry

These controls do not guarantee that the application behaves correctly. They establish that the team knows what it built, can reproduce the decision, and can reject an artifact whose identity or evidence changes.

3.2.16 Example network automation packaging pattern

The following Dockerfile brings the preceding controls together in a small packaging pattern. Read it as an example of deliberate build decisions—base image, dependency installation, ownership, and runtime identity—not as a production template that can be copied without review.

```
FROM python:3.13-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN python -m venv /opt/venv \
    && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

FROM python:3.13-slim
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONUNBUFFERED=1
RUN useradd --create-home --uid 10001 appuser
COPY --from=builder /opt/venv /opt/venv
WORKDIR /app
COPY automation/ ./automation/
COPY intended-state/schema/ ./intended-state/schema/
COPY templates/ ./templates/
COPY validation/ ./validation/
USER 10001
ENTRYPOINT ["python", "-m", "automation.python.cli"]
CMD ["--help"]
```

This example illustrates separation of build and runtime stages, dependency caching, a non-root identity, and an explicit entry point. A real project should pin the base digest, control dependency versions, add health behavior, and validate the image in CI.

Review example: a harmless-looking dependency change

Suppose a merge request changes only `requirements.txt`. Source tests pass, but the lock file now pulls a new transitive SSH library. The reviewer should ask three separate questions: did application behavior change, did the runtime inventory change, and does the new component alter the security boundary? A defensible pipeline rebuilds from a clean context, compares the SBOM with the previous release, runs connection and parser fixtures, scans the resulting digest, and records an approved exception if a finding cannot yet be fixed. Reusing an old scan report would miss the exact risk introduced by the dependency-only change.

3.3 Multitier application deployment

One container solves runtime reproducibility for one process. It does not define the relationships among an API, worker, queue, database, telemetry collector, and dashboard. The next engineering requirement is to describe those cooperating processes as one application while preserving separate responsibilities, failure boundaries, networks, and state.

3.3.1 Multitier service architecture

The architecture separates request handling from privileged execution so that only the worker crosses into the management network and every job retains durable state and evidence.

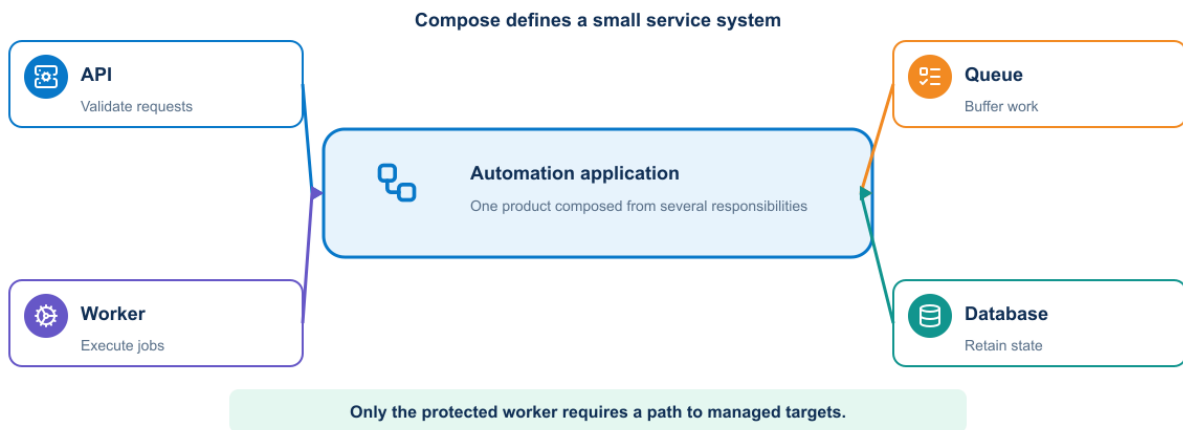


Figure 3-11. Docker Compose architecture for the network automation API, worker, supporting services, and network targets

Separating responsibilities allows each service to change, scale, recover, and receive access control independently. The course stack contains an automation API, job worker, queue, job database, telemetry collector, and dashboard.

The worker is the only service that needs direct management-plane access. The API accepts a reference to reviewed intent and an approved operation. It must not accept arbitrary CLI commands from callers.

3.3.2 Service contracts

This sequence highlights why the public API does not need a route or credential to a device. Privileged access begins only in the protected worker after a validated job reaches the queue.

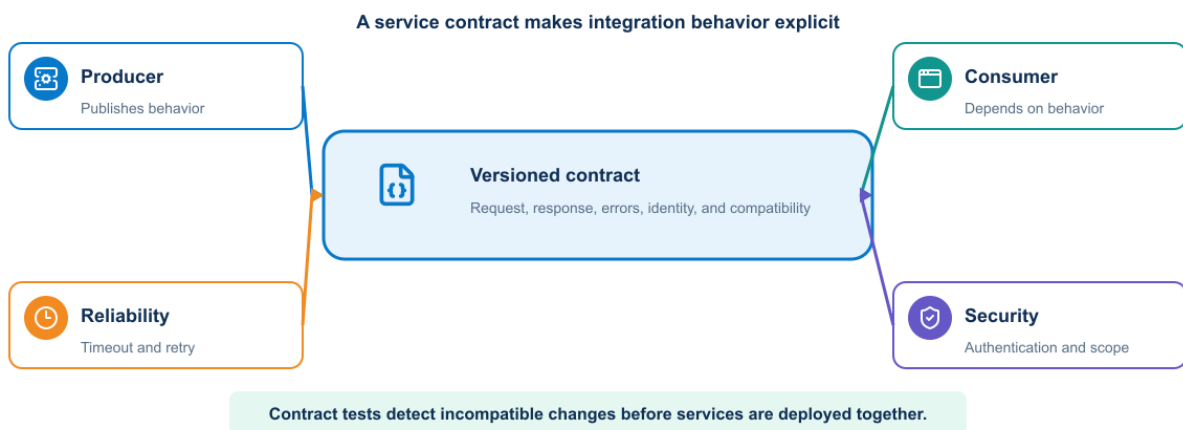


Figure 3-12. Service contract connecting producer, consumer, reliability, and security responsibilities

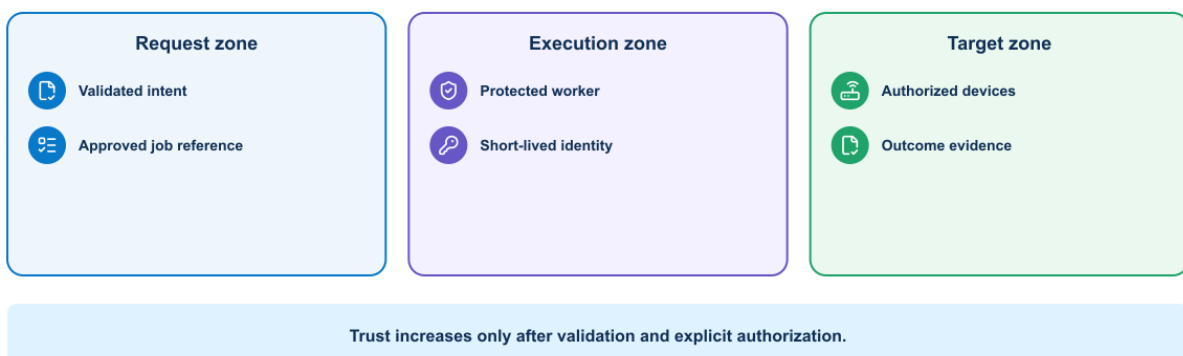


Figure 3-13. Approved job sequence across API, queue, protected worker, secret service, and device

The worker returns structured evidence and final status through the application boundary. The credential is scoped to the job and is never placed on the queue.

Each service needs an explicit contract:

- Protocol and port
- Addressing or discovery method
- Authentication and authorization requirements
- Request and response format
- Timeout and retry behavior
- Health behavior
- Version compatibility
- Data ownership

Containers change IP addresses when recreated. Consumers should use stable service names and declared ports rather than fixed container addresses.

Job contract

A worker and its callers need an explicit agreement about the data exchanged between them. This compact JSON example illustrates a job request that can be validated, queued, processed, and correlated with later evidence.

```
{
  "change_id": "CHG-2026-0042",
  "commit_sha": "0123456789abcdef",
  "intent_path": "requests/compliance-intent.yml",
  "inventory": "lab",
  "operation": "precheck",
  "requested_by": "gitlab-pipeline-1842"
}
```

The API rejects arbitrary local paths, unknown inventories, unsupported operations, or missing change identifiers. Deployment requires a protected pipeline claim or separate approval. The queue contains references and non-secret metadata, not device passwords.

3.3.3 Docker network model

Docker Compose normally creates a user-defined network for the project. Services on the network can resolve one another by service name. The application can connect to `database:5432`, for example, while clients reach the published frontend port.

Publishing a port makes a service reachable through the host. [EXPOSE](#) documents a container port but does not publish it. A database should normally remain on an internal network unless an authorized external consumer requires access.

Larger designs can use separate frontend and backend networks. The proxy joins the frontend and application networks. The application joins the application and data networks. The database joins only the data network.

3.3.4 Docker Compose model

A Compose file defines a related application stack. Main elements include:

- `services` for runtime components
- `networks` for communication boundaries
- `volumes` for persistent data
- `configs` and `secrets` where supported
- Environment values and mounted files
- Health checks and dependency conditions
- Resource and restart behavior

Compose is useful for development, integration testing, demonstrations, and smaller deployments. Kubernetes provides a broader orchestration model for clustered operation.

Illustrative Compose structure

The following Compose definition shows how the application tiers can be declared as one system while retaining separate runtime responsibilities. The values are intentionally minimal so that the service relationships remain visible.

```
services:
  automation-api:
    image: registry.example/network-devops:${AUTOMATION_IMAGE_TAG}
    command: ["serve-api"]
    networks: [api, services]
    ports: ["127.0.0.1:8080:8080"]
    depends_on:
      queue:
        condition: service_healthy

  worker:
    image: registry.example/network-devops:${AUTOMATION_IMAGE_TAG}
    command: ["run-worker"]
    networks: [services, management]
    read_only: true
    volumes:
      - evidence:/evidence
      - ./ssh/known_hosts:/etc/ssh/ssh_known_hosts:ro

  queue:
    image: redis:7-alpine
    networks: [services]
    volumes: [queue-data:/data]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  job-db:
    image: postgres:17-alpine
    networks: [services]
    volumes: [job-data:/var/lib/postgresql/data]

networks:
  api: {}
  services:
    internal: true
  management:
    external: true

volumes:
  queue-data: {}
  job-data: {}
  evidence: {}
```

This illustrates architecture rather than a complete production secret design. Live database credentials must come from a protected runtime source. The external management network must already exist and have an explicit security policy.

3.3.5 Configuration separation

The repository can contain safe defaults and `.env.example`, but it should not contain live secrets. Values vary by environment:

- External URL and listening port
- Database hostname and database name
- Log level
- Feature switches
- Credential references
- Monitoring endpoints

Configuration should be validated at startup. A service should fail clearly when a required value is missing instead of using an unsafe fallback.

Network-specific configuration includes inventory name, permitted device groups, protocol timeouts, concurrency limit, trusted CA path, SSH known-hosts path, evidence retention class, and read-only or change mode. A worker should not default to all devices or to change mode.

3.3.6 Persistent state

The database service needs a volume or external storage. Application containers should remain replaceable. Backup, restore, schema migration, and data ownership require explicit design.

A volume preserves data when a container is recreated, but it is not a backup. The same host failure can affect both the container and its local volume.

Schema changes must remain compatible with the deployment sequence. A destructive migration performed before new application instances become healthy can make rollback impossible.

The database does not replace Git as the source of intended state. It records runtime jobs and evidence relationships. Git stores reviewed intent and automation definitions. A queue holds temporary coordination state. Telemetry storage holds time-series observations.

3.3.7 Startup order and readiness

Process order does not prove service readiness. A database process may start before it accepts connections or completes recovery. The application should retry appropriate transient connections with a bounded policy.

Compose health checks can provide readiness evidence. Dependencies can wait for a healthy state, but the application still needs resilient connection behavior because services can fail after startup.

3.3.8 Timeouts and retries

Every remote call needs a timeout. An unlimited wait can exhaust threads or connections. Retrying can help with transient failures, but an aggressive retry loop can amplify overload.

Use bounded retries with delay and jitter where appropriate. Do not retry permanent failures such as invalid credentials or malformed requests without changing the input. Non-idempotent operations require special care because a repeated request might duplicate work.

3.3.9 Reverse proxy role

A reverse proxy can provide a stable client endpoint, TLS termination, request limits, static content, and routing. It should forward useful request identifiers and preserve client information according to the trust model.

The proxy health check and application health check answer different questions. Proxy availability does not prove that the application or database works.

In a production automation platform, the entry layer can terminate TLS and enforce authentication, request-size limits, and rate limits. It must not allow unauthenticated access to an endpoint that starts device changes.

3.3.10 Worker safety and concurrency

A worker processes network jobs. High concurrency can overwhelm device management planes or create conflicting changes. Use per-device locking and a low default concurrency. Jobs that touch the same routing domain may need a broader lock or ordered plan.

Operational Consideration

Increasing worker replicas also increases potential device concurrency. Application scaling and network blast radius therefore require separate limits; queue depth alone is not a safe scaling signal.

The worker verifies the commit and image digest associated with the job, retrieves scoped credentials only when needed, and discards them after use. Queue redelivery requires idempotent design or a guard that prevents a partially completed deployment from running twice.

3.3.11 Mock and real targets

Compose can provide a mock REST API or recorded-response service for offline pipeline development. This supports schema, error handling, pagination, and evidence tests. It cannot prove device configuration semantics, control-plane convergence, or end-to-end forwarding.

The course progresses from mock validation to an authorized virtual or sandbox device. The pipeline labels evidence with the target type so reviewers do not confuse simulated success with real operational validation.

3.3.12 Health model

The readiness chain shows why a healthy API process does not prove that a queued network job can pass through every dependency and complete successfully.



Figure 3-14. Readiness and failure behavior across the automation service dependency chain

A multitier application benefits from several health views:

- Liveness: whether a process needs restart
- Readiness: whether it can accept traffic now
- Dependency status: whether required downstream services work
- Functional check: whether a small end-to-end action succeeds

Do not make liveness depend on every remote service. A database outage could cause every application container to restart continuously, adding load without repairing the database.

3.3.13 Logging across services

Services should write structured logs to standard output or another supported collection path. Include timestamp, severity, service, version, environment, and request or correlation identifier. Do not log passwords, tokens, private data, or full sensitive payloads.

A correlation identifier allows the team to follow one request from the proxy through the application and database interaction.

3.3.14 Failure analysis

The following view separates the main failure domains in the stack.



Figure 3-15. Failure boundaries across request, execution, and dependency paths in a Compose application

Troubleshoot the stack from boundaries inward:

1. Confirm the expected containers exist and inspect their state.
2. Check health status and recent logs.
3. Confirm network membership and name resolution.
4. Test the application inside its network before testing the published endpoint.
5. Verify configuration without displaying secrets.
6. Check storage permissions and database readiness.
7. Reproduce the failing request with a request identifier.

`docker compose ps`, `docker compose logs`, `docker inspect`, `docker network inspect`, and targeted `curl` requests provide useful evidence.

Failure boundaries and controls

A multitier incident should be classified at the boundary that failed. Restarting the entire stack destroys useful evidence and may amplify the original problem.

Table 3-4. Compose Application Failure Boundaries and Controls

Symptom	Likely boundary	Evidence to collect	Appropriate control
API answers but jobs remain queued	API-to-queue or queue-to-worker	Request ID, queue depth, consumer state, worker logs	Contract test, queue health, bounded redelivery
Worker starts but cannot reach targets	Worker-to-management network	Worker route, DNS, firewall decision, endpoint TLS/SSH identity	Dedicated network attachment and explicit egress policy
Database container is running but API is unready	Application-to-database	Readiness result, connection error, migration status	Dependency-aware readiness and bounded connection retry
Job runs twice after worker restart	Queue acknowledgement and job-state ownership	Delivery count, job state transitions, operation identifier	Idempotency key, per-target lock, uncertain-state handling

Symptom	Likely boundary	Evidence to collect	Appropriate control
Previous application cannot start after rollback	Application-to-schema compatibility	Migration version, application error, release history	Expand-and-contract migration or forward remediation
Published endpoint works locally but not remotely	Host publishing, firewall, or proxy	Bound address, port mapping, proxy and firewall logs	Explicit exposure and end-to-end health check

The operating consequence determines severity. A failed dashboard query is different from a duplicated privileged job even when both appear as an HTTP error.

Practical failure: the API is healthy but no job completes

An HTTP 200 response from the API proves only that the request-facing process can answer. If jobs remain in `queued`, follow the job identifier across boundaries: confirm that the API published the message, inspect queue depth, check that the worker subscribed to the expected queue, and test management reachability from the worker namespace. A common cause is attaching the API to the published network while forgetting to attach the worker to the external management network. Restarting every container may hide the symptom without correcting the topology. The durable fix is a correct Compose network declaration plus an integration test that exercises one queued, read-only job.

3.4 Kubernetes orchestration and multidata-center operation

Compose can describe and operate the complete application on a small platform. It does not by itself provide a multi-node scheduler, automatic workload replacement across nodes, standardized rolling updates, cluster-wide resource placement, or a broad policy API. When scale, availability, scheduling, and platform-governance requirements justify that operational cost, the team can introduce Kubernetes. A protected runner or Compose deployment remains valid when those requirements do not exist.

3.4.1 Platform architecture

The platform view identifies which Kubernetes workloads need ordinary service connectivity and which worker path requires tightly controlled access to managed infrastructure.

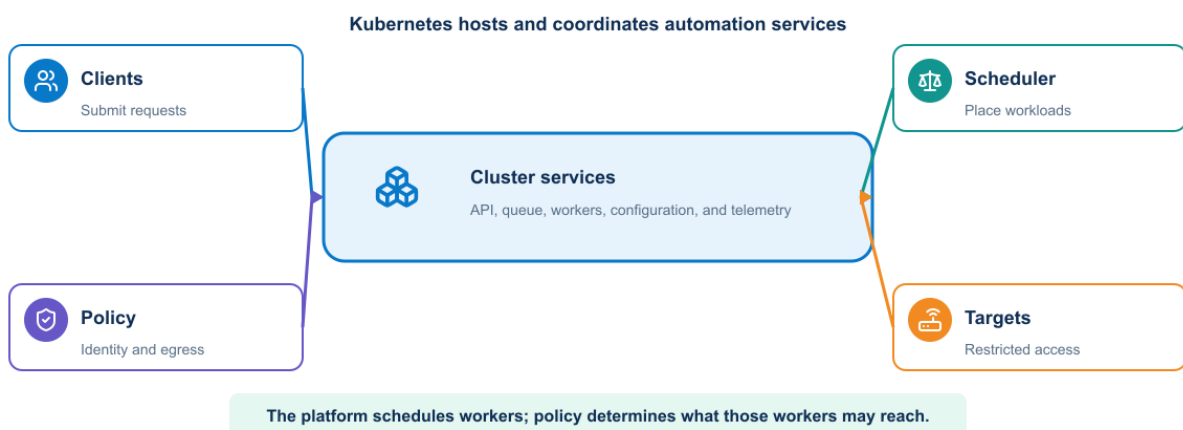


Figure 3-16. Kubernetes-hosted network automation platform and restricted management paths

Only job workers need network-device access. API, dashboard, and general validation Pods should not share that route by default. Kubernetes NetworkPolicy, external firewalls, worker placement, and service-account policy work together to enforce the design.

3.4.2 Should the team use Kubernetes?

The decision flow tests whether orchestration capabilities solve an actual operating requirement or merely add a larger platform and security burden.



Figure 3-17. Decision flow for choosing Kubernetes, Compose, or a protected runner for network automation

Kubernetes is useful when the platform needs several independently operated services, concurrent workers, declarative rollout, self-healing, workload scheduling, standardized observability, or integration with an existing organizational cluster platform.

It may be unjustified when:

- One team runs a small number of scheduled jobs.
- Docker Compose or a managed runner already meets availability needs.
- The team lacks cluster operations and security capability.
- Direct management-network routing from cluster workers is difficult to secure.
- Stateful queue, database, and evidence services would become less reliable.
- Platform complexity would exceed the value of automated scaling.

Table 3-5. Container Platform Selection Criteria

Requirement	Compose or runner	Kubernetes
Single learning workstation	Strong fit	Useful only for learning Minikube concepts
Few sequential network jobs	Simple and sufficient	Often excessive
Many isolated workers	Manual orchestration required	Deployment and Job patterns can help
Rolling platform updates	Basic service recreation	Native Deployment behavior
Existing enterprise cluster platform	Integration work required	Can reuse platform controls
Strong network segmentation to device zones	Host/firewall design	Cluster networking plus external controls
Team operating burden	Lower	Higher control-plane and policy complexity

The course uses Minikube to teach the model. It does not claim that the production network automation platform must use Kubernetes.

3.4.3 Three valid platform architectures

The comparison separates three legitimate operating models so that Kubernetes is evaluated against simpler alternatives rather than assumed to be the target state.



Figure 3-18. Comparison of a protected runner, Docker Compose platform, and Kubernetes automation platform

Table 3-6. Protected Runner Compose and Kubernetes Comparison

Factor	Protected GitLab runner and scripts	Docker Compose platform	Kubernetes platform
Best fit	Few scheduled or approved jobs	Small always-on API and worker service	Multiple services/pools with established cluster operations
Scale	Limited by runner and explicit concurrency	Vertical scale and manually managed workers	Declarative replicas, Jobs, scheduling, and quotas
Availability	Runner recovery or replacement	Host-level design and service restart	Multi-node scheduling if dependencies are also resilient
Worker isolation	Host/process/container controls	Separate services and Docker networks	Namespace, node, Pod, identity, RBAC, policy, and external firewall controls
Device reachability	Directly designed on runner host	Restricted worker bridges service and management networks	Only selected worker nodes/Pods receive controlled egress
Security burden	Runner hardening and secret delivery	Adds API, queue, database, and service trust	Adds cluster API, admission, RBAC, node, supply-chain, and CNI policy
Operational burden	Lowest	Moderate	Highest
Team skills	GitLab, Linux, scripting	Container and service operations	Kubernetes platform, networking, security, storage, and incident response

Kubernetes solves automation-platform scheduling and lifecycle problems. It does not validate network intent, discover the correct device, constrain a routing-domain blast radius, or prove forwarding health.

3.4.4 Kubernetes worker-to-device security

The security path shows the controls required between a validated queue item and an explicitly authorized device when a worker executes inside a cluster.

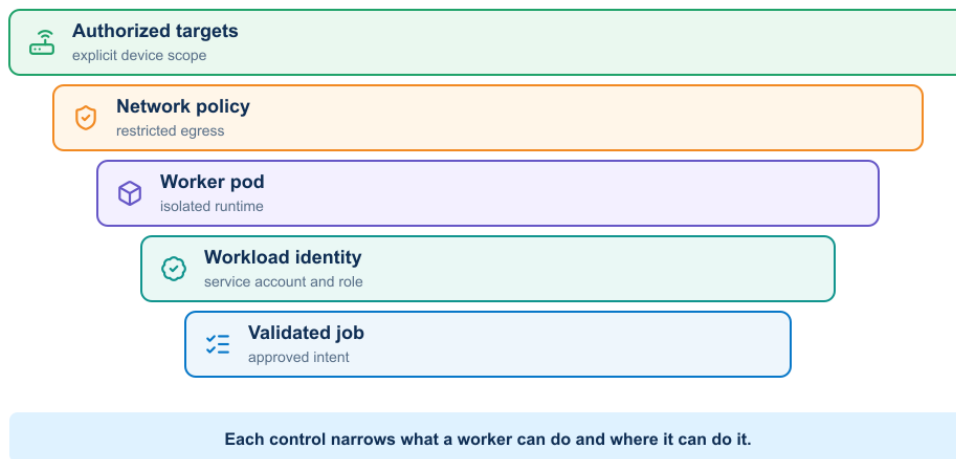


Figure 3-19. Kubernetes security controls from validated queue input through an isolated worker to explicitly authorized devices

NetworkPolicy controls Pod traffic only when the cluster networking implementation enforces it; it does not replace the external management firewall or device AAA. General API, dashboard, and validation workloads should have no device route. A worker receives a validated job, signed image, dedicated service account, short-lived credential, narrow egress rule, explicit target list, and independent evidence destination.

3.4.5 Kubernetes model

Users submit desired state to the Kubernetes API. Controllers observe stored state and work to make actual state match it. This reconciliation loop is central to the platform.

Kubernetes does not simply run a list of commands. A Deployment declares the desired application image, replica count, selection labels, update behavior, and Pod template. Controllers continuously reconcile that declaration.

3.4.6 Cluster architecture

The architecture separates reconciliation responsibilities from workload execution.

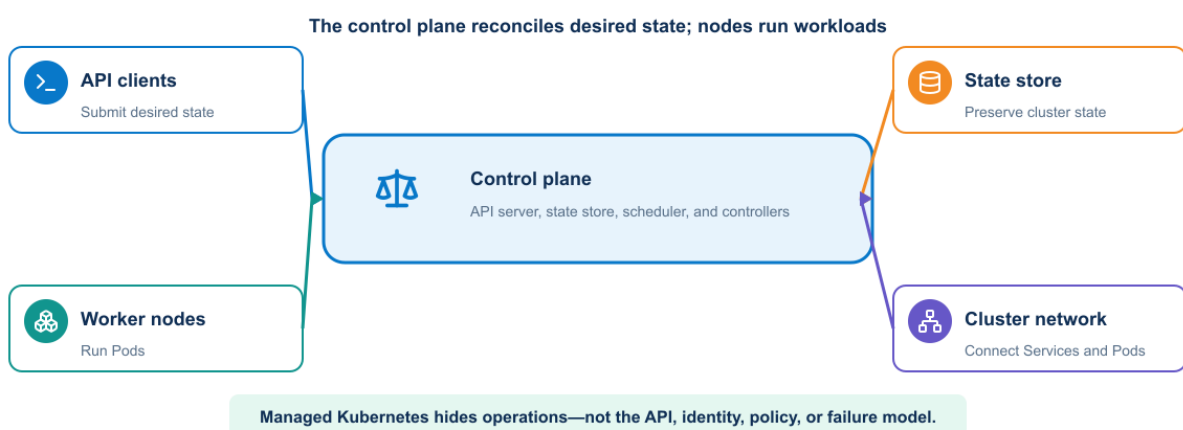


Figure 3-20. Kubernetes control plane, worker nodes, API clients, state store, and cluster network

The control plane includes:

- API server, which provides the main management interface
- State store, which preserves cluster configuration and state
- Scheduler, which assigns unscheduled Pods to nodes
- Controllers, which reconcile resources

Worker nodes run:

- A node agent that manages Pod lifecycle
- A container runtime
- Networking components that implement Service and Pod connectivity

Managed services may hide control-plane operation, but users still need to understand API behavior, identity, quotas, networking, and failure boundaries.

3.4.7 Core objects

Kubernetes expresses application intent through a set of related API objects. Understanding the responsibility and lifecycle of each object is essential before combining them into a deployment design.

Namespace

A Namespace groups resources and supports names, access control, quotas, and policy. It does not replace all security isolation.

Pod

A Pod is the smallest schedulable unit. Its containers share a network identity and selected storage. Pods are replaceable; clients should not depend on a Pod IP remaining stable.

Deployment and ReplicaSet

A Deployment manages stateless application replicas and controlled updates. It creates ReplicaSets, which maintain the requested number of matching Pods.

The automation API and validation service fit a Deployment. Long-running workers may also use a Deployment. A one-time change execution may fit a Kubernetes Job, but the platform must prevent job retry from repeating a partial network change.

Service

A Service provides stable discovery and traffic distribution for selected Pods. Selectors and labels connect the Service to the intended workload.

ConfigMap and Secret

A ConfigMap stores non-sensitive configuration. A Secret stores sensitive data in a Kubernetes object. Secret protection still requires encryption at rest, RBAC, careful mounting, and safe application behavior.

A deployment can use a ConfigMap for permitted inventory references, protocol timeouts, and feature settings. Network credentials should preferably arrive through workload identity or an external secret integration. Storing a long-lived device password in a manifest, even if base64-encoded, is unsafe.

Ingress or gateway

An ingress or gateway layer routes external traffic according to platform-specific implementation. TLS, hostname, path, and policy configuration depend on the selected controller.

PersistentVolume and claim

A PersistentVolume represents storage. A PersistentVolumeClaim requests storage with defined capacity and access behavior. Stateful applications also need backup, recovery, identity, and update design.

3.4.8 Manifests and API use

A manifest normally contains `apiVersion`, `kind`, `metadata`, and `spec`. Labels identify and group objects. Selectors create relationships and must remain consistent.

`kubectl` is a client for the Kubernetes API. It uses a context containing cluster, user, and optional namespace. Before any change, confirm the current context and namespace.

Declarative application with version-controlled manifests supports review and repeatability. Server-side defaults and admission policies can affect the stored object, so inspect the result.

3.4.9 Scheduling and resources

Resource requests influence scheduling and reserve capacity. Limits constrain use according to resource type and runtime behavior. Missing requests can cause poor placement. Unrealistic limits can create throttling or termination.

Placement can also use node selectors, affinity, anti-affinity, taints, tolerations, and topology constraints. These policies should express availability or hardware requirements without making workloads impossible to schedule.

Network workers may require nodes attached to a protected management zone. Label those nodes and use placement policy, but remember that anyone who can schedule an arbitrary Pod there may gain the same network path. Admission and RBAC must restrict workload creation.

3.4.10 Probes

Kubernetes assigns a different control decision to each probe type.

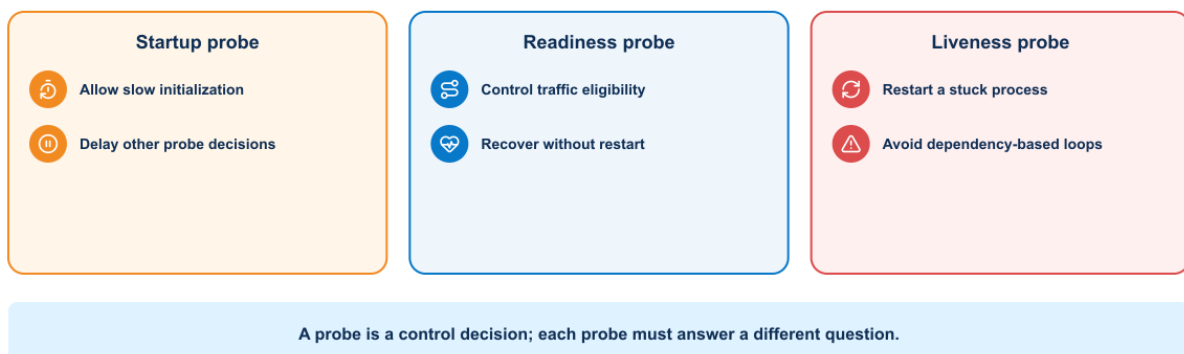


Figure 3-21. Distinct startup, readiness, and liveness probe decisions

Kubernetes supports:

- Startup probes for slow initialization
- Readiness probes for traffic eligibility
- Liveness probes for process recovery

Probe timing and thresholds should reflect application behavior. A liveness probe that fires during normal startup can create a restart loop. Readiness should change when the instance cannot serve requests but might recover without restart.

Compact deployment example

This fragment shows the controls that reviewers should look for rather than a complete production manifest:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: automation-api
```

```
spec:
  replicas: 2
  selector:
    matchLabels: {app: automation-api}
  template:
    metadata:
      labels: {app: automation-api}
    spec:
      serviceAccountName: automation-api
      containers:
        - name: api
          image: registry.example/automation@sha256:APPROVED_DIGEST
          ports: [{name: http, containerPort: 8080}]
          readinessProbe:
            httpGet: {path: /ready, port: http}
          livenessProbe:
            httpGet: {path: /live, port: http}
          resources:
            requests: {cpu: 100m, memory: 128Mi}
            limits: {cpu: 500m, memory: 512Mi}
          securityContext:
            runAsNonRoot: true
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: [ALL]}
```

The immutable digest preserves artifact identity. Separate readiness and liveness paths avoid restarting a recoverable instance merely because a dependency is temporarily unavailable. Resource values are hypotheses that must be adjusted from observed demand, and Pod security controls do not replace RBAC or network policy.

3.4.11 Scaling and self-healing

A Deployment replaces failed Pods and maintains replica count. This repairs instance loss, not application defects or failed dependencies.

Horizontal scaling can respond to metrics when resource requests and scaling signals are meaningful. Scaling an application cannot repair a saturated database or external rate limit. The team should understand which tier limits capacity.

Scaling workers increases simultaneous device sessions and change concurrency. Queue depth alone is not a safe scaling signal. Add per-device locks, global protocol limits, controller rate limits, and a maximum network blast-radius policy.

Self-healing restarts a failed Pod. It cannot determine whether a timed-out NETCONF commit partially changed a device. The replacement worker must inspect job and device state before retrying.

3.4.12 Rolling updates

A rolling update gradually replaces the old ReplicaSet. Readiness controls whether new Pods receive traffic. Update parameters control temporary excess capacity and allowed unavailability.

The release must support version overlap. API contracts, sessions, and database schemas must remain compatible during the rollout.

Kubernetes can pause, resume, and undo Deployment revisions, but rollback only changes the workload template. It does not reverse external database migrations or infrastructure side effects.

3.4.13 Advanced deployment patterns

Deployment patterns control how a new version is introduced and how risk is distributed during the transition. The appropriate pattern depends on capacity, compatibility, observability, and the speed at which traffic can be redirected or a release reversed.

Blue-green

Two versions run separately, and a Service or routing layer changes the active destination after validation. This needs extra capacity and clear data compatibility.

Canary

A routing mechanism sends limited traffic to a new version. Metrics compare the canary with the stable version. Promotion criteria must account for traffic volume and statistical confidence.

GitOps

GitOps uses a repository as the reviewed desired state and a cluster-side reconciler to apply it. The pipeline updates a versioned environment definition rather than holding broad credentials and directly issuing every cluster change.

GitOps still requires repository security, reconciliation policy, secret handling, health assessment, and recovery design.

3.4.14 Kubernetes networking

Pods receive routable cluster addresses according to the cluster network implementation. Services provide stable virtual access. NetworkPolicy can restrict permitted connections when the cluster network plugin enforces it.

The automation platform should allow only the service paths its architecture requires. Other paths should be denied where the cluster network implementation supports policy enforcement.

For the network automation platform, allow:

- Authorized GitLab or engineer clients to the automation API
- API to queue and job database as required
- Worker to queue, secret service, evidence service, and approved device endpoints
- Telemetry collector to defined receivers and storage
- Dashboard to its data sources

Deny API and dashboard Pods from direct device-management access. Kubernetes NetworkPolicy applies only when the cluster network implementation enforces it and does not replace external firewalls.

3.4.15 RBAC and service accounts

Use separate service accounts for API, worker, validation, and telemetry components. The worker may need permission to read a narrow secret reference or create a job artifact. It should not list every Secret in the namespace or modify cluster-wide resources.

GitLab's deployment identity should update only the course namespace and approved object kinds. Human administrators retain a separate break-glass path with strong audit.

3.4.16 Configuration and secrets

Configuration changes can update mounted files or environment inputs differently. Applications may need restart or dynamic reload. Record which behavior the application supports.

Secrets should not appear in manifests committed to the repository. Options include an external secret operator, encrypted secret workflow, CSI integration, or pipeline-controlled injection. Each option has its own trust boundary.

3.4.17 Monitoring and logging

Kubernetes operational visibility includes:

- Application metrics and logs
- Pod restart and readiness state
- Deployment availability
- Resource requests, use, throttling, and termination
- Kubernetes events
- Node and control-plane health
- Network and storage behavior
- Deployment annotations and image identity

Container logs need collection before Pods disappear. Dashboards should connect application symptoms to workload version, namespace, node, and recent rollout.

An alert should focus on sustained impact or loss of safety margin. A single Pod restart may be normal, while repeated restart loops or insufficient ready replicas require attention.

Network automation dashboards should also show queue delay, worker concurrency, per-device lock contention, API and NETCONF latency, job failure category, rollback state, and the network signals developed in Module 5. Correlate Pod rollout events with changes in automation job behavior.

3.4.18 Troubleshooting workflow

Use a consistent sequence:

1. Confirm context, namespace, and intended object name.
2. Inspect Deployment, ReplicaSet, Pod, and Service status.
3. Review conditions and recent events.
4. Inspect current and previous container logs.
5. Check image identity, configuration, probes, and resource settings.
6. Test service discovery and network reachability from an appropriate Pod.
7. Compare behavior with the pipeline and deployment event.

Useful commands include `kubectl get`, `kubectl describe`, `kubectl logs`, `kubectl events`, `kubectl rollout status`, `kubectl rollout history`, and `kubectl exec` when policy allows it.

3.4.19 Multiple data-center deployments

Kubernetes clusters normally form separate failure and administration domains. A multicluster design must address traffic steering, identity, policy consistency, data replication, configuration promotion, observability, and recovery.

Stretching one control plane across distant failure domains can create latency and quorum risks. Many designs use independent clusters and coordinate application delivery above them.

Networking and policy platforms may connect data-center, cloud, and Kubernetes environments. The design must define which controller owns each policy and how operational evidence crosses domains.

Avoid allowing independent clusters to change the same device concurrently. Assign device or site ownership, use a shared coordination service, or route all change jobs through one control boundary. Disaster recovery should preserve job state and ensure that an uncertain in-flight change is inspected before another region retries it.

3.5 Knowledge check

3.5.1 Container runtime fundamentals

Use these questions to verify that you can distinguish image, container, host, and runtime responsibilities.

1. Which dependencies does a container image control, and which remain part of the environment?
2. Why is an image digest stronger evidence than the `latest` tag?
3. Why should persistent application data not remain in the container writable layer?
4. When should a service port remain internal to a Docker network?
5. Why can access to the Docker socket create a serious security risk?

3.5.2 Secure image packaging

Use these questions to evaluate whether an image build is reproducible, traceable, and appropriately hardened.

1. What risks arise from an overly broad Docker build context?
2. Why does deleting a copied secret in a later layer fail to protect it?
3. How does a multistage build reduce the final runtime boundary?
4. Why should the pipeline promote a digest instead of rebuilding for each environment?
5. What information should accompany an accepted scanner exception?

3.5.3 Multitier deployment

Use these questions to confirm that you can reason about service discovery, state, health, and failure isolation.

1. Why should the application connect to a service name rather than a container IP address?
2. What is the difference between starting a database process and proving database readiness?
3. Why should a database volume not be described as a backup?
4. Which services need published ports in a three-tier stack?
5. Why can a dependency-based liveness check cause a restart storm?

3.5.4 Kubernetes orchestration

Use these questions to assess whether you can relate Kubernetes reconciliation, workload identity, rollout behavior, and multidata-center design to DevOps delivery controls.

1. What does reconciliation mean in Kubernetes?
2. Why should clients use a Service rather than a Pod IP?
3. How do requests and limits affect workload operation?
4. Why can a successful Deployment rollback fail to restore application behavior?
5. Which evidence should the pipeline collect after a failed rollout?

3.6 Summary

A container establishes a repeatable process and dependency boundary. A controlled Dockerfile turns that boundary into an identifiable, testable, and defensible image. Compose demonstrates how the same image participates in a service with APIs, workers, queues, databases, networks, health contracts, and persistent state. Kubernetes extends the operating model with scheduling, reconciliation, scaling, policy, and clustered availability when those capabilities justify the additional platform responsibility.

Across every platform, the engineering invariants remain the same: build once, promote by digest, keep configuration and secrets outside the image, separate application traffic from privileged target access, verify readiness at the service boundary, constrain concurrency and blast radius, correlate execution with operational evidence, and recover from observed state rather than assuming a retry is

safe. Module 1 supplies the delivery principles; Module 2 supplies the environment; Module 3 now supplies the deployable artifact and runtime contracts.

Two workflows must remain separate as the course continues. The **platform pipeline** will build, test, and deploy the automation software. The **network-change workflow** will use an approved platform version to validate intent, obtain approval, execute a bounded network operation, and retain evidence. Deploying a new application version must not automatically authorize a network change.

Executing all of these controls manually creates a new problem. Engineers must remember the correct build inputs, run tests in the correct order, preserve the digest, apply environment policy, wait for readiness, collect evidence, and make the same promotion decision every time. As the number of services and environments grows, a written checklist cannot provide consistent execution or rapid feedback. That unresolved delivery problem establishes the requirement for CI/CD.

What the learner now has: an identified application artifact and a deployable runtime platform.

What is still missing: a repeatable delivery process that coordinates qualification, promotion, deployment, validation, and recovery.

What the next module adds: Module 4 turns those manual steps into executable delivery policy while keeping software deployment separate from authorization of a network change. Continue to Continuous Integration, Delivery, and Deployment Validation.

MODULE 4

4 Continuous Integration Delivery and Deployment Validation

Central engineering question

Can we automate the delivery process safely?

SOURCE → TEST → ARTIFACT → PROMOTE → VERIFY

Learning objectives

- Design purpose-specific CI/CD pipelines with defensible gates.
- Build once, identify artifacts, and promote without rebuilding.
- Validate deployment outcomes and choose an appropriate recovery response.

Estimated theory time	4.5 hours
Lab connection	Apply the module controls to the cumulative application and delivery system
Core classroom topics	Concepts, decisions, workflows, and required controls
Advanced reference	Detailed implementation and platform considerations

4.1 CI/CD delivery flow

4.1.1 Software delivery pipeline

A pipeline does not treat a successful command or API response as final proof. Build completion proves that an artifact was created; deployment completion proves that a platform accepted a request. Health and acceptance checks must still prove that users or downstream systems receive the intended outcome. For a network automation application, that may include an existing read-only network test.

The conceptual flow is source → review → validate → test → build → scan → publish → deploy → validate → observe. GitLab implements that flow with merge requests, pipelines, jobs, runners, artifacts, rules, dependencies, environments, and approvals; those product objects are introduced only after the delivery responsibilities are clear.

4.1.2 Pipeline objectives

A useful pipeline should answer these questions:

- Does the change meet formatting and static-analysis rules?
- Does expected automation and network behavior still pass?
- Can the repository build the required immutable artifact?
- Does the artifact start and behave correctly?
- Does the change satisfy security policy?
- Which environment received the release?
- Can the team trace the deployment back to source and evidence?

The pipeline should fail early on inexpensive, high-value checks. Slow environment provisioning should not run when syntax or unit tests already fail.

4.1.3 GitLab pipeline model

GitLab reads `.gitlab-ci.yml` from the repository. A pipeline contains jobs organized into stages or a dependency graph. Runners execute jobs.

Important concepts include:

- Stage: broad ordering category
- Job: commands and execution settings for one responsibility
- Runner: agent that performs the job
- Artifact: output retained or passed to another job
- Cache: reusable data that improves speed but is not authoritative output
- Variable: configuration supplied to a job
- Environment: named deployment target with history
- Rule: condition controlling whether a job appears or runs

GitLab concepts in network automation

GitLab combines source control, review, pipeline execution, and evidence management. The table maps its principal delivery concepts to the responsibilities they can support in an automation system.

Table 4-1. GitLab Components and Network Automation Responsibilities

Concept	Network automation use
Pipeline	Complete evaluation of one commit and target context
Stage	Broad control point such as validate, render, precheck, deploy, or postcheck

Concept	Network automation use
Job	One bounded responsibility that produces a clear status and evidence
Runner	Execution agent placed according to trust and management reachability
Artifact	Rendered configuration, test report, plan, backup, diff, or sanitized evidence
Cache	Reusable package download that may disappear without affecting correctness
Variable	Environment name, image tag, timeout, or protected credential reference
Environment	Named test, staging, or production network context with deployment history
Rule	Condition that prevents change jobs on untrusted branches
Manual approval	Deliberate promotion decision after diff and evidence review

From issue to verified merge

A pipeline begins before `.gitlab-ci.yml` is evaluated. A work item or change record states the problem, scope, owner, risk, and acceptance criteria. A feature branch isolates the proposed source change. The merge request then becomes the review boundary that joins discussion, source difference, pipeline results, policy, and approval.

For the reference scenario, the merge request should let a reviewer answer five questions without reconstructing the author's workstation: What network outcome is requested? Which intent and application files changed? Which target and maximum scope are authorized? What candidate configuration and test evidence were produced? What conditions would stop or reverse deployment?

Approval applies to the reviewed commit and evidence. A new commit invalidates conclusions tied to the previous revision and should rerun required checks. Merging records integration into the protected branch; it does not itself authorize a live network change unless the environment policy explicitly defines that behavior.

Source-of-truth-triggered delivery

Repository activity is not the only meaningful pipeline trigger. When a source-of-truth object reaches a complete and valid state, its event can initiate a workflow whose purpose is to reconcile that intent. This is appropriate when the authoritative business event is an inventory or service-model change rather than a new application commit.

The webhook should carry a small reference such as an object identifier, not a password, full configuration, or trusted deployment payload. The first job retrieves the current object from the source-of-truth API and validates its type, relationships, scope, status, freshness, and automation eligibility. The result is serialized as a frozen intent artifact with a fingerprint. Test and production consume that same artifact; re-reading mutable intent immediately before production could deploy a different change from the one that passed testing.

4.1.4 Pipeline design

The complete pipeline can be read as two trust zones. General jobs interpret repository content and create evidence without management access. Protected jobs receive the approved digest, target, diff, and credential only after the gate. The handoff consists of immutable artifacts and approval context, not an instruction to rebuild the application on the protected runner. The compact reference architecture in Module 1 shows this separation.

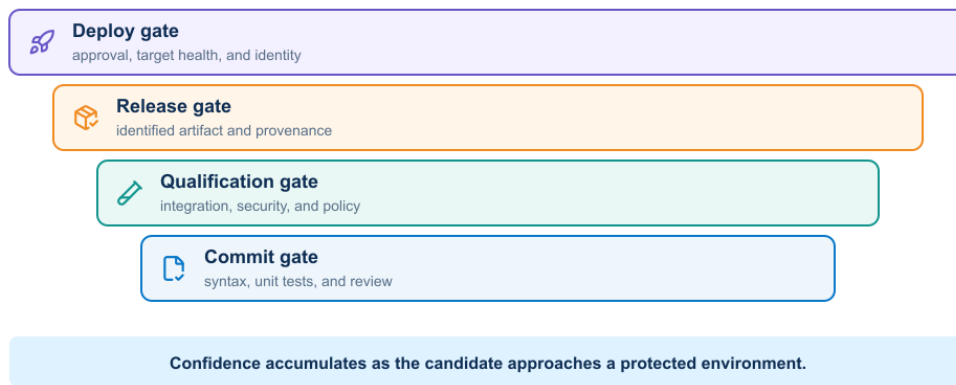


Figure 4-1. Progressive commit, qualification, release, and deployment gates

A typical stage sequence is validation, testing, build, inspection, integration, deployment, and verification.

Each job should have one clear outcome. A large script that builds, deploys, and tests without preserving intermediate evidence is difficult to diagnose and reuse.

Dependencies can express which artifacts a job actually needs. Independent jobs can run concurrently, reducing feedback time.

4.1.5 Validation and testing layers

No single test can establish that a change is safe. Effective pipelines build confidence progressively, starting with inexpensive source checks and advancing toward tests that require integrated services or representative environments.

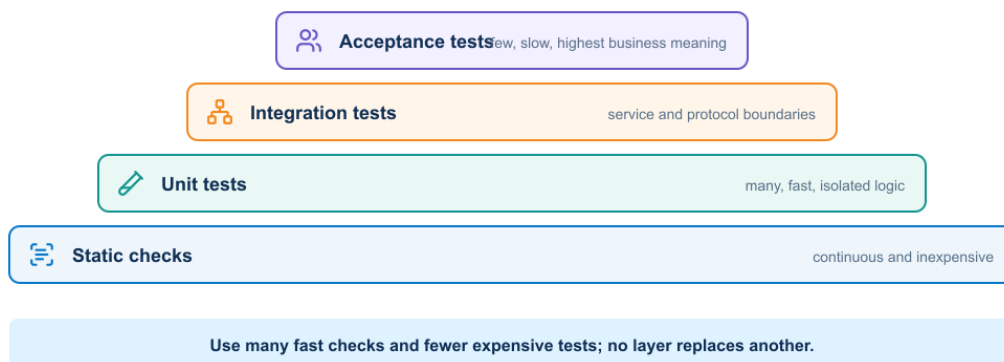


Figure 4-2. Test-confidence model from static checks through unit, integration, and acceptance tests

Static validation

Static checks inspect files without running the complete application. Examples include YAML parsing, Dockerfile linting, formatting checks, type checks, Terraform validation, and Kubernetes schema validation.

Unit testing

Unit tests isolate application behavior and should provide fast feedback. They should not require a live network, shared database, or production account.

Integration testing

Integration tests examine boundaries such as application-to-database interaction or an HTTP request through the proxy. They need controlled dependencies and reliable cleanup.

System and acceptance testing

These tests evaluate the assembled system and user-visible outcomes. They cost more to run, so teams select a focused set for each change and reserve broader suites for scheduled or release pipelines.

Network policy validation

Policy checks examine intent and derived configuration before device access. Depending on the scenario, they can enforce:

- VLAN range and reserved-ID rules
- Site-specific address allocation
- Prefix-overlap prevention
- Required descriptions and change identifiers
- Approved routing protocols, processes, peers, or areas
- Passive-interface requirements where appropriate
- Prohibition of broad redistribution or default-route changes
- Maximum number of devices and configuration lines changed

Policy should report the exact object and rule. A vague `compliance failed` result wastes review time.

Test environment and limitation matrix

Each validation layer answers a different question and depends on a different level of environmental realism. This matrix helps teams select the least expensive test that can provide meaningful evidence for a given risk.

Table 4-2. Test Environments Capabilities and Limitations

Test layer	Typical tools	Environment required	Detects	Important limitation
Static and schema	yamllint, JSON Schema, Pydantic, linters	No network	Syntax, type, required-field, and basic policy defects	Cannot prove renderer or device behavior
Unit and template	pytest, Jinja2 golden files	No network	Normalization, policy logic, rendered output, exception paths	Expected files can encode the same mistaken assumption as the code
Parser fixtures	pytest, Genie/TextFSM with sanitized output	No network	Parser regressions and missing-field handling	Recorded output does not reproduce timing or platform side effects
Mock API	pytest, HTTP/NETCONF mock	Mock network/service	Authentication branches, status codes, RPC errors, pagination, retry logic	A mock proves client behavior, not product semantics
Service integration	Compose, API/queue/database tests	Isolated local services	Contracts, persistence, worker state transitions, timeouts	Does not prove NOS configuration or protocol convergence
Virtual device	Virtual network appliance or simulator, pyATS	Disposable topology	Capabilities, command/model semantics, transactions, routing behavior	Images, scale, hardware, and timing can differ from production
Real lab or sandbox	pyATS, Ansible check/diff, probes	Dedicated equipment or authorized sandbox	Platform-specific behavior and end-to-end workflow	Availability, topology, and reset behavior constrain coverage

Test layer	Typical tools	Environment required	Detects	Important limitation
Production pre/post-check	Read-only APIs, pyATS, telemetry	Approved production read access	Actual baseline, scope, convergence, forwarding, and collateral effects	Observation must be bounded, safe, and cannot eliminate change risk

A strong pipeline uses many cheap offline tests and a smaller number of increasingly realistic tests. Production telemetry closes the feedback loop; it must not become the first place a predictable defect is tested.

4.1.6 Build once and promote

The build job should produce a versioned, immutable artifact. Later jobs deploy the same artifact. Rebuilding separately for staging and production allows dependencies or build conditions to change and invalidates earlier evidence.

For a container release, record the source commit, human-readable tag, and image digest. Deploy by digest for strong identity.

4.1.7 Artifacts and caches

The diagram distinguishes data required for correctness from data used only to improve speed.



Figure 4-3. Authoritative pipeline artifact compared with a disposable cache

A downstream job consumes an artifact deliberately. A cache may be missing or stale, so every job must remain correct without it.

Artifacts are outputs that the pipeline needs to retain, such as test reports, coverage results, deployment plans, an SBOM, or packaged configuration. They should have an appropriate expiration and access policy.

A cache accelerates work by reusing dependency downloads or build intermediates. Jobs must remain correct when the cache is empty. Never use a cache as the only copy of a release artifact or security report.

4.1.8 Runner design

GitLab coordinates pipelines, but it does not execute the shell commands in a job. That work occurs on runner infrastructure. Runner design therefore determines the operating system, tools, network paths, credentials, isolation, capacity, and residual state available to repository-controlled code.

This distinction matters in network automation. A validation job may need source code and a Python environment, while a deployment job may need a route to a management network and authority to request a short-lived device credential. Assigning both jobs to the same broadly trusted runner collapses two different security boundaries.

Runner terminology and job flow

Several related components are often called a **runner**, but they have different responsibilities:

- **GitLab instance:** creates the pipeline, schedules jobs, issues job-scoped credentials, receives logs, and records status and artifacts.
- **Runner configuration:** the logical GitLab resource that defines scope, tags, protection, and other scheduling attributes.
- **Runner manager:** the installed GitLab Runner process that reads `config.toml`, requests work, prepares execution, and reports results.
- **Executor:** the mechanism used to run a job, such as Shell, Docker, Kubernetes, or a supported autoscaling executor.
- **Job environment:** the actual shell, container, Pod, or machine in which the job commands execute.

The runner manager authenticates to GitLab with a runner authentication token. After GitLab assigns a job, the runner uses job-scoped information to fetch the required source and artifacts, invokes the executor, streams output, and returns status and artifacts. The runner authentication token identifies the runner configuration; it is not a device credential and should never be passed into the automation application as one.

Current GitLab releases use the runner creation workflow and runner authentication tokens. Legacy registration tokens are deprecated and may be disabled. Installation procedures must therefore be checked against the deployed GitLab and GitLab Runner versions rather than copied from an older example.

Scope, tags, and job selection

A project runner is available to assigned projects, a group runner to projects within its group hierarchy, and an instance runner more broadly across the GitLab instance. Wider scope improves reuse and capacity sharing but increases the number of repositories whose jobs may reach the runner. A privileged or management-connected runner should consequently have the narrowest practical scope.

Tags match jobs to runners with required capabilities. For example, `linux-container` may select an ordinary validation environment and `protected-network-runner` may select a restricted deployment environment. Tags describe scheduling capability; they are not an authorization control by themselves. Protection settings, branch and environment rules, repository permissions, runner scope, identity policy, and network enforcement must agree.

Executor tradeoffs

Executor choice changes the isolation and reproducibility of the job environment. It does not change the trust of the pipeline source or automatically restrict network reachability.

Table 4-3. GitLab Runner Executor Tradeoffs

Executor	Useful characteristic	Important limitation	Appropriate course role
Shell	Direct access to host tools and network	Jobs share the host and can leave files or alter installed state	Small, tightly controlled deployment runner where host ownership is explicit
Docker	Reproducible job image and disposable container filesystem	Host kernel is shared; Docker socket or privileged mode can expose host control	General lint, unit-test, render, and packaging jobs
Kubernetes	Ephemeral Pods, scheduling, quotas, and scalable capacity	Adds cluster RBAC, admission, CNI, image, namespace, and node trust	Shared validation capacity when an established cluster platform exists

Executor	Useful characteristic	Important limitation	Appropriate course role
Autoscaling machine or instance	Clean machine boundary and elastic capacity	Startup time, image maintenance, cloud identity, and cost require control	Bursty builds or stronger per-job isolation

The Shell executor is not inherently wrong, and the Docker executor is not inherently safe. A dedicated, patched Shell runner with a narrow network path can be more defensible for a protected deployment than a container runner that mounts the Docker socket and accepts untrusted projects.

General and protected runners

The runner model identifies where repository-controlled commands execute and why general validation and management-plane deployment require different trust levels.

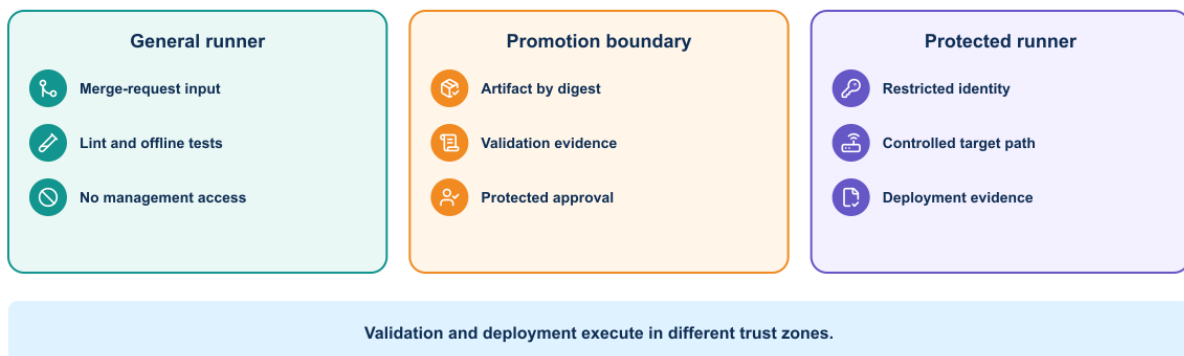


Figure 4-4. Separation of the general validation runner from the protected network runner and management zone

As shown in Module 1, the protected runner is the first pipeline execution component with management-plane reachability. Earlier jobs pass an approved artifact and evidence across that boundary; they do not inherit the same access.

- A general validation runner has Internet or registry access but no production device route or deployment secret.
- A protected network runner reaches the management network and runs only protected-branch or approved-environment jobs.

The protected runner should accept only explicitly tagged jobs from approved refs or environments, verify the artifact digest and target context, request a scoped identity just in time, and write evidence to an independently protected destination. Its firewall path should reach only required management endpoints. Runner concurrency must also respect device and routing-domain locks; compute capacity is not permission to increase network blast radius.

If Docker builds require a privileged mechanism, isolate that builder from the network runner. Mounting `/var/run/docker.sock` gives a job powerful control over the Docker host. A job that can control the host can inspect other containers, mounts, credentials, and network paths, so container boundaries no longer provide meaningful protection.

Verification

Submit an untrusted-branch job with the protected runner's tag and confirm that it remains unscheduled. Then verify from the runner host and job environment that the general runner has no management route and that the protected runner can reach only the approved endpoints. A green pipeline is not evidence that runner isolation works; the denied paths must also be tested.

4.1.9 Variables and secrets

Non-sensitive configuration may live in the repository. Secrets should use protected and masked variables or an external secret manager. Masking only reduces accidental log display; it does not prevent a malicious job from transmitting an available secret.

Control secret exposure through job rules, protected branches, protected environments, short-lived credentials, narrow permissions, and runner isolation.

4.1.10 Pipeline rules

Different events need different work:

- Merge-request pipelines validate proposed changes.
- Default-branch pipelines create or update approved artifacts.
- Tag pipelines can create releases.
- Scheduled pipelines can perform broader scans or maintenance.
- Manual jobs can provide controlled promotion or cleanup.

Rules should be understandable and tested. A critical security job that silently disappears due to a complex rule creates a dangerous gap.

One pipeline purpose per execution path

A practical repository often contains more than one workflow. Treat each as a product interface with its own trigger, input contract, privileges, target, acceptance criteria, and completion condition.

Table 4-4. Pipeline Purposes Triggers and Protected Resources

Pipeline purpose	Typical trigger	Privileged outcome	Required evidence
Application delivery	Merge request, protected branch, or release tag	Deploy an identified application image	Tests, digest, rollout and service health
Infrastructure lifecycle	Reviewed definition or downstream request	Create, update, or destroy resources	Plan, state identity, outputs, drift and cleanup
Network intent delivery	Source-of-truth event or approved request	Change authorized network state	Frozen intent, test, approval and production verification
Assurance and drift	Schedule or operational event	Usually read-only; may open remediation	Observed state, difference, severity and owner
Security maintenance	Dependency or policy event	Rebuild, rotate, quarantine, or revoke	Finding, decision, replacement and closure

A trigger variable can select an explicitly named workflow, but the pipeline must validate both event source and input. Unrelated jobs should be absent rather than receive unnecessary secrets and then harmlessly skip their commands.

4.1.11 Environments and promotion

GitLab environments record deployments to targets such as review, test, staging, and production. A review environment can give each merge request an isolated endpoint. A stop job removes it when no longer needed.

Promotion policy may require successful checks, peer approval, change-window conditions, or an environment owner. The approval should act on the tested artifact identity.

4.1.12 Pipeline efficiency

Pipeline optimization should shorten the time to trustworthy feedback, not merely reduce elapsed minutes. A fast pipeline that omits a required control or tests a different artifact is inefficient because it creates rework and operational risk.

Dependency graphs and safe concurrency

Stages provide an understandable broad order, while `needs` expresses the jobs and artifacts required by a particular consumer. This forms a directed acyclic graph: a job can start when its declared predecessors complete instead of waiting for unrelated work in an earlier stage.

Linting, schema validation, secret detection, and independent unit-test groups can often run concurrently. Rendering must wait for validated intent; deployment must wait for the approved artifact and pre-check evidence. Network jobs that affect the same device or routing domain remain serialized even when their software tests can run in parallel.

Reuse and maintainability

Repeated YAML eventually diverges. Small local patterns can use hidden jobs and `extends`; shared organizational policy can use controlled `include` files or CI/CD components where supported. YAML anchors operate within YAML processing and are useful for limited repetition, but excessive indirection makes a pipeline harder to review.

Reusable definitions must be versioned. Referencing a mutable shared template allows pipeline behavior to change without a commit in the consuming repository. Pin an approved version or commit, define ownership, test compatibility, and provide a deliberate update path. Parent-child pipelines can separate components in a large repository, but they also require clear artifact, variable, status, and cancellation behavior across the boundary.

Purpose-built job images

A job image should contain the toolchain required for one related class of work, with versions controlled and the image identified by digest. Installing Python, Ansible collections, scanners, and operating-system packages at the start of every job increases time and allows dependency resolution to change between runs.

The validation image should include only offline validation tools and trusted certificate material. It should not contain environment-specific inventory or credentials. A separate deployment image can include approved network clients while remaining unavailable to untrusted runners. Both images require patching, scanning, provenance, and an owner; a purpose-built image becomes technical debt if nobody maintains it.

Practical improvements include:

- Running independent jobs concurrently
- Reusing safe dependency caches
- Selecting tests based on affected components when confidence remains adequate
- Building a shared artifact once
- Avoiding unnecessary service startup
- Cancelling superseded pipelines

Efficiency must not remove evidence needed for safety.

4.1.13 Pipeline failure categories

A useful failure message separates:

- Source or test defect
- Dependency resolution failure
- Build failure
- Runner or platform problem
- Missing or unauthorized secret
- Environment provisioning failure
- Deployment failure
- Health or acceptance failure

Jobs should return a nonzero status on failure and preserve relevant evidence. Scripts that continue after a failed command can create false success.

4.1.14 Illustrative `.gitlab-ci.yml`

The pipeline below demonstrates how the validation layers can be ordered and how artifacts can pass evidence between jobs. It is deliberately illustrative: runners, credentials, approval rules, and deployment commands must be adapted to the target environment.

```
stages: [validate, render, test, build, precheck, deploy, postcheck]

default:
  image: registry.example/network-devops/validator@sha256:VALIDATOR_DIGEST
  before_script:
    - python --version
    - test -n "$CI_COMMIT_SHA"

variables:
  INTENT_FILE: examples/service-intent.yml
  TARGET_LIMIT: distribution-01
  TARGET_PLATFORM: lab-nos
  EVIDENCE_DIR: evidence-output

validate_intent:
  stage: validate
  script:
    - python -m automation.python.validate_intent "$INTENT_FILE"
    - yamllint "$INTENT_FILE" inventory/lab.yml
  artifacts:
    when: always
    paths: [evidence-output/intent-validation.json]

render_network_config:
  stage: render
  needs: [validate_intent]
  script:
    - python -m automation.python.render --intent "$INTENT_FILE" --platform
"$TARGET_PLATFORM"
  artifacts:
    paths: [evidence-output/rendered/]

offline_tests:
  stage: test
  needs: [render_network_config]
  script:
    - pytest --junitxml=evidence-output/junit.xml
    - ansible-playbook --syntax-check automation/ansible/playbooks/deploy.yml
  artifacts:
    when: always
    reports:
      junit: evidence-output/junit.xml

build_automation_image:
  stage: build
  needs: [offline_tests]
  script:
    - ./ci/build-image --commit "$CI_COMMIT_SHA" --metadata evidence-output/image.json
    - ./ci/create-sbom --metadata evidence-output/image.json --output evidence-
output/sbom.cdx.json
    - ./ci/scan-image --metadata evidence-output/image.json --report evidence-
```

```

output/image-scan.json
  - ./ci/export-digest --metadata evidence-output/image.json --dotenv evidence-
output/image.env
  artifacts:
    paths:
      - evidence-output/image.json
      - evidence-output/sbom.cdx.json
      - evidence-output/image-scan.json
    reports:
      dotenv: evidence-output/image.env

network_precheck:
  stage: precheck
  needs:
    - job: render_network_config
      artifacts: true
    - job: build_automation_image
      artifacts: true
  tags: [protected-network-runner]
  resource_group: distribution-01
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
  script:
    - python -m automation.python.precheck --inventory lab --limit "$TARGET_LIMIT"
  artifacts:
    when: always
    paths: [evidence-output/precheck/]

deploy_lab_target:
  stage: deploy
  tags: [protected-network-runner]
  resource_group: distribution-01
  environment:
    name: lab/distribution-01
  when: manual
  allow_failure: false
  needs:
    - job: network_precheck
      artifacts: true
    - job: build_automation_image
      artifacts: true
  script:
    - test -n "$AUTOMATION_IMAGE_DIGEST"
    - python -m automation.python.verify_target --inventory lab --limit "$TARGET_LIMIT"
    - python -m automation.python.deploy --image-digest "$AUTOMATION_IMAGE_DIGEST" --
approved-diff evidence-output/precheck/diff.json

network_postcheck:
  stage: postcheck
  tags: [protected-network-runner]
  resource_group: distribution-01
  needs: [deploy_lab_target]
  script:
    - pyats run job validation/pyats/service_validation_job.py --testbed-file
inventory/lab.yml
  artifacts:
    when: always
    paths: [evidence-output/postcheck/, archive/]

```

This is a teaching example. Commands under `./ci/` are repository-owned wrappers, not GitLab or industry-standard commands; each must fail nonzero on policy violation and produce the named evidence. `export-digest` writes `AUTOMATION_IMAGE_DIGEST` to the dotenv report after the image has been pushed and its registry digest resolved. GitLab syntax and feature availability depend on the deployed GitLab version and tier. Validate the pipeline against the target instance.

4.1.15 Explanation of the pipeline blocks

Each block in the example has a distinct control purpose. Together they make the delivery path visible, preserve evidence, and prevent a later stage from running when an earlier assumption has failed.

- `stages` makes the network controls visible in order.
- `default.image` pins the validation runtime instead of using a mutable tag.
- `INTENT_FILE` selects one reviewed source document.
- `validate_intent` checks structure before rendering or network access.
- `artifacts.when: always` preserves failure evidence as well as successful output.
- `needs` allows a job to start only after its required evidence exists.
- `offline_tests` exercises code, fixtures, and playbook syntax without device credentials.
- `build_automation_image` creates the image once, resolves its registry digest, and retains the SBOM and scan reports with the build metadata.
- The dotenv report passes the resolved digest as data; deployment rejects an empty digest and does not resolve a mutable tag again.
- `tags` routes sensitive jobs to the protected network runner.
- `resource_group` prevents concurrent changes to the same target scope.
- `rules` excludes feature branches from live-device work.
- `when: manual` creates an approval point after the pre-check and diff.
- `verify_target` rechecks identity immediately before change.
- `pyats run job` validates structured operational outcomes after configuration.

Real pipelines should pass the exact image digest and evidence artifacts between jobs, verify that the approved diff belongs to the same commit and target, and define cleanup or recovery behavior.

Practical review: passing jobs, wrong artifact

A staging test may pass while production receives a rebuilt image carrying the same tag. The dashboard looks green, yet the production bytes were never tested. To detect this class of error, the build job records the image digest; scan, integration, approval, and deployment jobs consume that digest as an artifact; and the deployment record reports the same value. If any stage resolves a mutable tag again, the evidence chain is broken and promotion should stop.

4.1.16 Why each gate exists

A gate is useful only when its failure has a defined meaning and response. The table relates each gate to the risk it controls and the action engineers should take when that control rejects a change.

Table 4-5. Pipeline Gates and Their Evidence

Gate	Why it exists	Example failure	Required response
Intent parsing	Reject unreadable source before interpreting it	Duplicate YAML key changes a value silently	Fail and correct source
Schema validation	Enforce the contract shape and types	VLAN ID is a string or validation criteria are missing	Fail before normalization
Normalization	Create one canonical representation	Host address is mistaken for the intended prefix	Fail with field-level detail

Gate	Why it exists	Example failure	Required response
Policy validation	Enforce organizational constraints	Prefix overlaps an allocation or redistribution is prohibited	Block or obtain reviewed exception
Configuration rendering	Produce deterministic platform output	Undefined variable omits an interface control	Fail and preserve rendered artifact
Linting	Detect structural and unsafe implementation defects	Invalid playbook, Python error, or ambiguous YAML	Fail before network access
Unit and parser tests	Exercise logic and structured-output handling	Genie output structure changes after dependency update	Correct code, dependency, or fixture
Image validation	Bind execution to a known runtime	Critical dependency vulnerability or root runtime user	Block publication/promotion
Network pre-check	Prove correct target and healthy baseline	Serial number mismatch or required peer already down	Stop; do not attribute baseline failure to the change
Configuration diff	Reveal exact proposed effect	Template removes an unrelated routing statement	Reject and correct intent/renderer
Blast-radius evaluation	Limit shared-infrastructure exposure	Inventory selector expands from one device to 80	Stop and require new review
Approval	Bind human decision to exact evidence	Commit, target, or diff changed after approval	Invalidate approval
Deployment	Apply only the approved transaction	Timeout leaves outcome uncertain	Stop writes and rediscover actual state
Protocol convergence	Wait for bounded control-plane stability	Neighbor remains in <code>EXSTART</code> or BGP prefix count drops	Stop rollout and diagnose
Post-check	Prove configuration and service outcome	Device accepts commands but route or path is absent	Roll back or remediate according to evidence
Telemetry observation	Detect delayed or collateral degradation	Packet loss rises after immediate checks pass	Halt promotion and invoke recovery policy

4.1.17 Kubernetes delivery integration

A Kubernetes delivery pipeline can:

1. Validate manifests and policy.
2. Build and scan the image once.
3. Record and sign the digest.
4. Deploy to a test namespace.
5. Wait for rollout readiness with a timeout.
6. Run functional tests.
7. Collect resource state, events, logs, and test reports.
8. Promote the same digest through a controlled change.
9. Observe release health.
10. Remove the temporary namespace.

Avoid mutable image tags and broad cluster-admin credentials. Give the deployment identity access only to the required namespace and resource types.

Verification

Confirm both platform state and service outcome. A successful Kubernetes rollout does not prove that a worker can process an approved job safely or that the resulting network state meets its acceptance criteria.

Platform pipeline and network change pipeline

The pipelines meet at a versioned automation platform but have different triggers and outcomes.

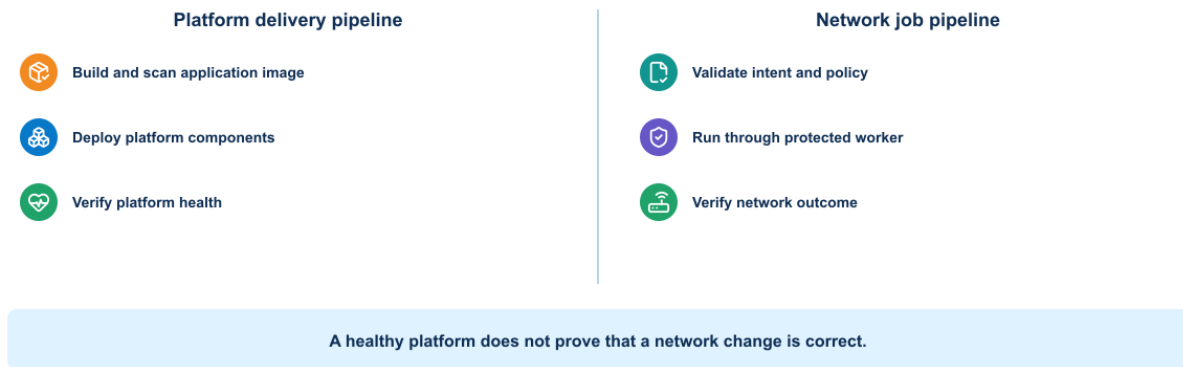


Figure 4-5. Separation between the platform delivery pipeline and network job pipeline

Deploying a new version of the automation application must **not** automatically authorize a network change.

Keep two concerns distinguishable:

- The platform pipeline tests and deploys the automation API, worker, validation service, and collectors.
- A network job pipeline submits reviewed change input to an approved platform version and evaluates network evidence.

The platform pipeline deploys software. The network-change workflow uses that software under a separate approval and evidence model.

Updating the worker image and changing network state in the same uncontrolled step makes troubleshooting difficult. Record both the platform image digest and the change-input commit in every job.

Test-before-production network promotion

A high-confidence network-change workflow promotes evidence as well as intent:

```
authoritative intent
  → policy and schema validation
  → on-demand representative environment
  → configuration deployment
  → independent operational verification
  → test-environment deletion
  → protected production approval
  → production deployment
  → independent production verification
```

A virtual C8000V in CML can provide realistic IOS XE configuration and protocol behavior, while it may not reproduce physical forwarding hardware, production scale, exact licenses, external dependencies, or timing. The validation report must state what the environment proves and what remains outside its scope.

Terraform owns the CML lab, virtual router, links, lifecycle, and deletion. Ansible owns configuration inside the reachable router. pyATS or another independent validator owns acceptance evidence. Keeping these responsibilities separate preserves useful ownership and failure boundaries.

The production gate requires the frozen intent fingerprint, source commit, deployment-image digest, test identity, test result, cleanup result, production identity, and approver to agree. A failed test blocks production. Failed cleanup also blocks production because it shows that the resource lifecycle is not under control.

4.2 Deployment validation and recovery

4.2.1 Three forms of state in an automation application

As introduced in Module 0, intended, configured, and operational state answer different questions. Here they become deployment-acceptance evidence: the pipeline determines when to collect them, how to evaluate them, and which result permits promotion or triggers recovery.

The loop below shows why stored configuration is an intermediate result. Operational observations must be compared with the original intent.

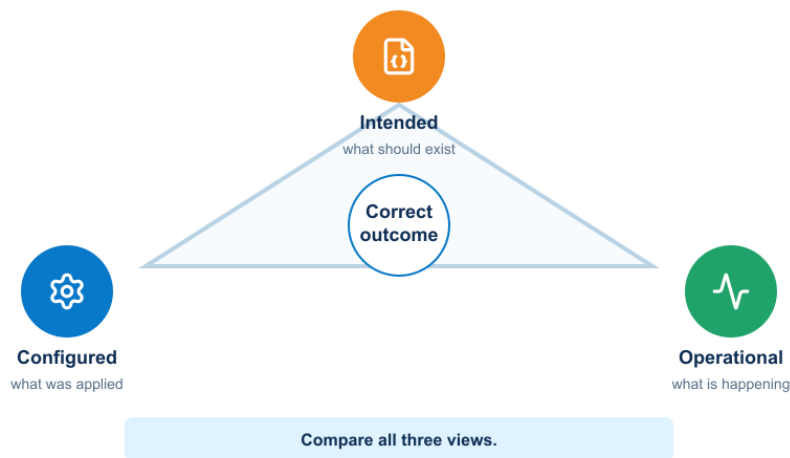


Figure 4-6. Relationship among intended, configured, and operational network state

Table 4-6. Intended Configured and Operational State Evidence

State	Meaning	Routing-service scenario
Intended state	Reviewed outcome the organization wants	Interface or service prefix, routing policy, and expected tests
Configuration state	Commands or modeled configuration stored by the device	Interface and routing configuration returned by the device
Operational state	Current protocol and forwarding behavior	Interface state, neighbor state, learned route, and reachability

The three states can disagree. A template may correctly represent intent while the device rejects part of it. The device may accept every command while the SVI stays down. The SVI may come up while the test peer never learns the route. A complete pipeline compares all three.

4.2.2 Network change state machine

Timeout and partial outcomes leave the normal promotion path rather than being treated as safe failures.



Figure 4-7. Network change states including unknown and recovery paths

An **UNKNOWN/PARTIAL** state is important. A timeout after sending configuration does not prove that nothing changed. The workflow must collect current state before retrying.

4.2.3 Evidence chain

Module 1 introduced evidence as a condition of promotion. Module 4 implements that principle by accumulating evidence as a release moves through the pipeline:

The evidence chain begins with source review and continues through static checks, unit tests, image inspection, integration tests, the infrastructure plan, pre-deployment checks, deployment, acceptance tests, and operational observation.

Each result should identify the source commit, artifact, pipeline, and target environment. This traceability supports approval, troubleshooting, and audit.

4.2.4 Build validation

Build validation confirms that the repository produces the expected artifact under controlled conditions. It may include:

- Dependency resolution and integrity
- Test execution and coverage
- Image creation
- Image metadata and runtime-user checks
- Vulnerability and secret scanning
- SBOM generation
- Artifact signature or provenance
- A minimal startup and health test

A successful build should not depend on files that exist only on a developer workstation.

4.2.5 Infrastructure validation

Infrastructure definitions need their own controls:

- Format and syntax checks
- Provider or module initialization
- Variable and schema validation
- Policy checks
- Plan generation
- Target and resource-count checks
- Cost or quota review when applicable
- Security review of network exposure and permissions

The plan is evidence, not approval by itself. Reviewers must understand the target and the meaning of the proposed actions.

4.2.6 Pre-deployment health checks

Before a release changes an environment, verify that the environment is safe to change. Useful checks include:

- Target identity and environment classification
- Current service health
- Available capacity
- Dependency reachability
- Credential validity without displaying the credential
- Required backups or snapshots
- Compatible database schema
- Absence of another conflicting deployment
- Availability of the previous known-good artifact

Deploying into an already degraded environment can make diagnosis and recovery harder.

Example pre-check set for a routing-service scenario

Pre-checks should establish whether the target is the expected system and whether its present condition permits the approved change. The following set illustrates checks that can stop deployment before any configuration is modified.

Table 4-7. Routing Service Predeployment Checks

Check	Reason	Blocking condition
DNS, route, and management port	Prove the runner can reach the management interface	Target unreachable or wrong path
Device identity and serial/hostname	Prevent change to the wrong system	Inventory and device identity differ
Authentication and authorization	Prove the service identity can perform the intended method	Login fails or privilege is insufficient
CPU and memory	Avoid adding change load to an unstable device	Threshold or trend violates policy
Interface and line-protocol state	Record baseline and detect unrelated failure	Required uplink or peer link is down
OSPF neighbor state	Confirm routing is healthy before change	Required neighbor is not FULL
Routing table and reachability	Establish baseline forwarding	Required baseline route or probe fails
Existing VLAN, SVI, and prefix	Detect collision or unmanaged prior state	Conflicting configuration exists
Running configuration backup	Support investigation and recovery	Backup cannot be collected or protected
Configuration lock/checkpoint capability	Select transaction and recovery approach	Required safety capability unavailable

Thresholds must be policy, not arbitrary constants hidden in code.

4.2.7 Configuration generation and diff

The pipeline loads the reviewed YAML intent, validates it, normalizes addresses, and renders platform-specific configuration. A Jinja2 fragment might be:

```

vlan {{ vlan.id }}
  name {{ vlan.name }}
!
interface {{ svi.name }}
  description {{ svi.description }}
  ip address {{ svi.ipv4_address | ipaddr('address') }} {{ svi.ipv4_address |
ipaddr('netmask') }}
  no shutdown
!
router ospf {{ routing.process_id }}
  passive-interface {{ svi.name }}
  network {{ routing.advertise_prefix | ipaddr('network') }} {{ routing.advertise_prefix |
ipaddr('hostmask') }} area {{ routing.area }}

```

Filters and exact syntax depend on the rendering environment and target network operating system. The pipeline tests rendered output with approved fixtures. It does not send a template containing undefined variables.

The proposed diff must identify additions, removals, replacements, and unexpected lines. Review should assess protocol effect, device count, configuration section, and recovery path rather than only line count.

4.2.8 Deployment interfaces and transactions

Deployment safety depends partly on the transaction semantics offered by the target interface. The following subsections compare common approaches and show why the same validation and recovery design cannot be assumed for every interface.

SSH CLI

CLI automation may enter configuration commands and collect output. It needs prompt handling, timeouts, error-pattern detection, and post-write verification. Command echo does not prove configuration acceptance.

NETCONF

NETCONF exchanges capabilities and structured RPCs. Where supported, candidate configuration and `validate`, confirmed commit, or rollback-on-error can improve transaction safety. Capabilities vary, so the client must inspect the server response rather than assume support.

RESTCONF

RESTCONF exposes YANG-modeled data through HTTP. The client must construct the correct resource path, content type, method, and payload for the device release. It should validate TLS, distinguish HTTP errors from YANG errors, and read state back after modification.

Ansible

Ansible can coordinate modules, templates, backups, and assertions across devices. Check and diff modes are useful only when the selected module and platform support them accurately. Review collection documentation and test behavior.

4.2.9 Scoped deployment controls

The change job should require an explicit environment, site, and device limit. It verifies inventory fingerprint, device identity, change ID, commit, approved diff hash, and automation image digest.

For multiple branches, use a canary and bounded batches. Stop when failure rate, protocol convergence, or telemetry crosses policy. Do not launch the maximum parallelism simply because the tool supports it.

Network blast-radius framework

Blast radius is multidimensional. Count devices, but also identify routing domains, redundancy pairs, controller scopes, tenants, sites, services, and management dependencies. Changing two route reflectors in the same cluster can be riskier than changing ten independent access switches.

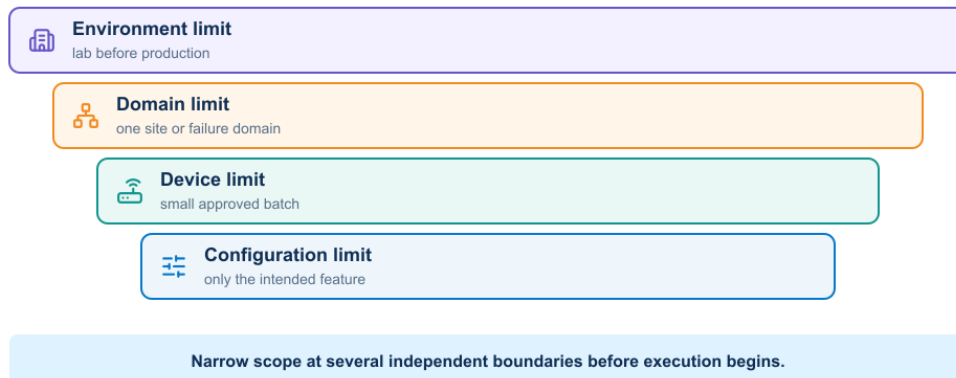


Figure 4-8. Nested configuration, device, domain, and environment blast-radius controls

- **Wrong-device prevention:** resolve a stable inventory identifier, connect through the expected management path, collect hostname/serial/platform, compare the inventory fingerprint, and display the final target set before approval.
- **Per-device locking:** prevent two jobs from writing to the same device. Treat an expired lock carefully because the earlier job may still be running.
- **Routing-domain locking:** serialize changes that touch the same adjacency, route-reflector cluster, redistribution boundary, or policy control point even when device names differ.
- **Bounded concurrency:** specify maximum parallel targets and maximum acceptable failures. A tool default is not a safety policy.
- **Rate limiting:** constrain API requests, login attempts, configuration operations, and controller jobs to avoid control-plane overload.
- **Maintenance windows:** enforce start and stop boundaries, required operators, and sufficient time for observation and recovery.
- **Credential scope:** bind identity to required methods, targets, configuration domains, and duration; read-only validation should not receive write privilege.
- **Out-of-band access:** verify recovery access before a change that can affect in-band management or routing reachability.

Stop conditions must be machine-readable where possible: identity mismatch, unhealthy baseline, unexpected diff, excessive target count, lost management access, convergence timeout, new critical logs, increased packet loss, or failure of an unaffected-service check.

4.2.10 Network post-checks

Post-checks should compare the new state with both intent and baseline:

- The requested interface or service object exists with the expected attributes.
- `Vlan120` has the correct description and address.
- Interface administrative and operational state match the lab expectation.
- OSPF process and passive-interface policy match intent.
- Required OSPF neighbor remains `FULL`.
- The peer routing table contains the scenario prefix through the expected protocol and next hop.
- The defined reachability probe succeeds with acceptable loss and latency.
- No required baseline neighbor, route, interface, or service disappeared.
- CPU, memory, errors, drops, and logs remain within policy.
- The running configuration contains no unexpected drift.

Some checks need a convergence window. Poll with a bounded timeout and preserve intermediate observations. A fixed long sleep wastes time and hides convergence behavior.

4.2.11 pyATS and Genie validation

pyATS provides a test framework, while Genie parsers convert supported device output into structured data. A testbed file defines devices and connection details, with secrets supplied externally.

Illustrative test logic follows. Parser commands and supported structured output depend on the target network OS, pyATS/Genie release, and installed parser packages.

```
from pyats import aetest

class CommonSetup(aetest.CommonSetup):
    @aetest.subsection
    def connect(self, testbed):
        device = testbed.devices["distribution-01"]
        device.connect(log_stdout=False)
        self.parent.parameters["device"] = device

class VerifyRouting(aetest.Testcase):
    @aetest.test
    def ospf_neighbor_full(self, device):
        parsed = device.parse("show ip ospf neighbor")
        states = collect_neighbor_states(parsed)
        self.failed("Required OSPF neighbor is not FULL") if "FULL" not in states else self.passed()

    @aetest.test
    def branch_prefix_present(self, device):
        routes = device.parse("show ip route 10.20.120.0 255.255.255.0")
        assert_expected_ospf_route(routes, "10.20.120.0/24")
```

Production code needs robust structured traversal, explicit expected neighbor identity, meaningful failure evidence, and parser-error handling. Tests should distinguish unavailable parser support from an absent network state.

4.2.12 Applied failure scenario: configuration accepted, routing service fails

In the course reference scenario, the pipeline successfully creates VLAN 120 and its gateway on `distribution-01` and applies the approved OSPF intent. The device returns no configuration error, but post-checks show:

Vlan120	10.20.120.1	YES manual up	up
Required OSPF neighbor		EXSTART/BDR	
Expected route 10.20.120.0/24		absent on routing-peer-01	
Reachability to 10.20.120.1		failed from routing-peer-01	

Failure Scenario

A timeout after a configuration request creates an uncertain outcome: the device may have changed even though the client received no success response. The workflow must rediscover configured and operational state before retrying, rolling back, or declaring failure.

The pipeline must fail the release and stop promotion. Possible causes include MTU mismatch, authentication mismatch, network type mismatch, access policy, or an unrelated peer condition. Removing the newly advertised lab prefix may not fix an existing peer problem. The correct response is evidence-driven:

1. Compare the neighbor with the pre-check baseline.
2. Inspect OSPF and interface logs and telemetry.

3. Determine whether the change affected the adjacency.
4. Roll back when the change caused degradation and rollback is safe.
5. Otherwise preserve the change state, mark the environment unhealthy, and remediate the peer under a separate controlled action.
6. Run the full post-check suite again.

4.2.13 Deployment strategies

Application deployment names are useful only after translating them into network control mechanisms:

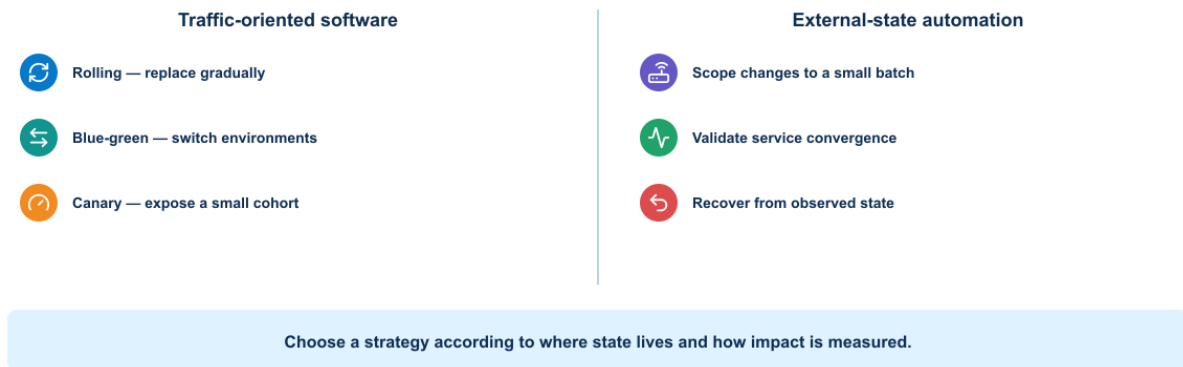


Figure 4-9. Comparison of traffic-oriented software deployment patterns and external-state automation controls

Table 4-8. Deployment Strategies and Network Automation Controls

Application pattern	Network interpretation	Network-specific gate
Rolling update	Serial device or bounded-batch rollout	Stop between batches on convergence, error-rate, or service policy
Blue-green	Parallel policy, VRF, controller object, or alternate path with controlled cutover	Prove state synchronization, route preference, and reversal path
Canary	One device, site, tenant, or maintenance domain first	Compare protocol, path, packet-loss, latency, and incident signals
Feature flag	Pre-stage configuration and activate a controlled policy/object later	Prevent stale dormant configuration and audit activation ownership
Recreate	Remove and replace a disposable lab service or virtual appliance	Rarely appropriate for a shared physical router or switch

Recreate

Stop the old version and start the new version. This is simple but normally creates interruption. It may suit a training or low-criticality environment.

Rolling update

Replace instances gradually while some old instances remain available. The application and schema must tolerate temporary version overlap.

Blue-green

Run old and new environments in parallel, validate the new environment, then switch traffic. Recovery can be fast if the old environment remains intact. The approach needs additional capacity and careful data handling.

Canary

Send a small portion of traffic to the new version and compare behavior before increasing exposure. Canary analysis needs reliable metrics and a clear decision policy.

Feature control

Deploy code with a feature disabled, then enable it for selected users or environments. This separates deployment from feature exposure but adds configuration lifecycle and cleanup work.

No strategy removes risk. Database changes, external side effects, stateful protocols, and long-running work need special handling.

4.2.14 Post-deployment validation

The validation layers introduced in Section 2.5 now run against the deployed release. They proceed from cheap technical checks to meaningful service behavior:

1. Confirm the expected artifact identity.
2. Confirm processes or workloads are ready.
3. Check application and dependency health.
4. Exercise a small functional transaction.
5. Review error rate, latency, saturation, and logs.
6. Confirm that rollback or remediation remains possible.

Synthetic transactions should use isolated test data and safe cleanup.

For a network change, replace an application-only [HTTP 200](#) test with layered evidence: confirm the stored configuration, interface state, protocol adjacency, expected and forbidden routes, next hop, forwarding path, loss and latency, device resources, new errors, and unaffected baseline services. The selected checks must derive from intent rather than from whatever commands are convenient to collect.

4.2.15 Smoke, integration, and acceptance checks

A smoke test answers whether basic critical behavior works. An integration check examines a component boundary. An acceptance test evaluates a user or business outcome.

The pipeline needs a compact set that completes quickly while detecting common release failures. Broader tests can run in the on-demand environment or on a schedule.

4.2.16 Idempotence and repeatability

An idempotent operation reaches the same intended state when repeated. It does not mean the operation performs no work or produces identical logs. Deployment scripts should inspect current state and change only what is required.

Repeatability matters when a job is retried after an uncertain failure. The workflow should avoid duplicating resources or corrupting state.

4.2.17 Failure handling

The pipeline should stop at the failing boundary and preserve evidence. Cleanup should remove disposable resources without hiding the original error.

Different failures need different responses:

- A deterministic test defect requires a source change.
- A transient registry timeout may justify a bounded retry.
- An authentication failure requires corrected access, not repeated attempts.
- A failed health check may require rollback or investigation.
- A partially applied infrastructure change requires state inspection before retry.

Practical failure: a healthy container with an incompatible dependency

Assume the new API container starts and its liveness probe passes, but workers fail when reading jobs created by the previous version. The deployment platform sees a running process; users see stalled automation. The post-deployment check must therefore submit a representative job and verify its terminal state, not merely call `/health`. If the database change is backward compatible, shift traffic back to the previous image and investigate. If the migration is irreversible, rolling back the image may make matters worse; stop promotion, preserve the queue and schema evidence, and use the documented forward-remediation path.

4.2.18 Rollback and remediation

The failure categories and `UNKNOWN/PARTIAL` state introduced earlier determine whether recovery can use rollback, forward remediation, or investigation. Recovery begins with causality and reversibility, not with an automatic rollback command.

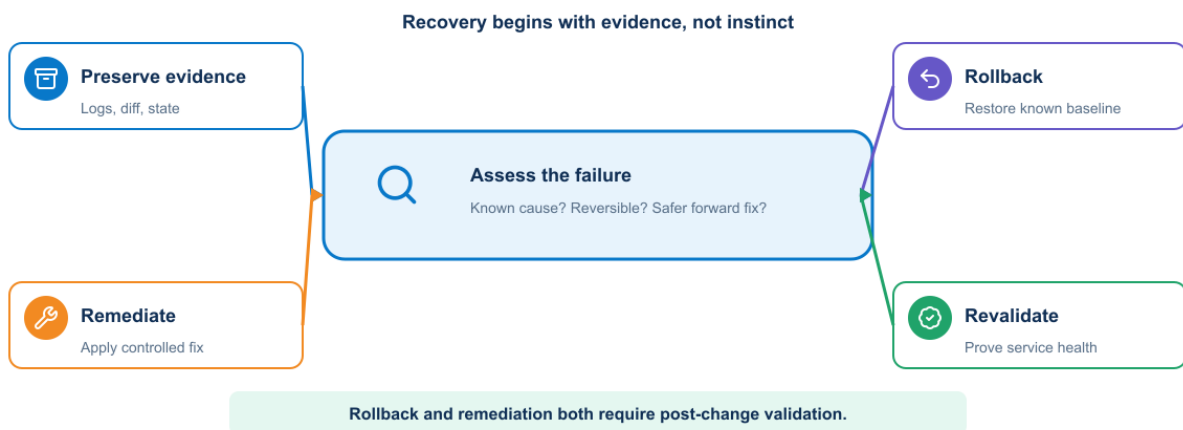


Figure 4-10. Decision tree for investigation, rollback, or forward remediation

Both recovery paths end in renewed validation; reversing commands is not itself proof of restored service.

Rollback restores a prior artifact or configuration. It works best for stateless application changes with compatible data. Some database or infrastructure changes cannot be reversed safely.

Network recovery mechanisms have different guarantees:

- **Checkpoint or configuration replace:** restores a known device snapshot, but may overwrite legitimate concurrent changes unless locking and scope are correct.
- **NETCONF confirmed commit:** automatically reverts an unconfirmed transaction when supported and correctly timed; capability discovery and session behavior matter.
- **Controller transaction rollback:** uses the controller's ownership and transaction model; direct device edits may create unmanaged divergence.
- **Inverse intent:** removes or reverses only the intended delta, but must be generated and tested like any other change.
- **Forward remediation:** preserves valid new state and applies a new corrective action when reversal would be unsafe or the failure is unrelated.
- **Manual recovery:** uses a tested runbook and preferably out-of-band access when automation identity, reachability, or evidence cannot be trusted.

Forward remediation applies a new corrective change. Teams often need both options. The release plan should define the trigger, responsible role, required evidence, and data implications.

Test recovery before an incident. An undocumented rollback command that no one has exercised is only a hypothesis.

4.2.19 Improved deployment flow

An improved flow uses an on-demand environment, immutable artifact, automatic health checks, and controlled promotion:

The improved flow validates the merge request, builds the image once, creates a test environment, deploys the image digest, runs system tests, collects evidence, and removes the test environment. Approval then promotes the same digest for final verification and observation.

4.3 Knowledge check

4.3.1 CI/CD delivery flow

Use these questions to test your understanding of pipeline structure, evidence flow, and the boundaries between build, test, and deployment.

1. How does an artifact differ from a cache?
2. Why should untrusted branches not use a privileged deployment runner?
3. Why should the pipeline build a release artifact only once?
4. Which checks should run before an expensive test environment is created?
5. Why is a masked variable insufficient protection against a malicious job?
6. How do runner scope, tags, protection, executor, and network policy control different parts of job execution?
7. When does `needs` improve feedback time, and why must network-change concurrency still be controlled separately?
8. What risk is introduced when a pipeline includes a mutable shared template?

4.3.2 Deployment validation and recovery

Use these questions to assess whether you can connect an approved artifact to controlled deployment, post-change evidence, and recovery decisions.

1. What information connects a post-deployment test to the source change it validates?
2. Why should a pipeline inspect environment health before changing it?
3. Which deployment strategy requires the application to tolerate old and new versions at the same time?
4. Why might database migration prevent a simple rollback?
5. When is retrying a failed operation unsafe?

4.4 Summary

CI/CD converts delivery policy into executable, purpose-specific workflows. Repository and source-of-truth events enter through different contracts; frozen intent, immutable artifacts, disposable test environments, independent validation, cleanup, and protected promotion build evidence before privileged production work. Terraform owns resource lifecycle, Ansible owns configuration, and operational tests determine whether the outcome matches intent. Pipeline success is not the final objective; delivery is complete only when the service works, evidence is retained, and temporary resources have been removed.

What the learner now has: a controlled delivery system that builds once, promotes by digest, separates software deployment from network-change authorization, and verifies operational outcomes.

What is still missing: continuous assurance that identities, trust boundaries, telemetry, and retained evidence remain dependable after deployment.

What the next module adds: Module 5 applies security and observability across the complete delivery and operating system. Continue to Security and Observability.

MODULE 5

5 Security and Observability

Central engineering question

Can we trust the system and explain what happened?

IDENTITY → CONTROL → TELEMETRY → RESPONSE

Learning objectives

- Define trust boundaries for source, runners, identities, artifacts, and targets.
- Correlate logs, metrics, traces, events, and deployment evidence.
- Use operational feedback to improve security, stability, and delivery.

Estimated theory time	3 hours
Lab connection	Apply the module controls to the cumulative application and delivery system
Core classroom topics	Concepts, decisions, workflows, and required controls
Advanced reference	Detailed implementation and platform considerations

5.1 Security across the delivery system

5.1.1 DevOps trust boundaries

The architecture separates repository-controlled validation from privileged deployment and preserves an independent audit path.

This view extends the Module 1 delivery architecture rather than replacing it: the same validation, artifact, approval, protected execution, and evidence components are shown with their trust boundaries and permitted crossings made explicit.



Figure 5-1. NetDevOps trust boundaries through protected execution and independent audit

Crossing a boundary requires authenticated identity, authorized purpose, encrypted transport, input validation, controlled output, and audit evidence.

5.1.2 Security throughout delivery

Security operates across the complete delivery lifetime.

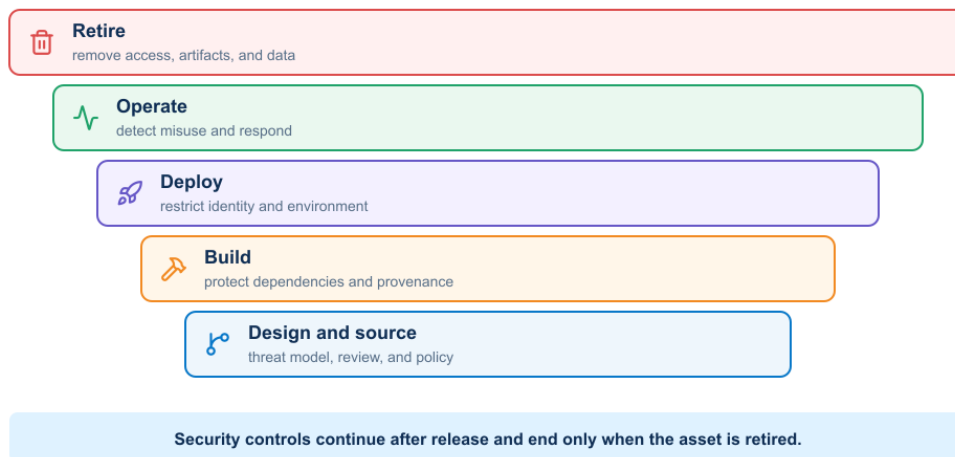


Figure 5-2. Layered security responsibilities from design and build through operation and retirement

The lifecycle controls include:

- Threat modelling identifies assets, trust boundaries, attackers, and misuse cases.
- Source controls protect branches, reviews, and repository access.
- Build controls protect dependencies, runners, and artifacts.
- Deployment controls restrict credentials and environments.
- Runtime controls limit privileges, network paths, and data access.
- Monitoring detects suspicious or unsafe behavior.

- Retirement removes credentials, artifacts, data, and infrastructure safely.

Moving a final security review earlier can find defects sooner, but early checks do not replace runtime protection and operational response.

5.1.3 Threat modelling a NetDevOps workflow

Typical network automation workflows contain several trust boundaries:

- Developer workstation to GitLab
- GitLab to runner
- Runner to registry and deployment environment
- Frontend to application
- Application to database
- Pipeline to Terraform, Ansible, Vault, Docker, and Kubernetes APIs
- Monitoring system to notification destination

For each boundary, identify authentication, authorization, encryption, input validation, logging, failure behavior, and credential scope.

Assets and threats

Security design begins by identifying what must be protected and how it could be misused or exposed. The table connects representative delivery assets with threats and the direction of an appropriate control.

Table 5-1. Delivery Assets Threats and Controls

Asset	Example threat	Control direction
Intended-state repository	Unauthorized VLAN, prefix, or routing-policy change	Protected branch, review, signed commits where required, policy checks
Pipeline definition	Job modified to extract secrets or bypass validation	Required ownership review and protected <code>main</code>
Runner	Host compromise or untrusted job execution	Dedicated runner, patching, isolation, restricted tags and network
Automation image	Dependency substitution or registry overwrite	Locked dependencies, scanning, digest pinning, signing, provenance
SSH key or API credential	Theft and reuse outside the pipeline	Short lifetime, scoped identity, source restrictions, rotation
Network management plane	Unauthorized access or lateral movement	Firewall, AAA, RBAC, protocol security, audit
Evidence	Exposure or alteration of configuration and topology	Encryption, access control, integrity, retention policy

5.1.4 Secrets management

Short-lived credentials have a lifecycle tied to workload identity and job scope: establish identity, authorize a specific purpose, issue the credential, use and audit it, and then revoke or allow it to expire. A secret is sensitive information used to authenticate or protect another asset. Examples include passwords, API tokens, private keys, signing keys, and database credentials.

Good secret handling includes:

- Store secrets outside source and image layers.
- Grant access to the smallest required identity and environment.
- Prefer short-lived credentials.
- Rotate and revoke credentials.

- Record access without recording secret values.
- Prevent command traces and debug output from exposing values.
- Scan repositories and images for accidental disclosure.

Vault can issue or store training credentials, while GitLab protected variables can supply selected jobs. Kubernetes Secrets provide an API object and distribution mechanism, but their confidentiality depends on cluster encryption, RBAC, node security, and application handling.

Vault is most valuable when the workload proves its identity and receives only the material required for its current purpose. A validation job should not obtain a production device password. An infrastructure job may need a CML API token but no production route. A production deployment job may need one device credential but no authority to administer Vault.

A mature design separates secret paths and policies for source-of-truth access, test-platform lifecycle, ephemeral test devices, production devices, registry and signing operations, and audit delivery. Prefer workload identity or GitLab OIDC-to-Vault authentication over reusable bootstrap credentials. Where AppRole is necessary, limit SecretID lifetime and uses, issue a narrowly scoped token, and keep token lifetime shorter than the job. Vault audit logs record access decisions, never returned values.

Network credential types

Automation workflows may require several forms of authentication material, each with different storage, rotation, and exposure risks. Common examples include:

- SSH private keys and known-hosts trust
- Device usernames and passwords
- TACACS+ or RADIUS-backed service identities
- NETCONF credentials
- RESTCONF basic, token, or certificate credentials
- Controller API tokens and session cookies
- Vault AppRole, OIDC, or workload identity credentials
- Registry and artifact-signing credentials

Host-key and TLS validation protect endpoint identity. A secret sent to an impostor is still compromised even when the secret itself is strong.

Prefer a dedicated automation identity with command authorization or API permissions limited to the intended configuration domain. A read-only collector should use a separate identity from the deployment worker.

5.1.5 Pipeline security

The runner trust zones introduced in Module 4 are now evaluated as security boundaries rather than repeated as pipeline mechanics.

The pipeline is a privileged software system. Threats include a malicious dependency, compromised runner, altered pipeline file, exposed variable, poisoned cache, substituted image, or unauthorized promotion.

Controls include:

- Protected branches and required review
- CODEOWNERS or designated review for pipeline and infrastructure files
- Isolated runners for trusted work
- Pinned dependencies and verified sources
- Immutable artifacts and digests
- SBOM generation and vulnerability scanning
- Artifact signing and provenance
- Protected environments and deployment approvals
- Least-privilege service identities

- Tamper-resistant audit records

Scan results need policy. The team should define which findings block a merge, which require review, and how a temporary exception expires.

Git security

Protect the default branch, require merge requests, restrict force pushes, and require successful pipelines. Changes to `.gitlab-ci.yml`, container definitions, credential integrations, inventory, and deployment roles deserve designated reviewers.

A reviewer should inspect semantic intent, not merely approve a familiar author. A small YAML change can advertise the wrong prefix to the entire routing domain.

Runner security

The protected runner should:

- Run only projects and branches explicitly allowed.
- Avoid shared use by untrusted workloads.
- Use ephemeral job execution where practical.
- Reach only required management endpoints and ports.
- Retrieve credentials just in time.
- Prevent jobs from reading other job workspaces or caches.
- Send audit and host logs to a separate system.
- Receive timely operating-system and runtime updates.
- Avoid exposing the Docker socket unless the architecture explicitly accepts host-level control.

Runner registration and authentication tokens are sensitive. Rotating device credentials does not repair a runner that remains compromised.

Security tests and their boundaries

Security testing is a set of complementary controls rather than one scanner. Each control inspects a different representation of the system and therefore misses different problems.

Table 5-2. Security Test Types and Evidence Boundaries

Control	Primary question	Useful network-automation target	Important limitation
Static application security testing	Does source contain a recognizable insecure code pattern?	Python API, worker, validation and credential-handling code	Cannot prove runtime exploitability or correct authorization design
Dependency scanning	Do declared third-party components match known vulnerabilities?	Python packages and Ansible collections where supported	Findings depend on accurate inventory and vulnerability data
Secret detection	Does repository content resemble a credential or private key?	Source, history, examples, playbooks, and pipeline files	A miss does not prove that no secret exists; a hit still requires revocation analysis
Container scanning	Do operating-system packages and other supported image contents contain known findings or policy violations?	Validation and deployment images	Coverage varies by scanner and image contents; it does not assess the host, runner, runtime identity, or management firewall
Infrastructure-as-Code scanning	Does a declarative resource violate a known security rule?	Terraform, Compose, and Kubernetes definitions	Cannot prove that deployed state matches the reviewed definition

Control	Primary question	Useful network-automation target	Important limitation
Dynamic application security testing	Does a running HTTP service expose detectable behavior?	Automation API in an isolated environment	Requires a safe target and does not examine device protocols or internal code paths
License policy	Are component licenses compatible with distribution policy?	Packaged libraries and container contents	Legal interpretation and exceptions still require organizational review

Run inexpensive source and dependency controls early, then apply runtime tests to an isolated deployed service. A finding must retain scanner version, rule, affected component, evidence, severity, exploit context, and artifact digest. Otherwise a later reviewer cannot determine whether a rebuilt image still contains the same problem.

Security results need a triage workflow. Confirm the component is present and reachable, determine whether the vulnerable path is used, assign an owner and deadline, and choose remediation, compensating control, or a time-limited accepted exception. Suppression without a reason and expiry date converts a visible risk into invisible debt.

Key Point

Passing every configured scanner does not establish that a release is secure. Scanners find selected known patterns; architecture review, least privilege, endpoint verification, runtime evidence, and incident readiness address risks outside those patterns.

5.1.6 Compromised-runner scenario

Contain access before rebuilding the execution environment. The affected period remains untrusted until job history and managed state are independently verified.



Figure 5-3. Compromised runner incident-response cycle from containment through independent verification

Assume an attacker gains code execution on the protected GitLab runner. The incident review must answer:

Table 5-3. Compromised Runner Response Actions and Evidence

Question	Architectural consequence	Control
What credentials can it access?	Static secrets can be copied and reused	Short-lived issuance, narrow scope, no shared administrator credential, rapid revocation
Which devices and ports can it reach?	Broad management routing enables lateral movement	Explicit egress allowlists, management firewall, separate worker zones

Question	Architectural consequence	Control
Can it modify approved configuration?	Cached artifacts or mutable tags may bypass review	Bind approval to commit, target, diff hash, and signed image digest
Can it falsify evidence?	Local logs cannot be trusted after host compromise	Stream AAA, runner, pipeline, and device evidence to an independent protected sink
Can it sign an image?	Online signing keys can legitimize malicious artifacts	Keyless/OIDC signing or isolated signer with policy and transparency evidence
Can it pivot into the management network?	A runner becomes a general-purpose foothold	No interactive user path, network segmentation, device command authorization, monitoring

Containment disables the runner, blocks new jobs, removes its network route, revokes runner registration and every reachable credential, and preserves forensic evidence. Recovery rebuilds from a trusted source, issues new identities, verifies device and repository state, and restores service under explicit approval.

No pipeline setting makes a privileged runner harmless. Architecture must assume that a boundary can fail.

5.1.7 Secure protocol use

Encryption alone does not make an automation channel trustworthy. Client configuration must also authenticate the endpoint, constrain credentials, handle errors safely, and preserve enough evidence for investigation.

SSH

Use modern algorithms supported by organizational policy, validate host keys, restrict service-account commands, and protect private keys. Do not automatically trust a new key during a deployment job.

NETCONF

NETCONF over SSH inherits SSH authentication and host identity requirements. Validate capabilities and RPC errors. Limit the account to required datastores or configuration domains where the platform supports it.

RESTCONF and controller APIs

Validate TLS certificates and hostnames. Limit tokens by scope and lifetime. Do not place tokens in URLs. Handle redirect, proxy, and debug logging behavior carefully. Rate-limit clients and distinguish authorization failure from a transient platform error.

5.1.8 Application security

The application should validate untrusted input, encode output for its context, use parameterized database operations, enforce authorization on the server, and protect sessions or tokens.

TLS protects data in transit when certificate identity and trust are validated. Disabling certificate verification may simplify a lab test but destroys authentication of the endpoint. Training environments should establish a safe trust path rather than normalize insecure flags.

Logs and error responses should provide enough context for support without exposing internal secrets or sensitive data.

5.1.9 Container and orchestrator security

Module 3 established the mechanics of building and running container images. This section revisits those mechanics as security boundaries: which source and base image can be trusted, what the workload may do at runtime, which identities it receives, and which systems it may reach.

The Kubernetes worker model from Module 3 is now evaluated through workload identity, RBAC, secret access, and restricted egress. These controls secure the platform; they do not merge the platform-release and network-change approval paths established in Module 4.

Container controls include trusted base images, non-root execution, removed capabilities, read-only filesystems, controlled mounts, resource limits, and restricted network access.

Kubernetes adds RBAC, service accounts, namespaces, network policy, security context, admission policy, secret handling, audit, and node security. A namespace is an organizational and policy boundary, but it is not a complete hostile-tenant isolation mechanism by itself.

The Kubernetes deployment worker should use a service account with only the required secret reference and job permissions. NetworkPolicy should permit the worker to reach approved management endpoints while blocking the API and dashboard services from direct device access.

5.1.10 Advanced architecture reference

Sections 2.10–2.16 are optional architecture reference material. They may be skipped during the main classroom path, which continues at Section 3 with observability.

Modern application architecture

A modern application often separates user interface, APIs, background processing, data services, and platform integrations. It may use containers and managed services, but the architecture should follow requirements rather than fashion.

Important qualities include:

- Clear component responsibilities
- Explicit API and data contracts
- Replaceable runtime instances
- External configuration
- Automated deployment and recovery
- Operational instrumentation
- Security boundaries
- Tolerance of expected dependency failure

The twelve-factor application principles provide useful guidance for configuration, backing services, build and run separation, disposable processes, environment parity, logs, and administrative tasks. Teams should apply the principles according to the system's actual needs.

Microservices

Microservices divide a system into independently deployable services aligned with bounded responsibilities. Potential benefits include independent release, targeted scaling, fault isolation, and team ownership.

Costs include distributed transactions, network failure, version compatibility, more deployment units, more telemetry, duplicated platform work, and harder local testing. A modular monolith may offer a better starting point when independent scale or release is not required.

Service boundaries should follow ownership and data behavior. Splitting code into containers without independent responsibility creates a distributed monolith.

Synchronous and asynchronous interaction

Synchronous APIs provide immediate responses but couple availability and latency between services. Asynchronous messaging can absorb bursts and reduce temporal coupling, but it introduces eventual consistency, duplicate delivery, ordering questions, and message lifecycle management.

Consumers should handle duplicate messages safely. Producers and consumers need compatible schema evolution. Operational evidence should expose queue delay and failed-message behavior.

Public, private, and hybrid deployments

A private environment can provide direct control, locality, and integration with existing systems. Public cloud can provide rapid provisioning, managed services, geographic options, and consumption-based scaling. Neither model is inherently safer or cheaper in every case.

A mixed deployment may be described as hybrid or multicloud depending on whether it combines private and public environments or uses services from multiple public providers. The design should state the actual placement and dependency model rather than rely on the label.

Multicloud design considerations

Evaluate:

- Application and data portability
- Identity federation and authorization
- Network connectivity, latency, and failure domains
- Data residency and regulatory obligations
- Key and secret management
- Logging and operational visibility
- Service differences and provider-specific APIs
- Cost, quotas, and egress charges
- Backup, recovery, and exit strategy
- Team skills and operating complexity

Using only common-denominator services can reduce provider dependence but may sacrifice valuable managed capabilities. An abstraction layer also becomes software that the team must own.

Network platform role

Networking, security, observability, and controller platforms can connect application and infrastructure domains. APIs and policy systems can participate in controlled workflows, while telemetry can supply network context for application behavior.

Automation must respect controller ownership and transactional behavior. Direct device changes that bypass the system of record can create conflict and drift.

Architecture decision records

An architecture decision record captures the decision, context, considered options, consequences, and status. Useful project decisions include:

- Why the lab uses one evolving repository for reusable learning artifacts
- Why the application begins with Compose and later uses Kubernetes
- Which tool owns infrastructure and which owns configuration
- How secrets reach pipeline jobs
- Which artifact identity is promoted
- Which telemetry signals control rollback or alerting

Recording decisions prevents future maintainers from treating deliberate constraints as accidental choices.

Practical control chain for a protected deployment

A merge request that changes `.gitlab-ci.yml`, deployment policy, or secret-retrieval code requires review from the platform or security owner. After merge, an unprivileged runner builds and scans the image without a route to managed infrastructure. A protected job exchanges its workload identity for a short-lived credential, verifies the approved image digest, runs on a restricted runner, and loses the credential when the job ends. Repository approval, artifact signature, workload identity, network segmentation, and audit logging protect different boundaries; none is a substitute for the others.

This design also narrows incident response. If the general runner is compromised, revoke its registry and source access without rotating every device credential. If the protected runner is compromised, stop deployment jobs, revoke the workload identity, preserve runner and secret-service audit logs, and treat any job executed during the exposure window as untrusted.

5.2 Observability and stability engineering

5.2.1 Monitoring, observability, and telemetry

Telemetry supplies data; monitoring evaluates known conditions; observability combines signals and context to explain unfamiliar behavior.

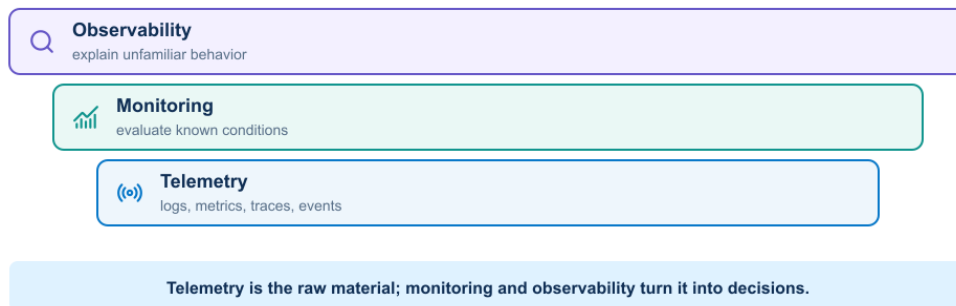


Figure 5-4. Relationship between telemetry, monitoring, observability, context, and action

Table 5-4. Telemetry Monitoring and Observability Comparison

Term	Meaning	Network example
Monitoring	Evaluate known conditions against expected behavior	Alert when a required OSPF neighbor is not FULL for five minutes
Observability	Ability to explain internal behavior from available outputs	Correlate a route loss with interface errors, a config change, and OSPF events
Telemetry	Data emitted or collected from systems	Interface counters, YANG-modeled streams, syslog, traces, and job metrics

Telemetry is the data. Monitoring evaluates selected signals. Observability is a property of the complete system, including instrumentation, context, retention, and investigation workflows.

5.2.2 Feedback architecture requirements

Device signals, application signals, and deployment events need a common correlation path. This expands the feedback path in the Module 1 delivery architecture rather than creating a separate monitoring destination.

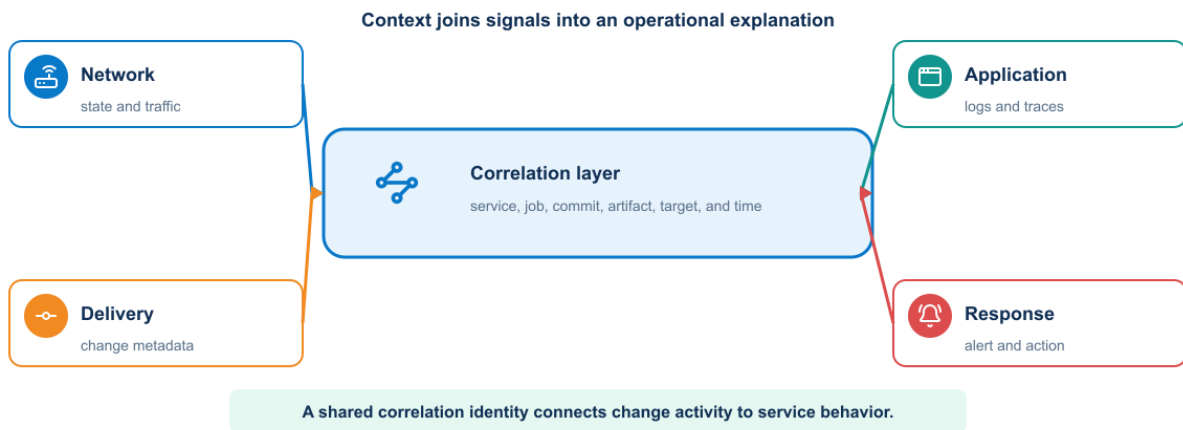


Figure 5-5. Observability architecture joining network, application, and pipeline signals

Shared identifiers and timestamps allow several storage systems to present one operational narrative.

Every record should carry enough dimensions to identify environment, site, device, interface or protocol instance, collection method, and time. Change and pipeline identifiers connect delivery events to operational effects.

5.2.3 Monitoring and observability

Monitoring evaluates known conditions. It asks questions such as whether an endpoint is reachable, error rate exceeds a threshold, or disk space is low.

Observability describes how well a team can understand internal behavior from system outputs. It supports investigation of conditions that the team did not predict in advance.

The two reinforce each other. Monitoring detects important known failures. Rich, connected telemetry helps explain them.

5.2.4 Operational signals

Observability draws on several complementary forms of evidence. Metrics reveal patterns, logs preserve discrete events, traces connect work across components, and external checks confirm the service outcome visible to a consumer.

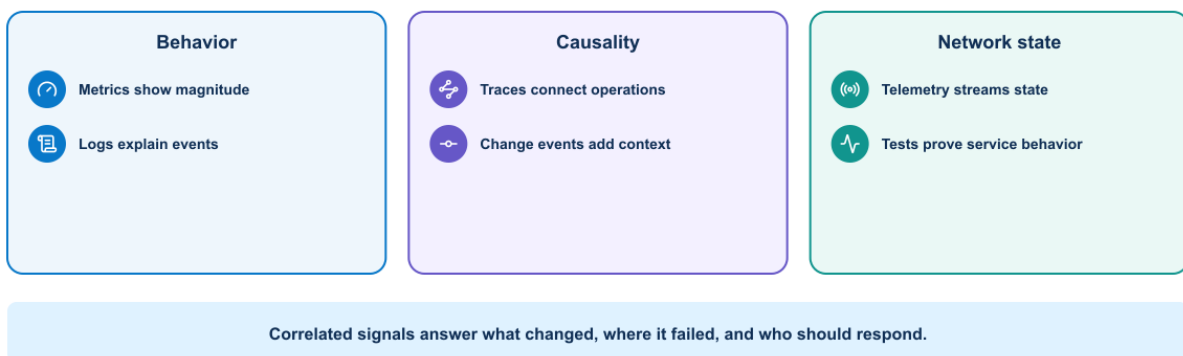


Figure 5-6. Operational signals grouped by behavior, causality, and network state

Metrics

Metrics are numeric measurements associated with time and labels. They support aggregation, comparison, trends, dashboards, and alerts. Common application signals include request rate, error rate, latency, queue depth, resource use, and dependency behavior.

Labels must remain controlled. User identifiers, request identifiers, or arbitrary URLs can create excessive cardinality and storage cost.

Logs

Logs record discrete events. Structured logs make fields searchable and reduce parsing ambiguity. A useful application event may contain timestamp, severity, service, version, environment, event name, and correlation identifier.

Logs should provide diagnostic context without exposing credentials, session tokens, personal information, or sensitive payloads.

Traces

Distributed traces follow a request across service boundaries. Spans describe work performed by each component and show timing, errors, and relationships. Trace and correlation identifiers can connect traces with logs.

Events and changes

Deployment, configuration, scaling, and infrastructure events add essential context. A dashboard should make it possible to compare a behavior change with a deployment or platform event.

5.2.5 Network data collection methods

No collection method supplies every signal. The following sections distinguish event streams, counters, polled state, modeled subscriptions, and application instrumentation so that each is used for evidence it can actually provide.

Syslog

Syslog provides event-oriented messages from network devices. It is valuable for interface transitions, routing changes, authentication events, configuration actions, and system faults. Configure accurate time, consistent severity policy, protected transport where supported, and centralized retention.

Text varies by platform and release. Parse important messages into structured fields while preserving the original record. Do not treat absence of a syslog message as proof that a condition did not occur.

SNMP

SNMP remains useful for widely supported counters and status. Prefer SNMPv3 with authentication and privacy when available. Counter semantics, polling interval, rollover, discontinuity, and interface identity affect interpretation.

Polling every interface at a very short interval can load devices and collectors. Select intervals based on the operational question.

REST API and CLI polling

Controller APIs, RESTCONF, NETCONF, and structured CLI collection can answer targeted questions. Polling offers explicit control but consumes management-plane capacity and produces snapshots. Apply timeouts, rate limits, and caching where appropriate.

Model-driven telemetry

Model-driven telemetry streams YANG-addressed data using a supported transport and encoding. Dial-out has the device initiate a subscription toward a collector. Dial-in has the collector establish and manage the subscription.

Before deployment, confirm the network operating-system release, model path, subscription mode, encoding, transport, update policy, and receiver compatibility. A syntactically valid sensor path can still produce no data if the platform does not support it operationally.

OpenTelemetry

OpenTelemetry provides common APIs, SDKs, semantic conventions, and collector components for application metrics, logs, and traces. It is particularly useful for the automation API and workers. Device telemetry does not automatically become OpenTelemetry data; a collector or translation layer may normalize network observations into the chosen model.

Use a trace to follow a job from API request to queue, worker, credential lookup, device RPC, validation, and evidence write. Do not include secret values or full configurations in span attributes.

5.2.6 Health checks

Health checks serve different consumers:

- Liveness tells the platform whether the process is stuck and should restart.
- Readiness tells the load balancer whether the instance can receive traffic.
- Startup health gives a slow application time to initialize before ongoing checks begin.
- Synthetic health exercises a user-visible behavior from outside the service.

Checks should be fast, deterministic, and inexpensive. A check that always returns success protects nothing. A check that depends on every external system may create false restarts.

5.2.7 Service objectives

A service-level indicator measures behavior that matters to consumers, such as successful request ratio or response latency. A service-level objective defines the desired target over a period.

Error budget is the allowed amount of unreliability within the objective. It provides a way to balance feature delivery and reliability work. When the service consumes the budget too quickly, the team may slow releases and address stability.

An internal component metric can help diagnosis but does not automatically represent user experience.

5.2.8 Metrics collection architecture

A metrics system usually contains:

- Instrumented application or exporter
- Collection agent or scraper
- Time-series storage
- Query and visualization layer
- Alert evaluation and notification path

The scenario environment exports automation-service metrics and collects selected device or simulated data. Useful measures include:

Table 5-5. Metrics Collection Components and Responsibilities

Domain	Metrics
Interfaces	Utilization, errors, discards, drops, status transitions
Routing	OSPF/BGP neighbor state, route count, convergence time, flap count
Device resources	CPU, memory, environmental state, process health
Service path	Latency, packet loss, reachability, DNS or application probe result
Device API	Connection time, RPC duration, HTTP status, timeout and rate-limit count

Domain	Metrics
Automation	Queue depth, job duration, success rate, retry count, devices changed
Delivery	Pipeline duration, gate failures, rollback rate, evidence completeness

5.2.9 Log collection architecture

Application and platform logs flow through a collector or agent into searchable storage and a visualization interface. Traditional ELK terminology refers to Elasticsearch, Logstash, and Kibana, although modern deployments may use other shippers and compatible storage components.

The architecture must account for parsing, buffering, backpressure, retention, access control, time synchronization, and index or field design.

If the logging platform fails, the application should avoid blocking indefinitely. It may buffer a controlled amount, degrade logging, or use local output collected by the platform.

5.2.10 Dashboards

A dashboard should answer an operational question. A service overview might show traffic, failures, latency, saturation, dependency health, and recent deployments.

Avoid displaying many unrelated metrics without context. Provide units, useful time ranges, thresholds, environment and version filters, and links to relevant logs or traces.

Different audiences need different views. A service owner needs diagnostic detail, while a course demonstration dashboard should clearly show the effect of a deployment or failure.

Network change dashboard scenario

For a routing-service example, a dashboard can display:

- Current service-interface state and most recent transition
- Required OSPF neighbor state and flap history
- Presence and next hop of the expected lab prefix at the test peer
- Reachability loss, latency, and packet loss
- Device CPU and memory during the change window
- Interface errors and drops on the path
- Automation job duration and result
- Git commit, pipeline, change ID, image digest, and deployment timestamp
- Relevant syslog events and links to protected evidence

The dashboard should distinguish `no data` from zero and identify whether a signal comes from a real device, simulator, or mock.

5.2.11 Alert design

An alert should indicate a condition that requires timely action. It needs:

- A meaningful signal connected to impact or impending impact
- A threshold and duration that limit transient noise
- Clear environment and service identity
- Severity and ownership
- A concise description
- A runbook or first diagnostic steps
- Notification routing and escalation

Alerting on symptoms such as sustained user-facing failures is often more actionable than alerting on every low-level fluctuation.

Example routing alert:

```

Condition: required routing neighbor for distribution-01 is not healthy for 5 minutes
AND service reachability probe fails
Severity: high
Context: device, neighbor, last known state, change ID, latest deployment
Owner: network operations
Runbook: verify management reachability, link state, MTU, authentication, logs, and recent
diff

```

Combining state and impact can reduce noise, but the team may still retain a lower-severity event for a neighbor transition that recovers quickly.

5.2.12 Webhook notifications

Alert systems can notify a webhook listener, collaboration platform, or incident-management system. Protect the destination token, validate TLS, restrict message content, and handle delivery failure.

A notification should include what failed, where, when, current value, relevant threshold, and a link to evidence. It should not include secrets or a full sensitive log record.

5.2.13 Application instrumentation

Instrumentation should begin with a small, stable set of signals:

- Request count by route class, method, and result class
- Request duration distribution
- In-progress work
- Dependency failures and latency
- Build version and environment information

Business or workflow metrics can show whether the service produces its intended outcome. Their meaning and privacy requirements must be documented.

For the automation platform, instrument request and job count, queue delay, device connection duration, RPC or command duration, validation failure category, configuration lines changed, rollback outcome, and evidence-write result. Never use a device password, token, full command output, or unbounded job identifier as a metric label.

5.2.14 Change-aware observability and correlation

An investigation follows commit SHA → pipeline ID → image digest → deployment event → automation job → change ID → target device → protocol or forwarding behavior → telemetry → alert → investigation → feedback. Correlation narrows the search; it does not by itself prove causality. The objective is not merely to detect that a routing neighbor or application process is unhealthy. It is to determine whether behavior changed during or after a particular software release or approved network operation, and to assemble enough evidence to evaluate that relationship.

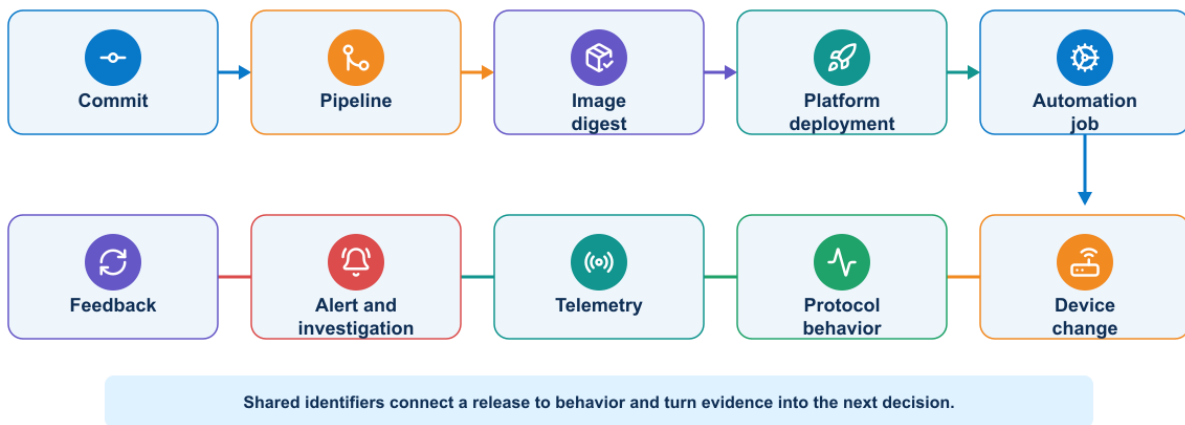


Figure 5-7. Change-aware feedback from commit and image digest through deployment, network behavior, telemetry, investigation, and improvement

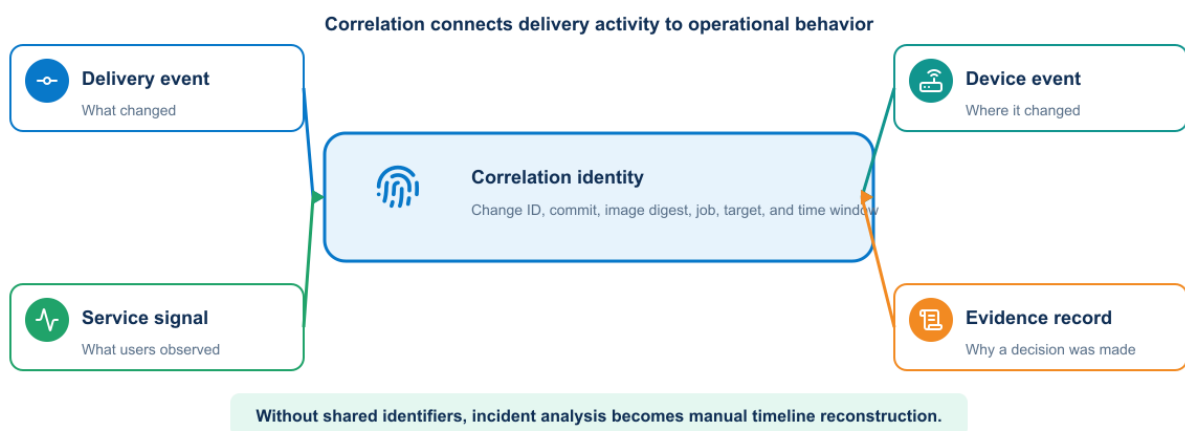


Figure 5-8. Correlation model joining delivery events, device events, service signals, and evidence

Correlation turns separate data into a delivery feedback loop:

For example, a commit starts a pipeline with a recorded automation image digest and change identifier. The deployment timestamp, worker activity, device event, and acceptance result form one ordered timeline. The exact network signals depend on the operation being delivered.

The pipeline emits a change event before and after deployment. Dashboards annotate the event, and evidence records the telemetry window. This supports both successful convergence analysis and failure investigation.

A useful correlation view should let an engineer answer, without manually joining several systems:

- What changed, who approved it, and which exact commit and image executed?
- Which devices, interfaces, routing processes, tenants, or controller objects were touched?
- Did the routing protocol converge within the expected window?
- Did route selection or the forwarding path change beyond the approved scope?
- Did packet loss, latency, drops, or errors increase?
- Which pipeline introduced the change, and did later jobs touch the same domain?
- Was the automation platform healthy, or did queue delay, worker failure, clock skew, or missing telemetry distort the result?

Join records with shared identifiers: `commit`, `pipeline`, `digest`, `change_id`, `target`, `environment`, and `timestamp`. These values belong in event, log, trace, and evidence records; only bounded dimensions should become metric labels. Credentials, full command output, and unbounded request strings are never metric labels.

Pipeline audit events

Console logs help diagnosis but are weak as the only audit record. They are formatted for humans, vary by tool, may be truncated, and often require broad CI-platform access. A structured audit event should be sent at job start and completion and at each privileged or decision-bearing task.

Table 5-6. Pipeline Audit Events and Required Context

Context	Useful audit fields
Source	Project, ref, commit, pipeline source, intent identifier and fingerprint
Actor	Trigger identity, job identity, runner identity and production approver
Execution	Pipeline, job, stage, action, timestamps, duration and exit status
Artifact	Image, immutable digest, provenance reference and plan fingerprint
Environment	Test or production, resource owner, environment identifier and cleanup status
Target	Device identity, endpoint, interface, requested prefix and authorized scope
Result	Changed status, assertion, observed state, outcome and sanitized error category

Audit every material transition: intent resolution, policy decision, resource plan, resource creation, endpoint readiness, configuration task, validation assertion, environment deletion, approval, production change, and final verification. A shared schema allows one timeline to join Terraform, Ansible, pyATS, GitLab, Vault, NetBox, Kubernetes, and application events.

More detail does not justify secret collection. Never forward tokens, passwords, authorization headers, private keys, Vault responses, complete configurations, unrestricted environment dumps, or raw command output without classification and filtering. The audit path should fail visibly; loss of required evidence can be a release-blocking condition for privileged work.

Practical incident trace

At 10:04 a deployment finishes, at 10:05 queue delay rises, and at 10:06 the first job times out. CPU and memory are normal. A structured worker log shows `dependency=job-db`, `error=connection_pool_exhausted`, together with the image digest and pipeline ID. The team can now separate an application-release problem from device reachability. The alert should point to the correlated timeline and runbook; it should not page merely because one request was slow.

```
{
  "timestamp": "2026-09-11T10:06:14Z",
  "service": "worker",
  "release": "sha256:7ab...",
  "pipeline_id": "1842",
  "change_id": "CHG-2026-0042",
  "dependency": "job-db",
  "outcome": "timeout",
  "duration_ms": 5000
}
```

The same values do not all belong in metric labels. `service`, `outcome`, and a bounded dependency name are useful dimensions. A unique change identifier belongs in logs or traces, because using it as a time-series label creates unbounded cardinality.

5.2.15 Telemetry quality

Operational data can be missing, delayed, duplicated, mislabelled, or out of order. Collection success does not guarantee correct interpretation.

Time synchronization matters across application hosts, containers, network devices, and pipeline systems. Retention and sampling policies should preserve enough evidence for expected investigations.

5.2.16 Stability engineering

Reliability improves when the design anticipates failure:

- Timeouts prevent indefinite waits.
- Retries address selected transient faults.
- Circuit breakers limit repeated calls to a failing dependency.
- Bulkheads isolate resource pools.
- Backpressure prevents uncontrolled queues.
- Rate limits protect shared capacity.
- Graceful degradation preserves essential behavior.

Every mechanism has tradeoffs. Retries increase load. Circuit breakers can reject work after a dependency recovers until their state changes. Operational signals must show what the mechanism is doing.

5.2.17 Chaos engineering

Chaos engineering uses controlled experiments to test a specific resilience hypothesis. It is not random destruction.

A responsible experiment defines:

1. Expected steady behavior.
2. A limited failure condition.
3. Scope and safety boundaries.
4. Abort conditions.
5. Observations and success criteria.
6. Recovery and cleanup.

In a training environment, deleting one disposable application instance or temporarily blocking one dependency can demonstrate self-healing or alert behavior. The experiment must stay within the assigned environment.

5.3 Knowledge check

5.3.1 Security across the delivery system

Use these questions to test whether you can apply least privilege, secret protection, trust verification, and architectural isolation to a delivery workflow.

1. Why does masking a pipeline variable not provide complete secret protection?
2. Which repository changes deserve stricter ownership or review?
3. What costs accompany a microservices architecture?
4. Which factors influence workload placement across private and public environments?
5. Why should the team document tool ownership of infrastructure settings?
6. Why do SAST, dependency scanning, container scanning, and DAST provide different evidence?
7. Which information must accompany a temporary vulnerability exception?

5.3.2 Observability and stability engineering

Use these questions to verify that you can turn operational signals into service understanding and actionable delivery feedback.

1. How does observability differ from monitoring?
2. Why should request identifiers not become unrestricted metric labels?
3. What is the difference between liveness and readiness?
4. Which information makes an alert actionable?
5. What separates a chaos experiment from uncontrolled failure injection?

5.4 Summary

Security and observability form the operating control plane of DevOps. Vault policies separate source-of-truth, test-platform, test-device, production, registry, and audit privileges. Protected source, identified artifacts, isolated runners, short-lived credentials, restricted paths, runtime hardening, and independent audit reduce misuse. Structured events join NetBox intent, GitLab execution, Terraform resources, Ansible tasks, validation, approval, production outcome, and cleanup into one defensible timeline. Together they return trustworthy operational evidence to the next controlled decision.

What the learner now has: a production-style delivery system with protected trust boundaries and change-aware operational evidence.

What is still missing: nothing in the course control chain; further work is environment-specific hardening, scale, and operational maturity.

What the next module adds: there is no further theory module; the final capstone uses trustworthy release and network-change feedback to drive the next controlled improvement.